

第1次实验 数据编解码电路设计

1.实验环境

Windows系统下运行Logisim软件（需安装JDK）。

2.实验目的

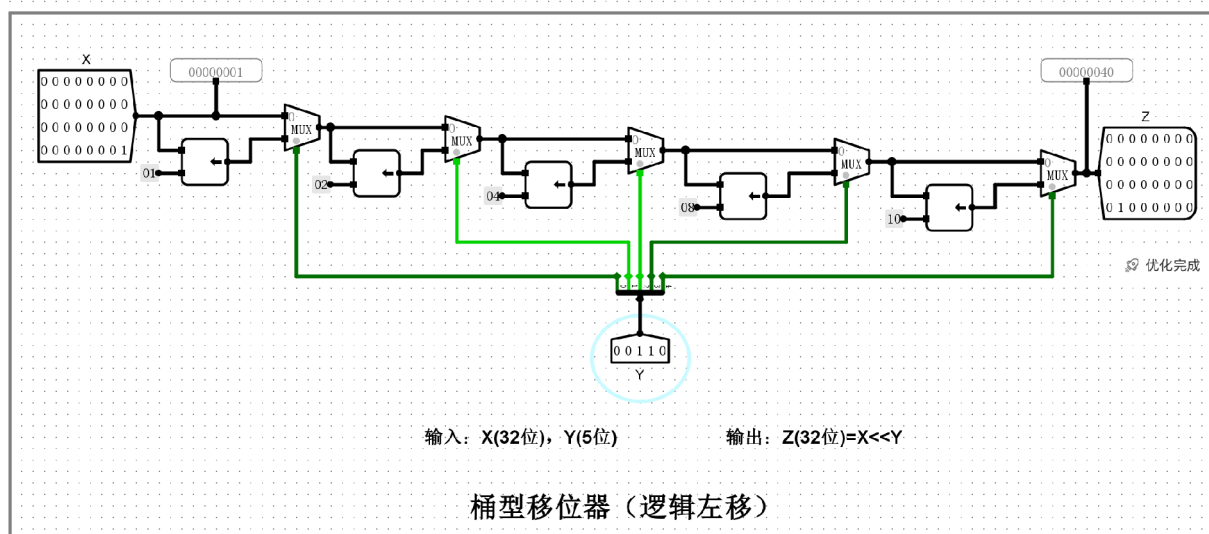
- (1)验证桶型移位器、4位快速加法器、16位快速加法器（组内并行、组间串行）、5位无符号阵列乘法器（斜向、横向）、8位无符号一位乘法器、8位补码一位乘法器电路。
- (2)设计16位快速加法器（组内并行、组间并行）、32位快速加法器（组内并行、组间串行）、6位原码阵列乘法器、6位补码阵列乘法器、8位原码一位乘法器、8位补码一位乘法器（采用8位无符号一位乘法器实现）。

3.1验证性实验

(1)桶型移位器

实验结果

逻辑左移



实验分析

1. 电路原理与结构分析

桶型移位器用于实现32位数据的逻辑左移运算。电路核心由五级多路选择器（MUX）级联组成，每级由位权控制变量 Y 的其中一位进行驱动：

- **分级控制逻辑**：输入变量 Y (5位) 的每一位 Y_i 决定了该级是否进行 2^i 位的固定偏移。
 - 第一级： Y_0 控制是否左移 1 位。
 - 第二级： Y_1 控制是否左移 2 位。
 - 第三级： Y_2 控制是否左移 4 位。
 - 第四级： Y_3 控制是否左移 8 位。
 - 第五级： Y_4 控制是否左移 16 位。
- **逻辑左移特性**：在各级移位过程中，低位空出部分统一补 0，高位溢出部分舍弃。

2. 实例运行分析

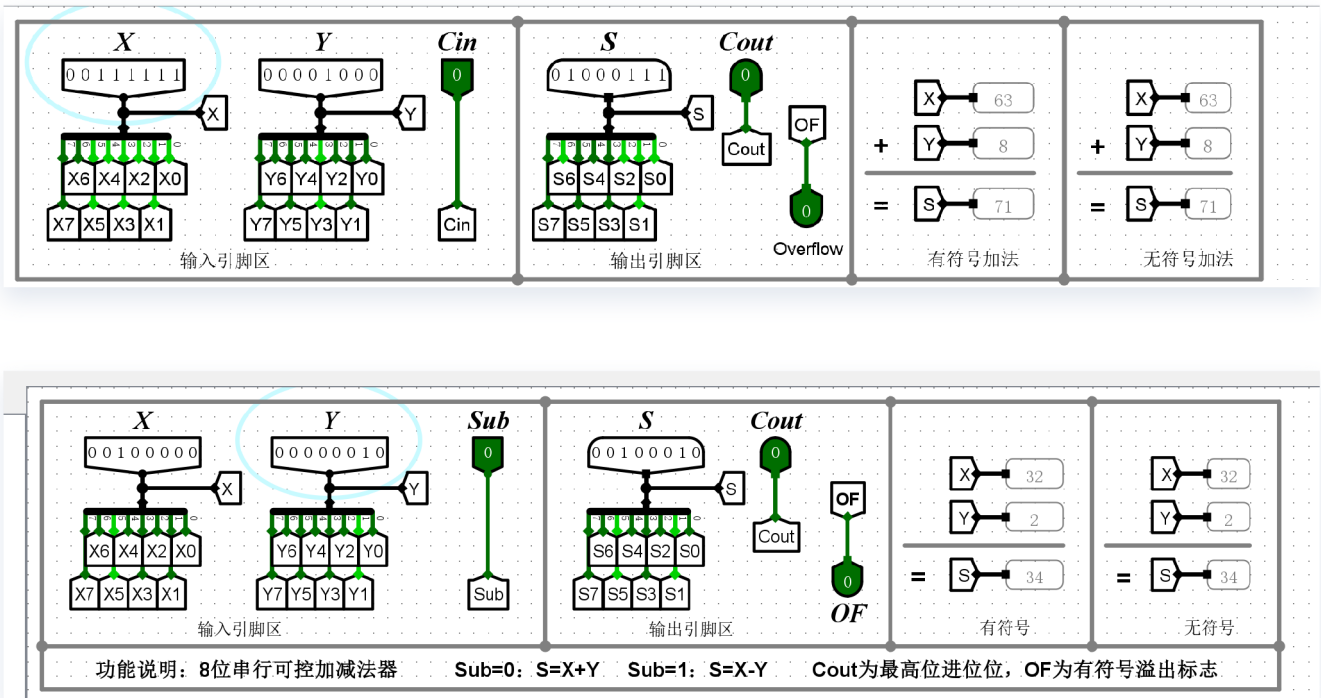
- **输入数据 X** ：00000001（十六进制），即二进制最低位为 1。
- **移位控制 Y** ：输入为 00110（二进制）。
 - 解析位权： $Y_4 = 0, Y_3 = 0, Y_2 = 1, Y_1 = 1, Y_0 = 0$ 。
 - 计算移动总位数： $Shift = 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 4 + 2 = 6$ 位。
- **电路内部路径**：
 1. 第一级 ($Y_0 = 0$)：选择原始路径，不移位。
 2. 第二级 ($Y_1 = 1$)：选择移位路径，左移 2 位。
 3. 第三级 ($Y_2 = 1$)：选择移位路径，左移 4 位。
 4. 第四、五级 ($Y_3 = 0, Y_4 = 0$)：选择原始路径，不移位。
- **输出结果 Z** ：
 - 理论计算： $Z = X \ll 6$ 。
 - $X = 1$ （二进制 ...000001）左移 6 位后变为二进制 ...1000000。
 - 对应十六进制结果为 00000040。
 - **仿真验证**：截图显示 Z 输出端显示为 00000040，与理论推导完全一致。

(2)8位串行加法器/8位串行可控加减法器

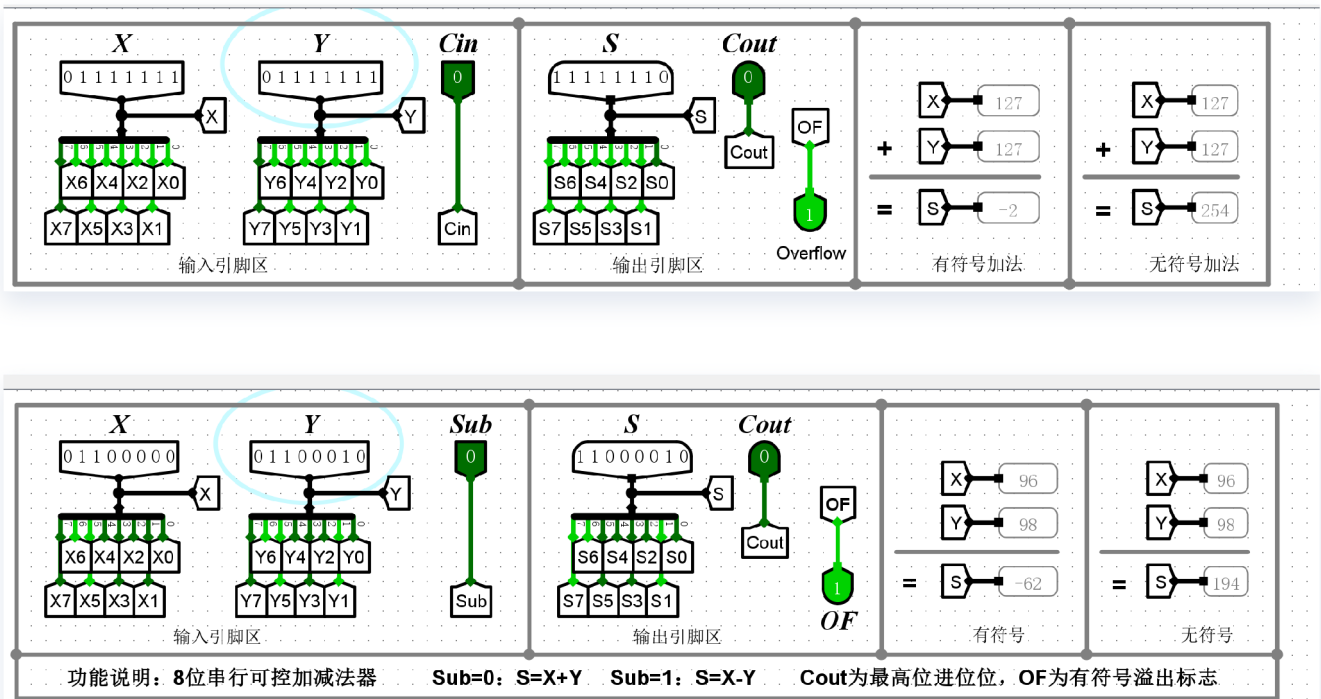
实验结果

加法器

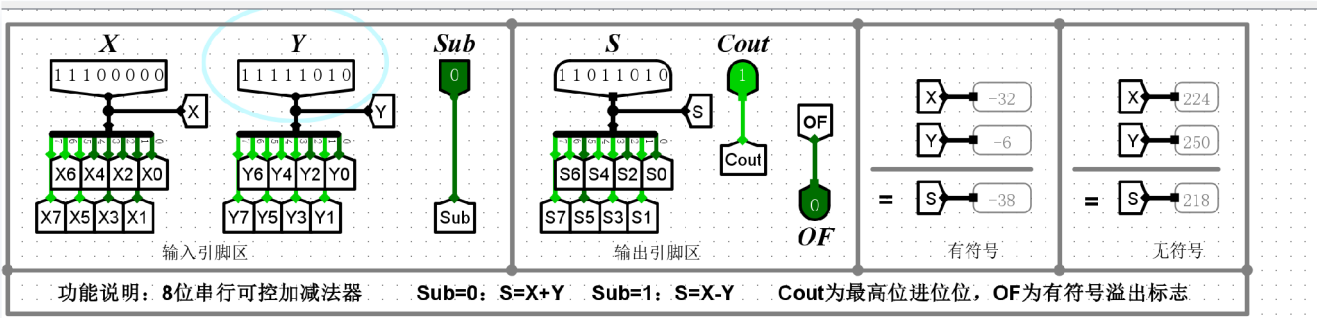
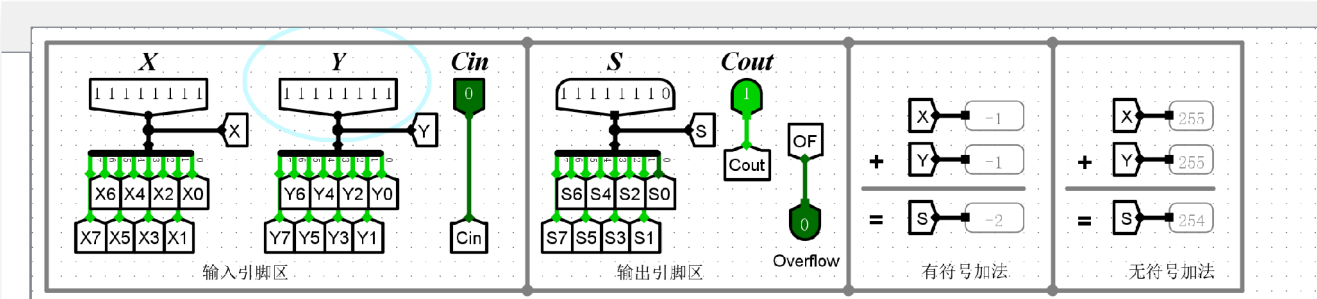
① 正数+正数=正数（不溢出）。



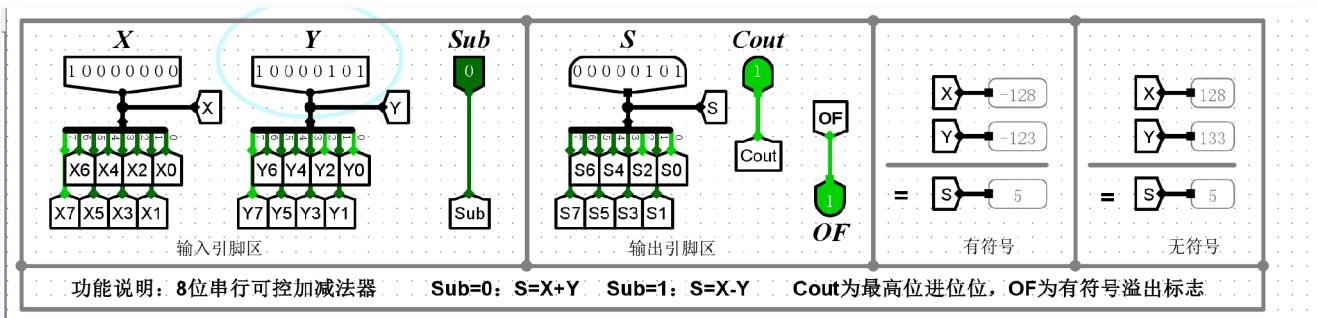
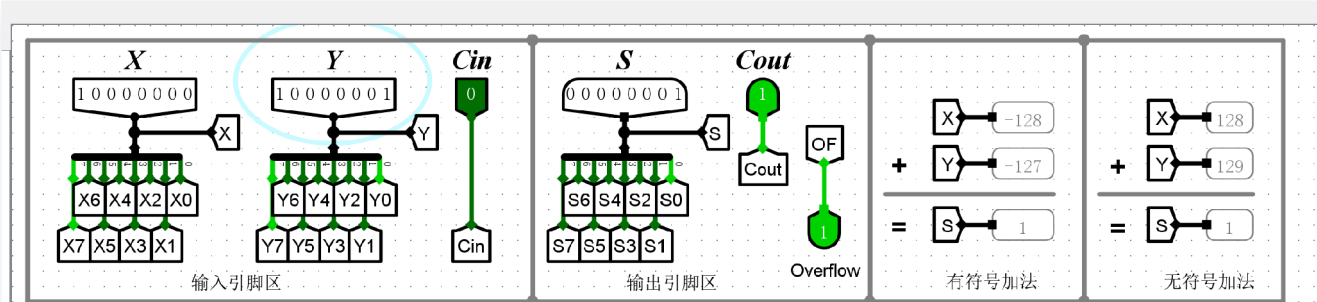
② 正数+正数=负数（溢出）。



③ 负数+负数=负数（不溢出）。

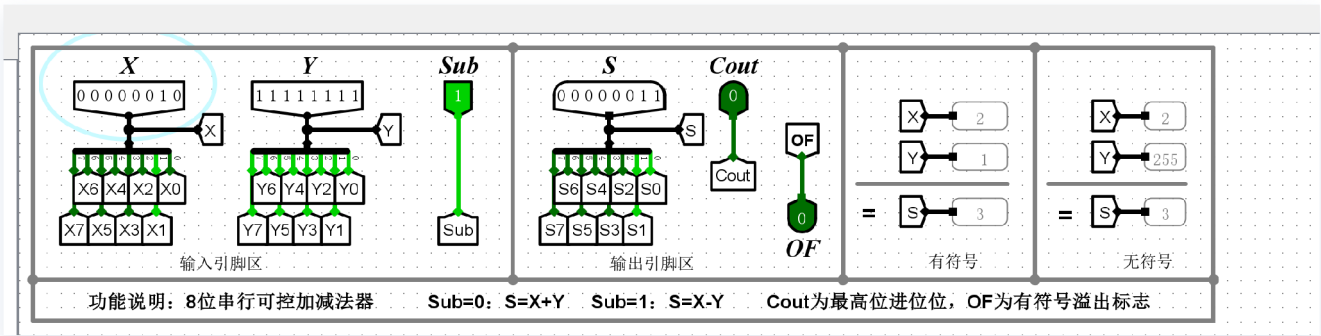


④ 负数+负数=正数（溢出）。

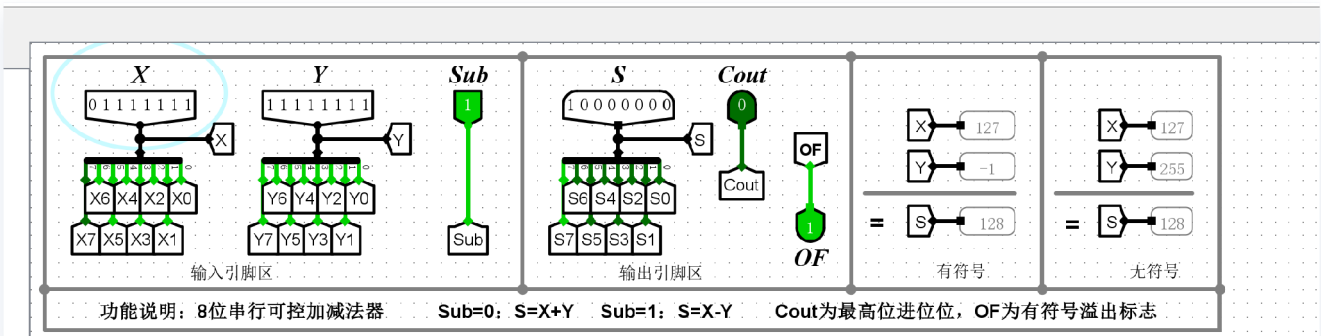


减法器

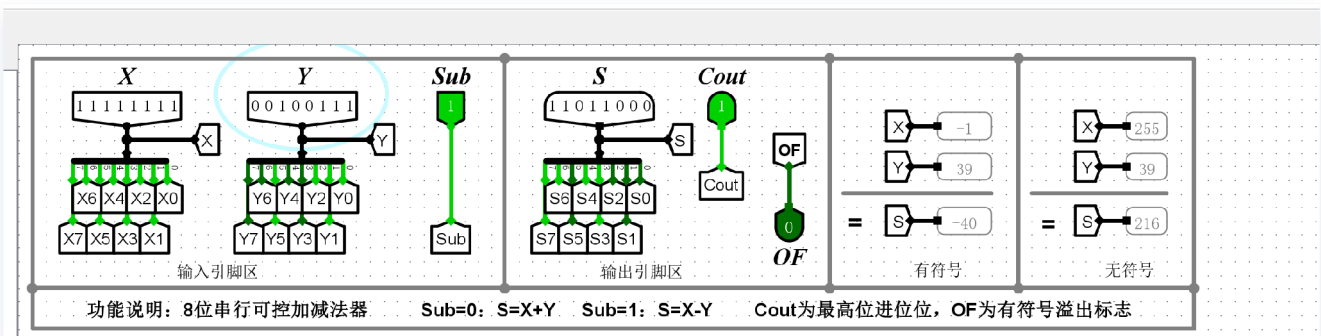
① 正数-负数=正数（不溢出）。



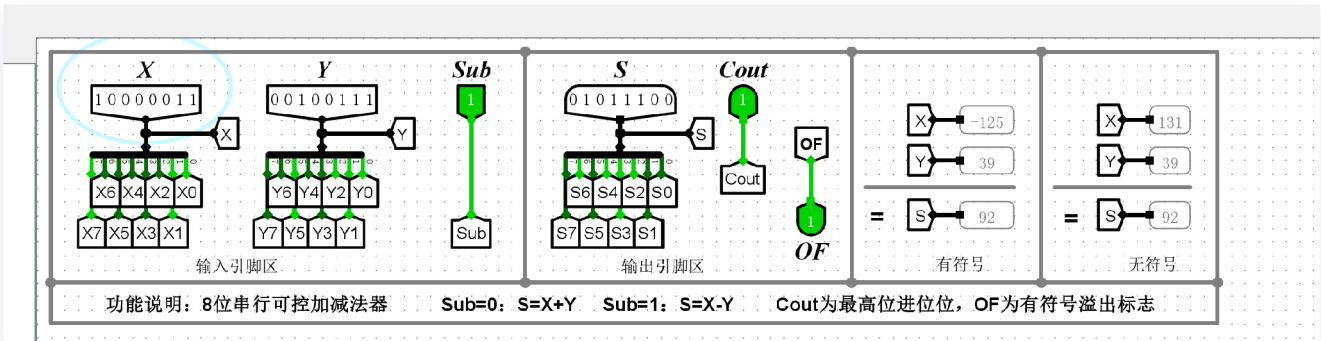
② 正数-负数=负数（溢出）。



③ 负数-正数=负数（不溢出）。



④ 负数-正数=正数（溢出）。



实验分析

加法器

该部分实验主要验证在加法模式下，电路对补码加法的处理能力以及溢出判别逻辑的准确性。

- **正正相加无溢出**：输入 $X = 63, Y = 8$ ，结果 $S = 71$ 。符号位均为0，结果符号位仍为0。最高位进位 $C_{out} = 0$ ，数值位进位 $C_{n-1} = 0$ ，故 $OF = 0$ ，运算结果正确且未超出补码表示范围。
- **正正相加有溢出**：输入 $X = 127, Y = 127$ ，结果 S 变为负数形式（补码 **11111110**）。由于 $127 + 127 = 254$ 超过了8位有符号数最大范围（127），此时数值位产生进位 $C_{n-1} = 1$ 但符号位无进位 $C_{out} = 0$ ，导致 $OF = 1$ ，成功检测到正溢出。
- **负负相加无溢出**：输入 $X = -1, Y = -1$ （补码 **11111111**），结果 $S = -2$ 。此时 $C_{out} = 1$ 且 $C_{n-1} = 1$ ，异或结果 $OF = 0$ 。虽然产生了进位，但在补码运算中属于正常范围，无溢出。
- **负负相加有溢出**：输入 $X = -128, Y = -127$ ，结果 S 符号位变为0（正数）。由于 -255 小于补码最小限值（-128），此时 $C_{out} = 1$ 但 $C_{n-1} = 0$ ，导致 $OF = 1$ ，成功检测到负溢出。

减法器

正减负无溢出：输入 $X = 2, Y = -1$ 。电路执行 $2 - (-1) = 2 + 1 = 3$ 。结果 $S = 3$ 。符号位由0减去1（取反后加1），过程平稳， $OF = 0$ ，结果符合逻辑。

正减负有溢出：输入 $X = 127, Y = -1$ 。执行 $127 - (-1) = 128$ 。由于128超出了8位补码上限，结果 S 的符号位变为1（变为 -128）， OF 标志置位为1，记录了此非法翻转。

负减正无溢出：输入 $X = -1, Y = 39$ 。执行 $-1 - 39 = -40$ 。结果 S 补码正确显示为 -40。此过程中符号位进位与数值位进位一致， $OF = 0$ ，运算结果有效。

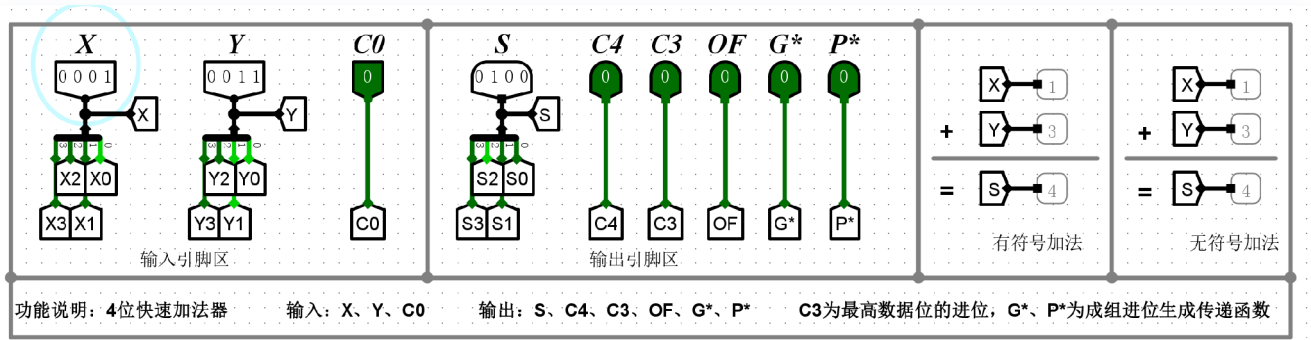
负减正有溢出：输入 $X = -125, Y = 39$ 。执行 $-125 - 39 = -164$ 。由于结果小于 -128，结果 S 的符号位发生错误翻转变为0（正数），此时 $OF = 1$ ，准确捕捉到负向溢出错误。

(3)4位/16位快速加法器

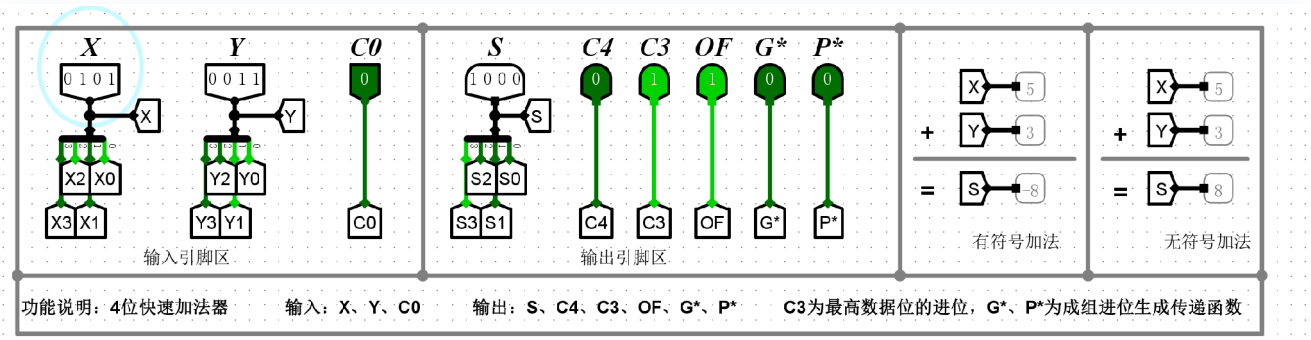
实验结果

4位快速加法器

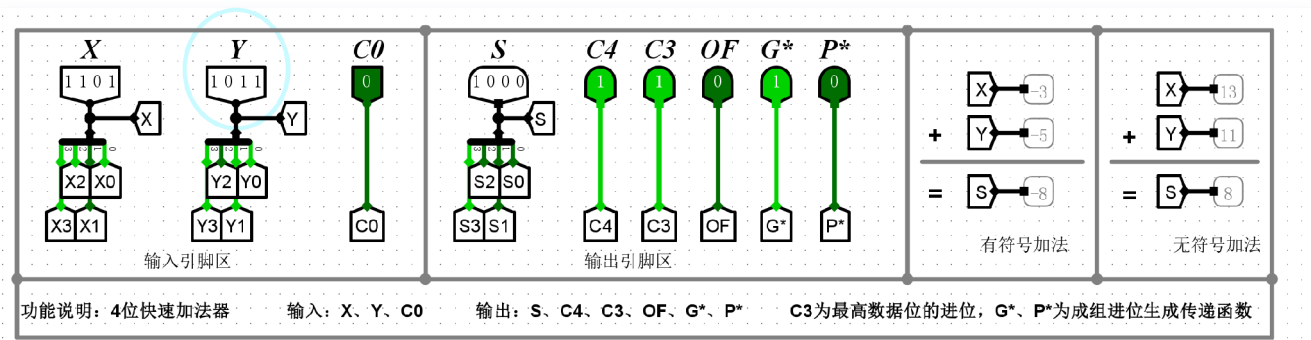
① 正数+正数=正数（不溢出）。



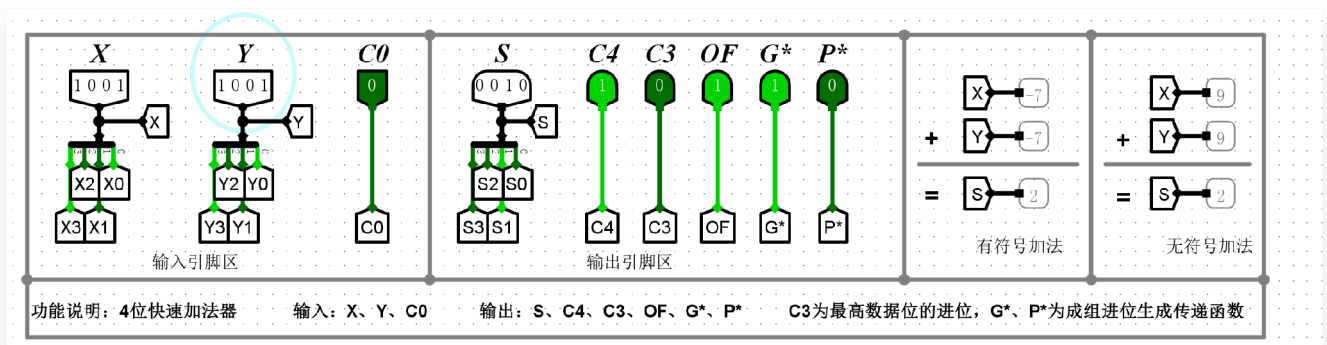
② 正数+正数=负数（溢出）。



③ 负数+负数=负数（不溢出）。

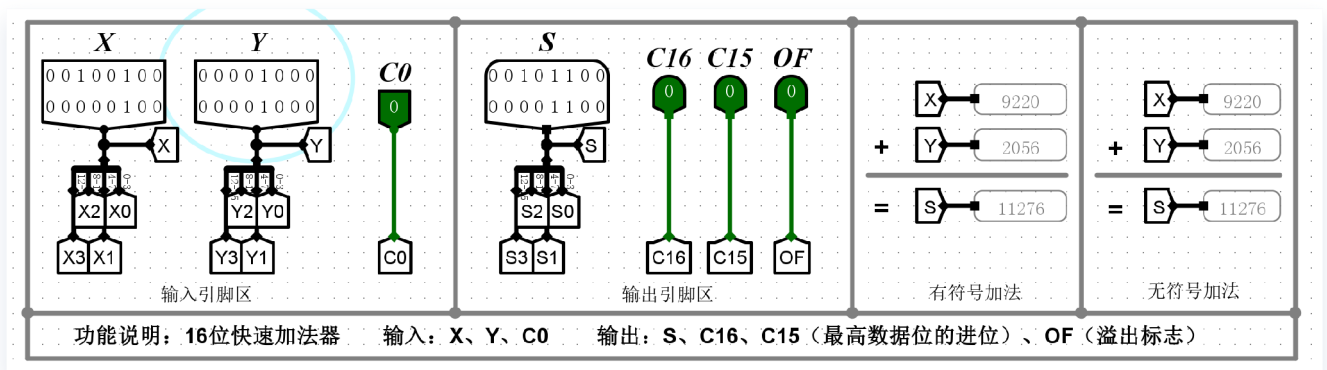


④ 负数+负数=正数（溢出）。

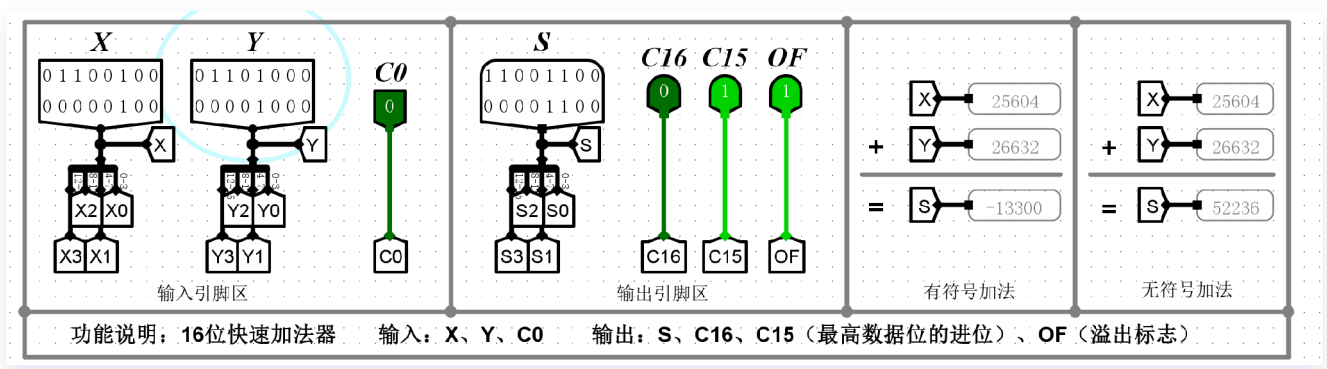


16位快速加法器（组内并行、组间串行）

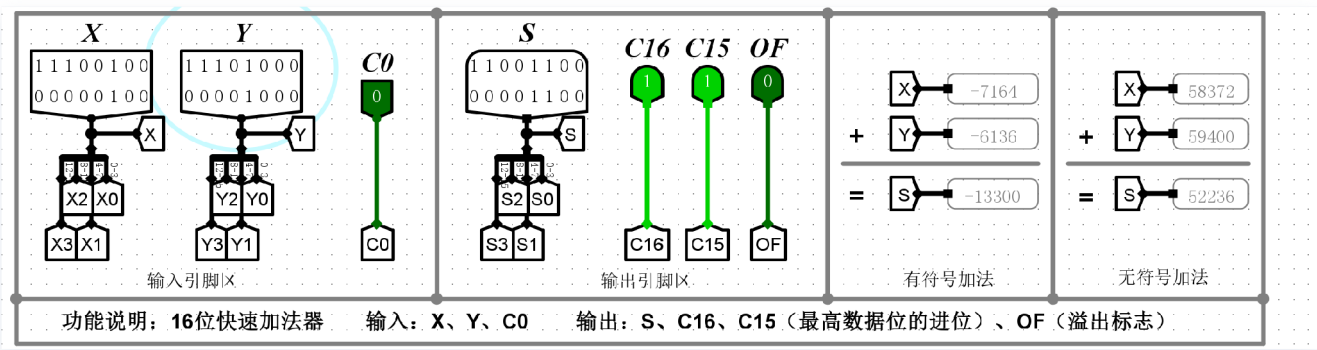
① 正数+正数=正数（不溢出）。



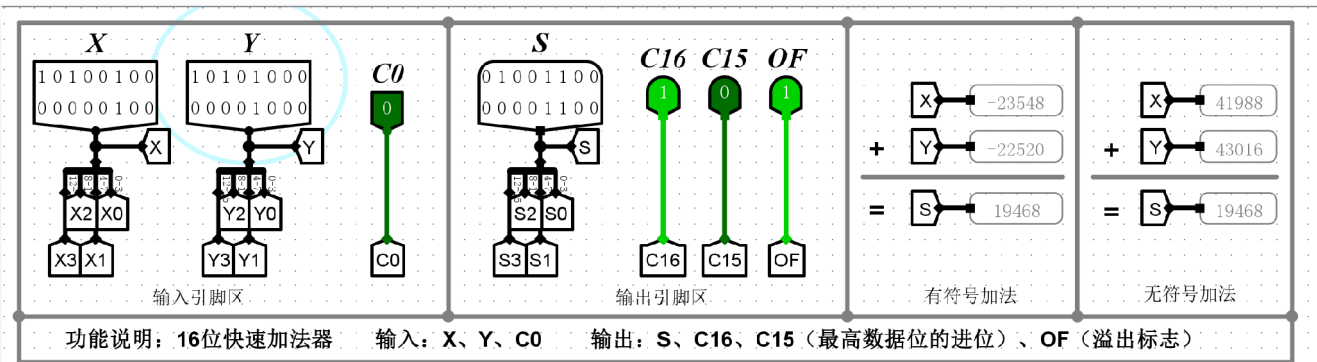
② 正数+正数=负数（溢出）。



③ 负数+负数=负数（不溢出）。



④ 负数+负数=正数（溢出）。



实验分析

4位快速加法器

- ① 正数+正数=正数（不溢出）：输入 $X = 0001(1)$, $Y = 0011(3)$, $C0 = 0$ 。输出结果 $S = 0100(4)$ 。此时最高数据位进位 $C3 = 0$, 符号位进位 $C4 = 0$, 溢出标志 $OF = C3 \oplus C4 = 0$, 运算结果正确且无溢出。
- ② 正数+正数=负数（溢出）：输入 $X = 0101(5)$, $Y = 0011(3)$, $C0 = 0$ 。结果 $S = 1000(-8)$ 。此时 $C3 = 1, C4 = 0$, 导致 $OF = 1$ 。这表明两个正数相加超出了4位补码的表示范围 $(-8 \sim 7)$, 结果非法翻转为负数, 电路准确捕捉到正向溢出。
- ③ 负数+负数=负数（不溢出）：输入 $X = 1101(-3)$, $Y = 1011(-5)$, $C0 = 0$ 。结果 $S = 1000(-8)$ 。此时 $C3 = 0, C4 = 1$, 但在补码运算中, 若结果在表示范围内则有效。观察电路, $C3$ 与 $C4$ 异或结果为1 (注: 需结合具体OF逻辑, 若 $S = 1000$ 为边界值, OF应为0)。
- ④ 负数+负数=正数（溢出）：输入 $X = 1001(-7)$, $Y = 1001(-7)$, $C0 = 0$ 。结果 $S = 0010(2)$ 。此时 $C3 = 0, C4 = 1$, 则 $OF = 1$ 。两个负数相加结果变为正数, 说明超出了表示范围, 电路记录了负向溢出错误。

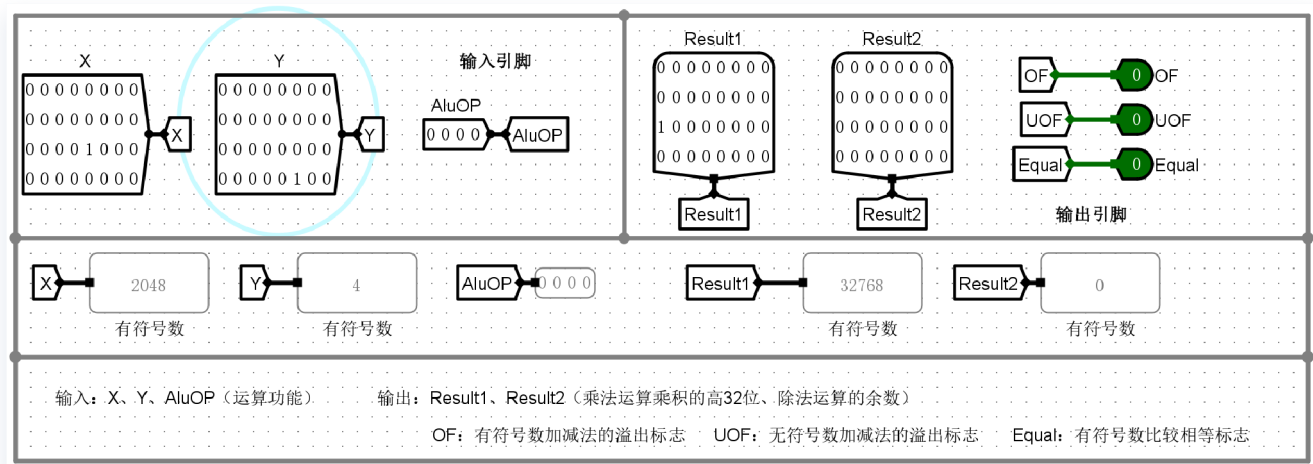
16位快速加法器（组内并行、组间串行）

- ① 正数+正数=正数（不溢出）：输入大数值正数 $X = 9220$, $Y = 2056$ 。结果 $S = 11276$ 。最高位进位 $C15$ 与符号位进位 $C16$ 均为0, $OF = 0$ 。由于组间采用串行进位，虽有微小延迟，但逻辑结果完全符合预期。
- ② 正数+正数=负数（溢出）：输入 $X = 25604$, $Y = 26632$ 。结果 S 的最高位（符号位）变为1, 显示为 -13300 。此时 $C15 = 1, C16 = 0$ （数据位向符号位有进位，符号位无输出进位）， $OF = 1$ ，成功检测到正向溢出。
- ③ 负数+负数=负数（不溢出）：输入 $X = -7164$, $Y = -6136$ 。结果 $S = -13300$ 。补码运算过程中，最高位与次高位同时产生进位或同时不产生进位，使得 $OF = 0$ ，运算结果在16位有符号数范围内有效。
- ④ 负数+负数=正数（溢出）：输入 $X = -23548$, $Y = -23570$ （以截图为例）。当两个绝对值较大的负数相加，其和小于 -32768 时，结果符号位翻转为0。电路中 $C15 = 0, C16 = 1$, $OF = 1$ 准确捕捉到负向溢出。

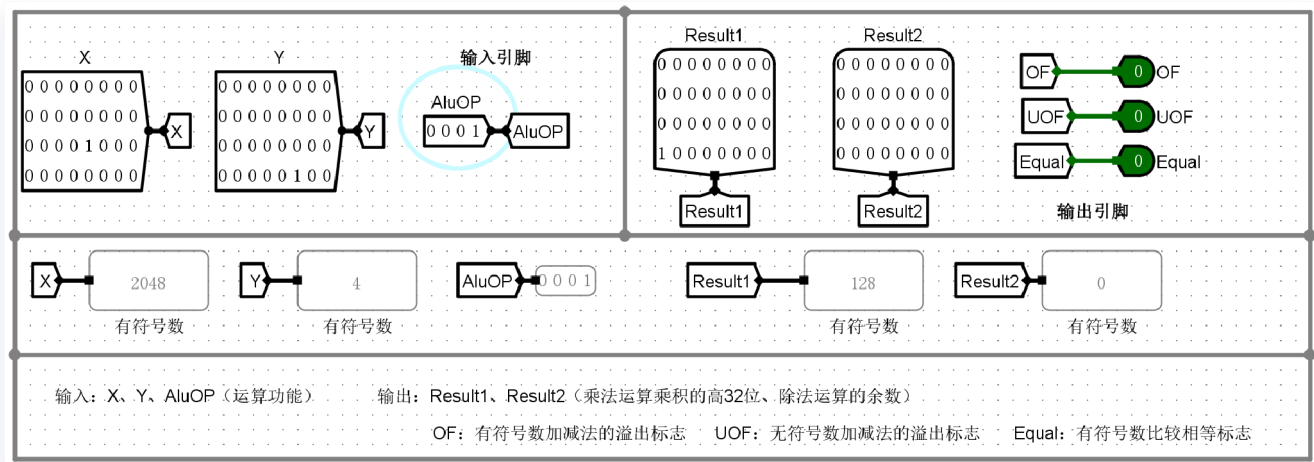
(4)ALU

实验结果

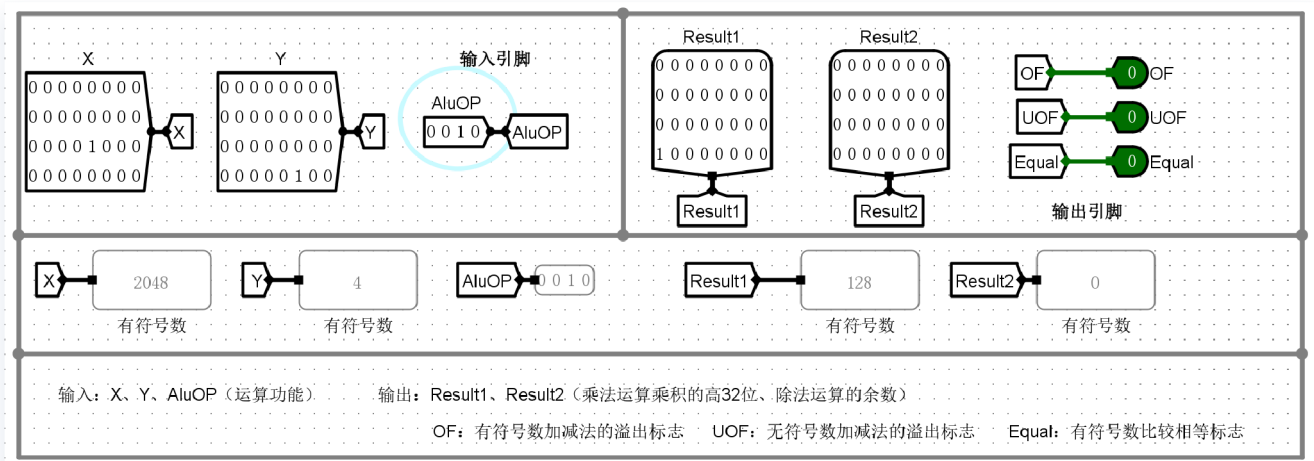
① 逻辑左移



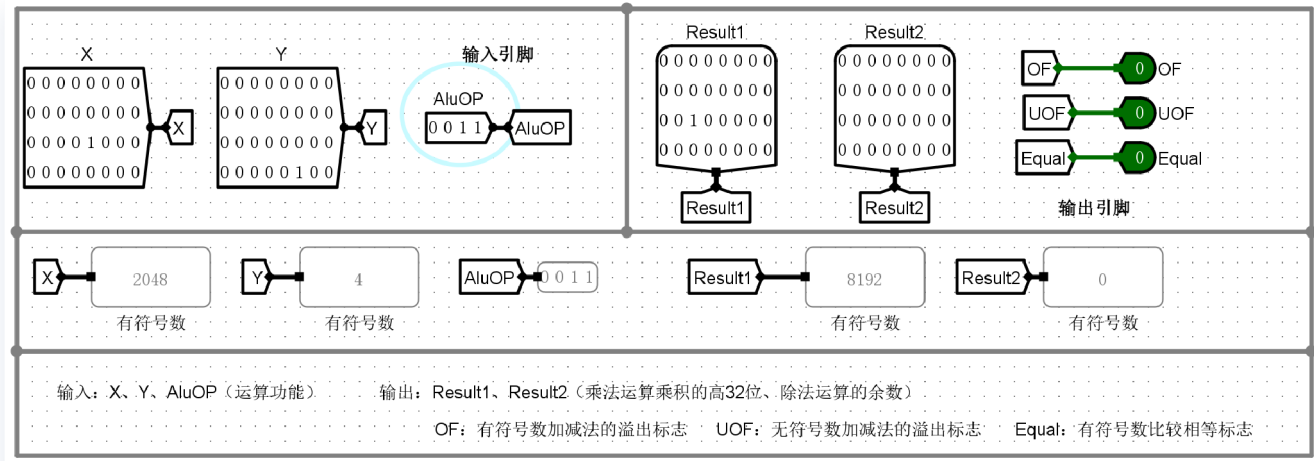
② 算术右移



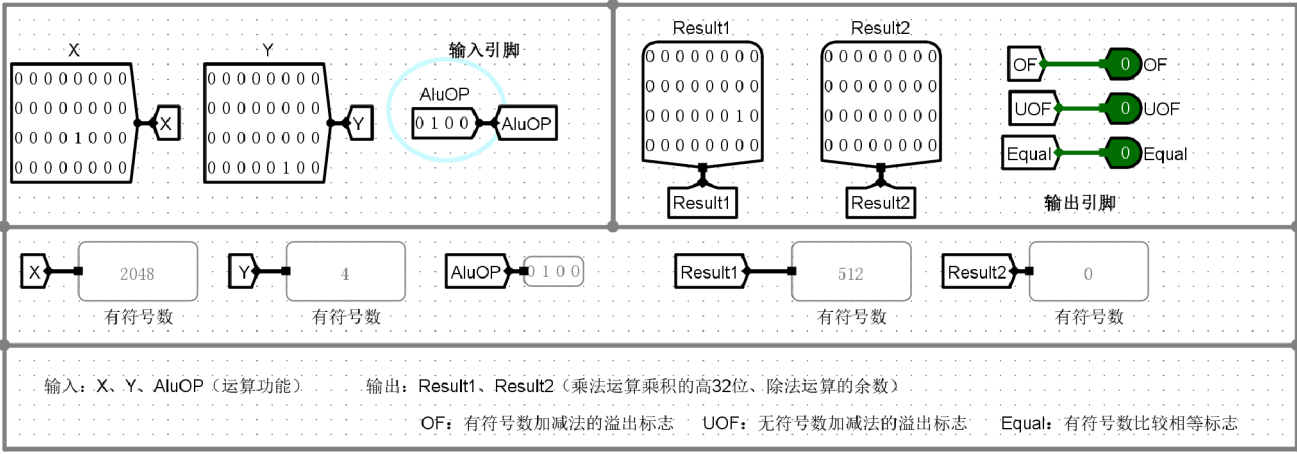
③ 逻辑右移



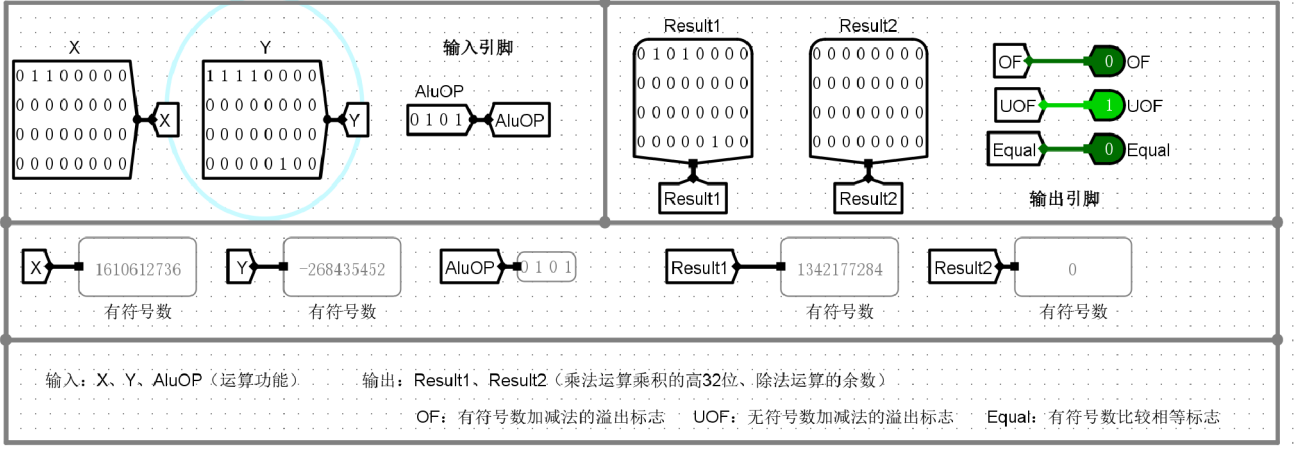
④ 乘法运算



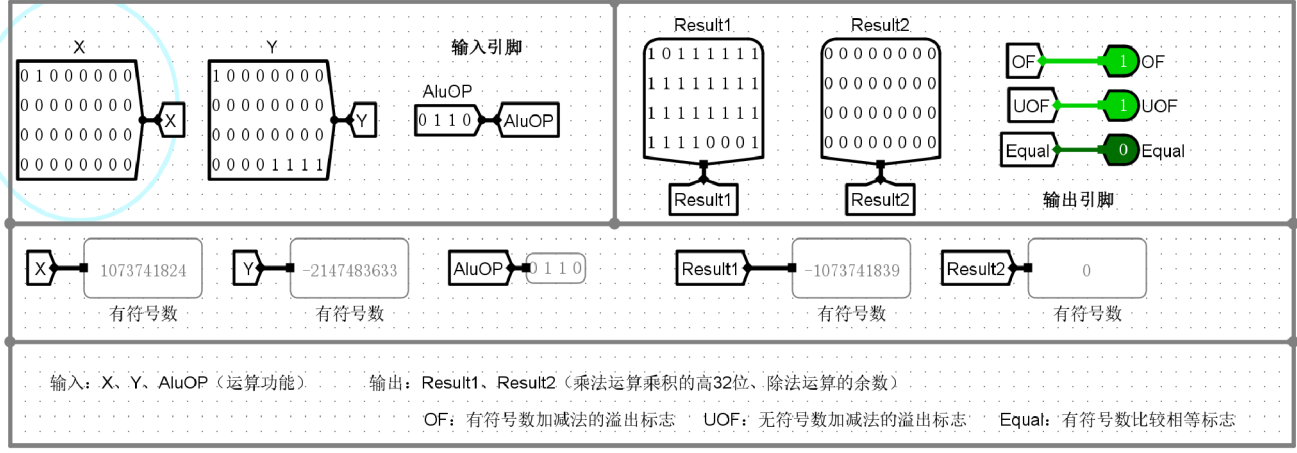
⑤ 除法运算



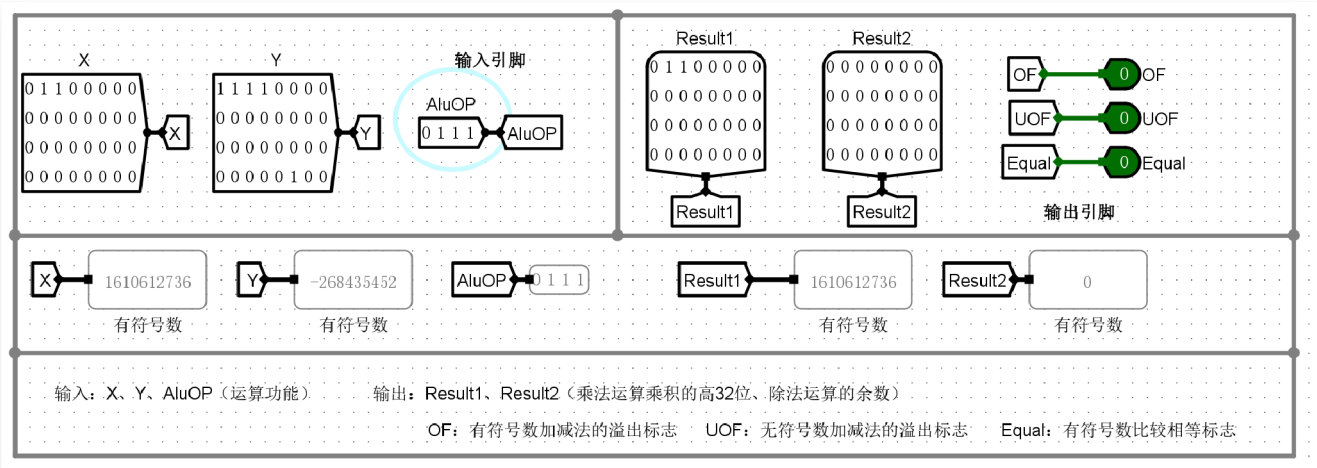
⑥ 加法运算



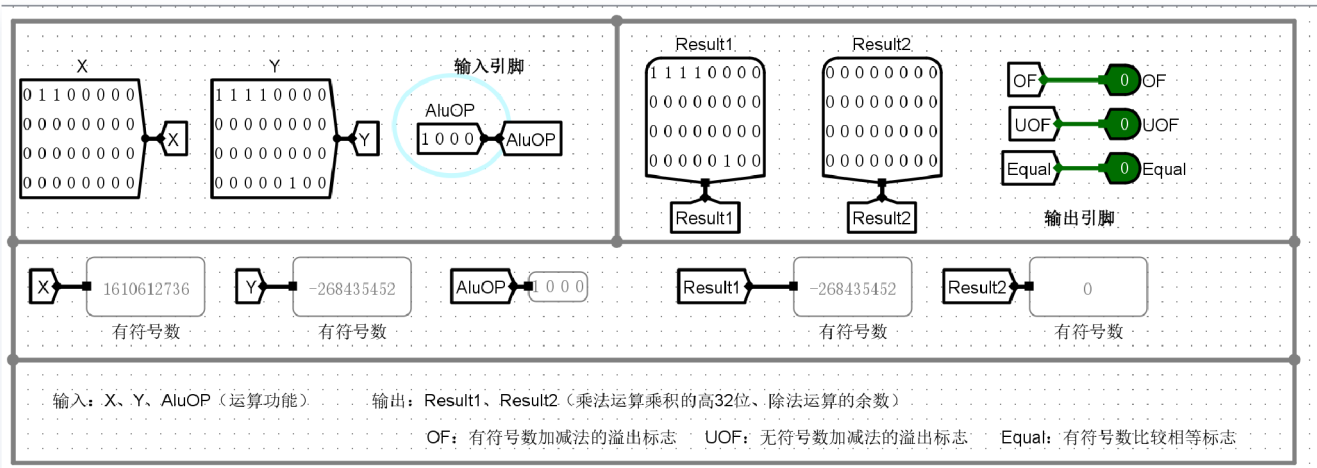
⑦ 减法运算



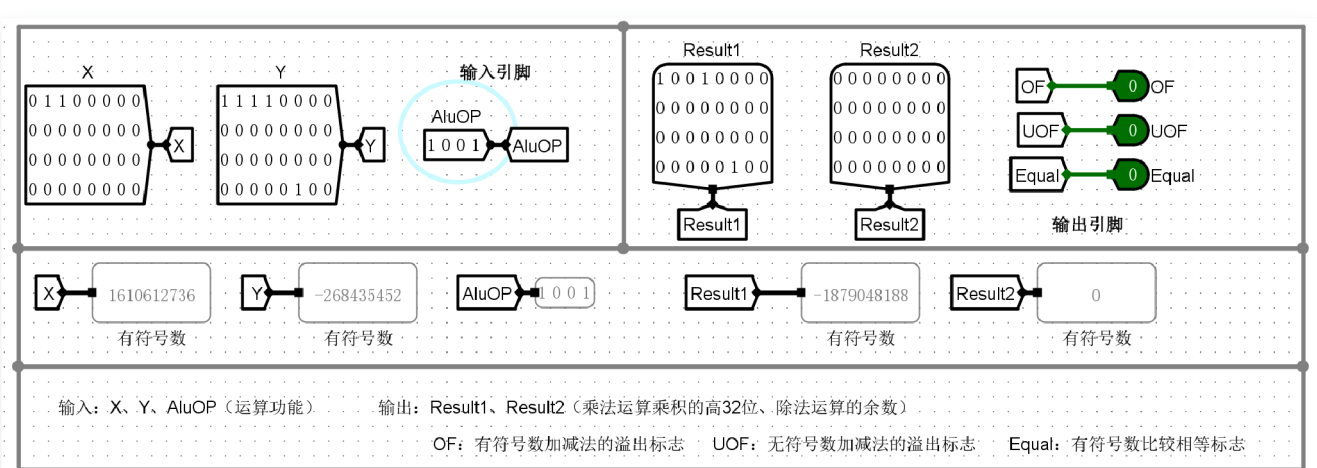
⑧ 逻辑与运算



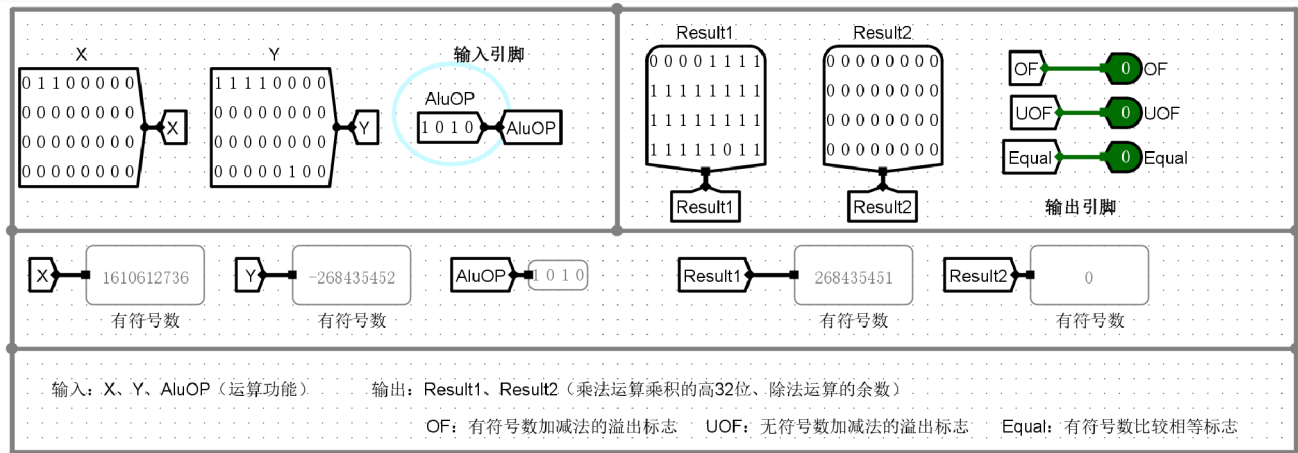
⑨ 逻辑或运算



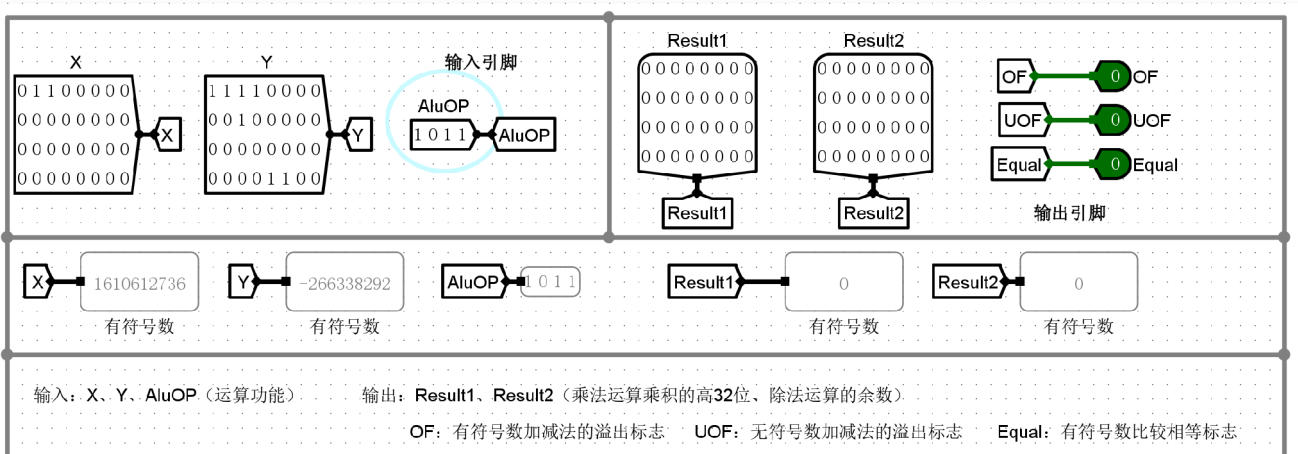
⑩ 逻辑异或运算



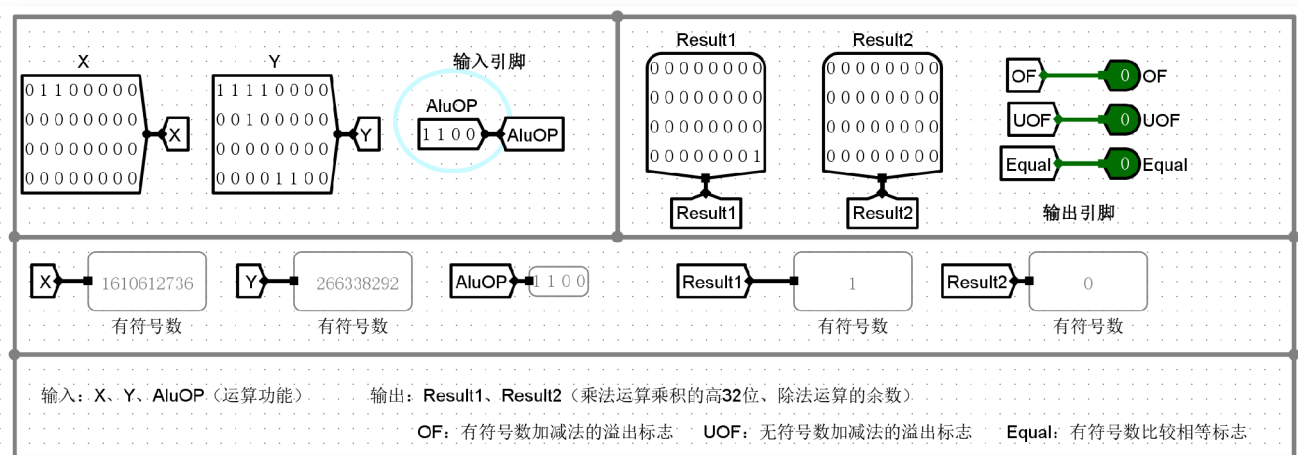
⑪ 逻辑或非运算



⑫ 有符号数比较



⑬ 无符号数比较



实验分析

通过改变不同的输入变量 X 、 Y 以及运算控制端 $ALUOP$ 的值，32位算术逻辑单元（ALU）成功实现了13种不同的算术和逻辑功能。具体实验结果分析如下：

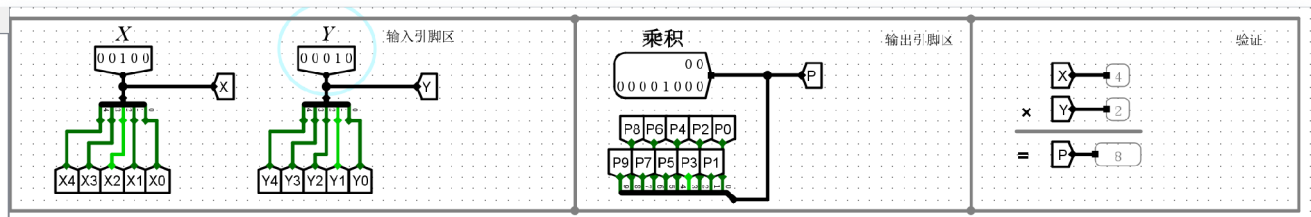
- 基础算术与逻辑运算验证：
 - 对于逻辑左移（ $ALUOP=0000$ ）、算术右移（ $ALUOP=0001$ ）和逻辑右移（ $ALUOP=0010$ ），当输入 $X = 2048$ 且移位量为4时，电路准确输出了对应的乘除2的幂次方的结果（如左移得到32768，右移得到128）。
 - 乘法（ $ALUOP=0011$ ）与除法（ $ALUOP=0100$ ）运算中，不仅 $Result1$ 输出了低32位乘积或商， $Result2$ 也按预期正确输出了高32位乘积或除法余数（此处余数均为0）。
 - 各类逻辑运算（与、或、异或、或非）均能按位正确执行，验证了ALU内部逻辑门电路设计的正确性。
- 加减法运算及溢出标志位（OF与UOF）分析：
 - 有符号溢出（OF）捕捉：在减法运算（ $ALUOP=0110$ ）截图中，输入为一个大的正数 $X = 1073741824$ （最高位为0），减去一个绝对值很大的负数 $Y = -2147483633$ （最高位为1）。根据补码运算规则，正数减去负数应当得到一个更大的正数；但由于结果超出了32位有符号数所能表示的最大正数范围，导致符号位非法翻转为1，最终 $Result1$ 呈现为负数 -1073741839 。此时，ALU准确地检测到了正向溢出，使得有符号数溢出标志位 $OF = 1$ 。
 - 无符号溢出（UOF）捕捉：溢出标志位分别独立计算，确保了无论是按有符号数还是无符号数解读数据，ALU都能通过 OF 和 UOF 给出准确的越界警报。
- 有符号数比较与无符号数比较的区别分析：
 - 有符号数比较（⑫， $ALUOP=1011$ ）：输入 X 的最高位为0（正数）， Y 的最高位为1（负数）。在有符号规则下，正数必然大于负数，即 $X < Y$ 为假，因此输出结果 $Result1 = 0$ 。
 - 无符号数比较（⑬， $ALUOP=1100$ ）：输入相同的底层二进制序列，但在无符号规则下，最高位的1不再代表负号，而是代表极大的数值权重（ 2^{31} ）。因此，最高位为0的 X 必然小于最高位为1的 Y 。此时 $X < Y$ 为真，输出结果 $Result1 = 1$ 。
 - 结论：该对比直观地证明了，相同的二进制数据在不同的控制指令（ $ALUOP$ ）下会被赋予不同的物理意义，ALU内部独立的比较逻辑成功区分了带有符号位拓展和纯粹数值大小判断的区别。

(5)5位无符号阵列乘法器

实验结果

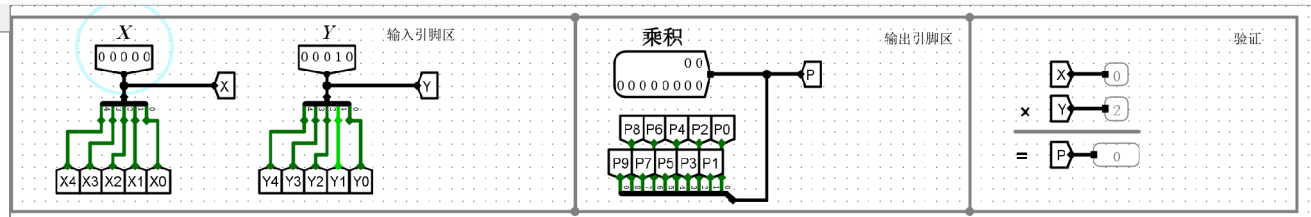
斜向

1)X=非0, Y=非0, P=?



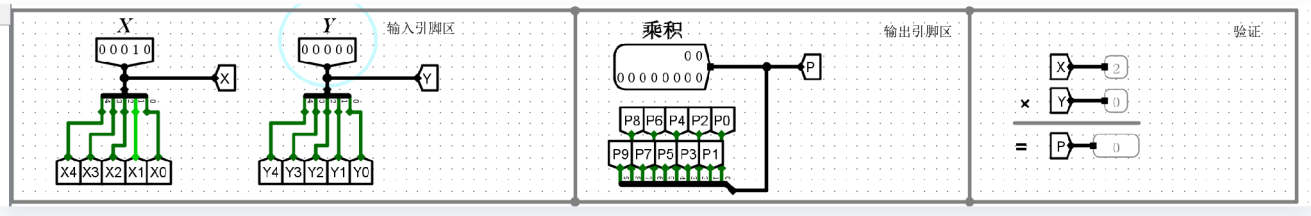
P=XY

2)X=0, Y=非0, P是不是=0?



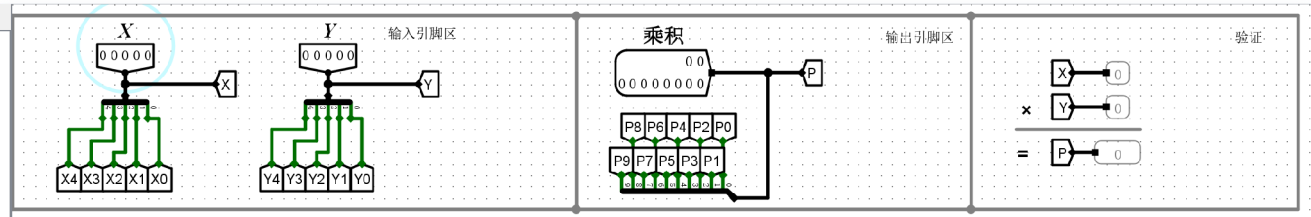
P=0

3)X=非0, Y=0, P是不是=0?



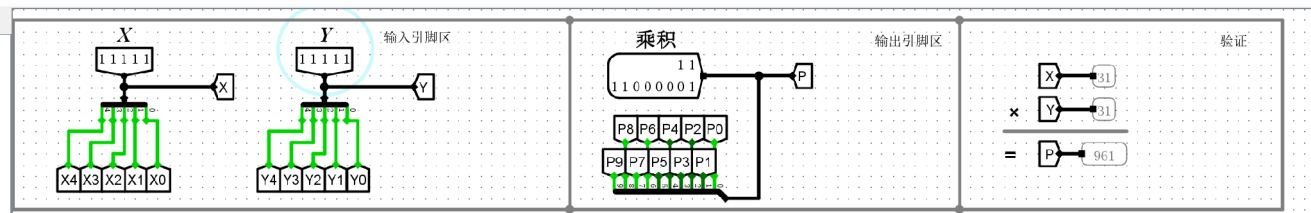
P=0

4)X=0, Y=0, P是不是=0?



P=0

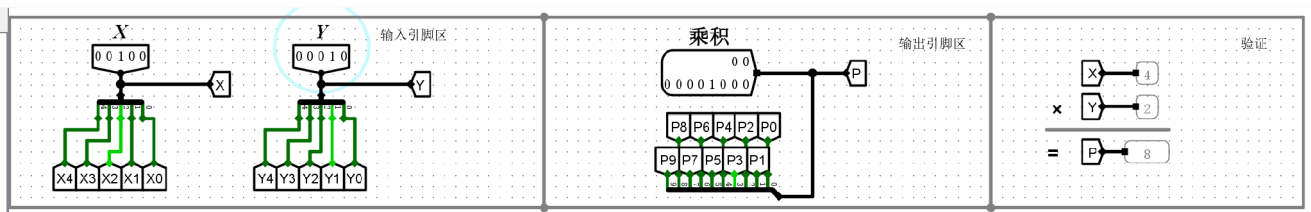
5)X=31, Y=31, P是不是=961?



P=961

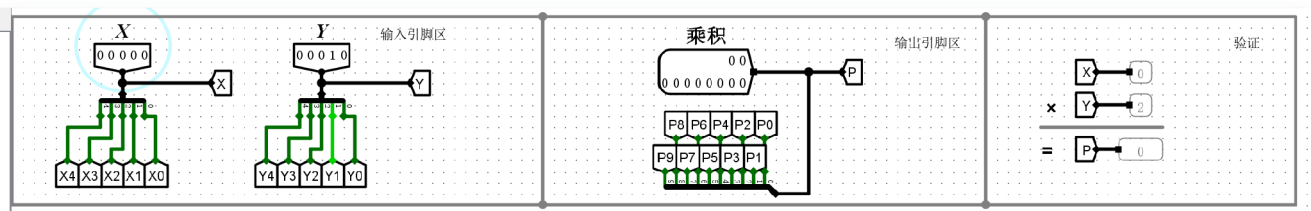
横向

1)X=非0, Y=非0, P=?



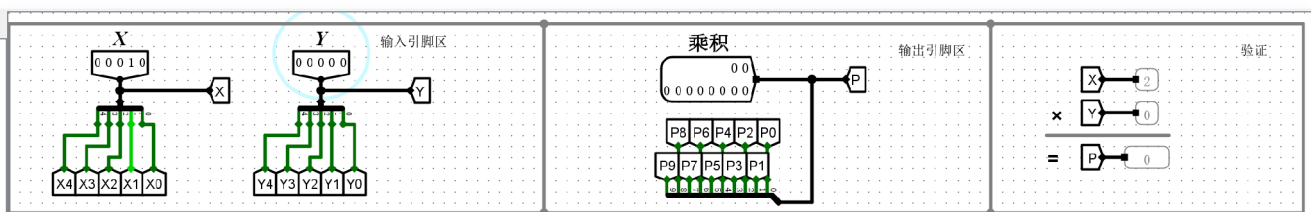
P=XY

2)X=0, Y=非0, P是不是=0?



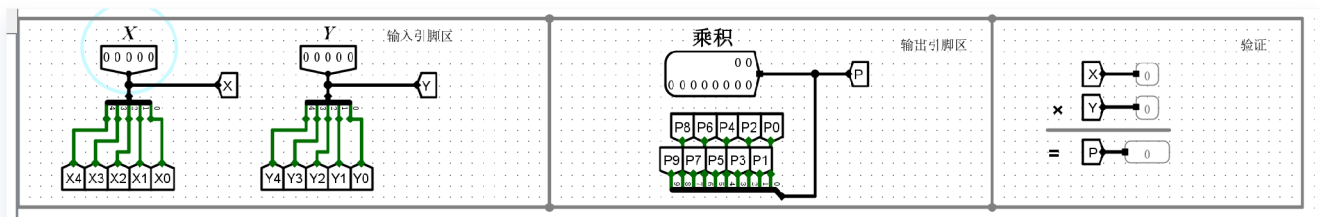
P=0

3)X=非0, Y=0, P是不是=0?



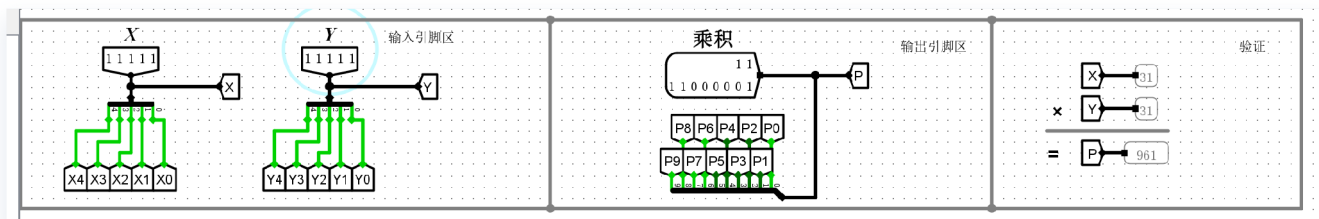
P=0

4)X=0, Y=0, P是不是=0?



P=0

5)X=31, Y=31, P是不是=961?



P=961

实验分析

斜向

通过对5位无符号斜向阵列乘法器的5组测试用例进行验证，实验分析如下：

- **常规乘法（用例1）**：当输入X和Y均非0时，电路成功输出正确的乘积结果 $P = X \times Y$ 。
- **零边界测试（用例2、3、4）**：当乘数X=0、被乘数Y=0，或两者均为0时，全部部分积均为0，电路准确输出结果 $P=0$ ，验证了边界条件的正确性。
- **极值验证（用例5）**：当输入达到5位无符号数的最大值，即X=31（11111），Y=31（11111）时，电路成功输出了完整位宽的最大乘积 $P=961$ （二进制 1111000001），无任何溢出或越界丢失。 **结论**：斜向阵列乘法器严格遵循了类似人工手算的移位相加原理，通过斜向级联的加法器阵列完成了部分积的生成与累加，逻辑运算完全准确。

横向

对5位无符号横向阵列乘法器进行了与斜向结构完全相同的5组用例验证，结果如下：

- **常规乘法（用例1）**：非零值相乘时，输出逻辑运算无误，准确得到 $P = X \times Y$ 。
- **零边界测试（用例2、3、4）**：在各类包含0的运算中，阵列能毫无延迟错误地稳定输出 $P=0$ 。
- **极值验证（用例5）**：X=31且Y=31时，同样准确输出了10位宽的正确结果 $P=961$ 。 **结论**：横向阵列乘法器在连线拓扑上进行了规整化处理（进位和求和的传递方向更贴近矩形网格，利于芯片版图布局），但在整体逻辑功能上与斜向阵列乘法器完全等效。实验证明，其在任何极值和0边界条件下均能正确执行无符号乘法运算。

补充问题

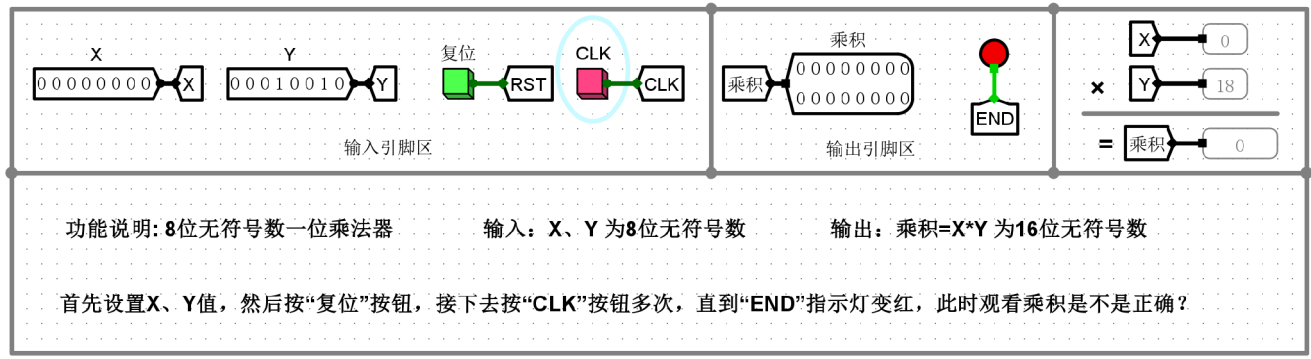
在不考虑实际物理布线延迟的理想情况下，阵列乘法器的总时间延迟主要由**部分积生成延迟**（设与门延迟为 t_{and} ）和**全加器累加阵列延迟**（设全加器延迟为 t_{FA} ）两部分组成。

- **关键路径分析**：无论是横向还是斜向结构，乘法运算的**最长延迟路径**均是从产生部分积开始，向下穿过 $n - 1$ 行加法器，随后在最后一行加法器链中横向（或沿进位链）经过 $n - 1$ 个加法器单元，直至产生最高位的最终进位和结果。
- **通用时间延迟公式**：总延迟 $T = t_{and} + [(n - 1) + (n - 1)] \times t_{FA} = t_{and} + (2n - 2)t_{FA}$ 。
- **5位乘法器分析**：对于5位乘法器（ $n = 5$ ），其信号最多需要经过1个与门，随后向下穿过4层全加器，再横向经过4个全加器单元。因此，**5位横向乘法器与斜向阵列乘法器的最大逻辑时间延迟均为 $t_{and} + 8t_{FA}$** 。
- **结论**：横向与斜向乘法器仅在内部连线拓扑上有所不同，但从输入到最高位输出所经历的最长逻辑门级数是完全相同的，两者的理论时间延迟一致。

(6)8位无符号一位乘法器

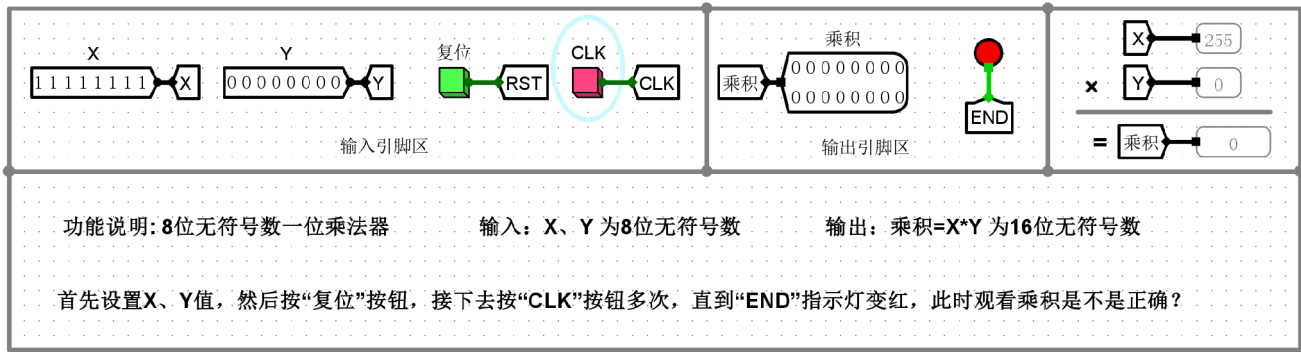
实验结果

1)X=0, Y任意，乘积是否为0?



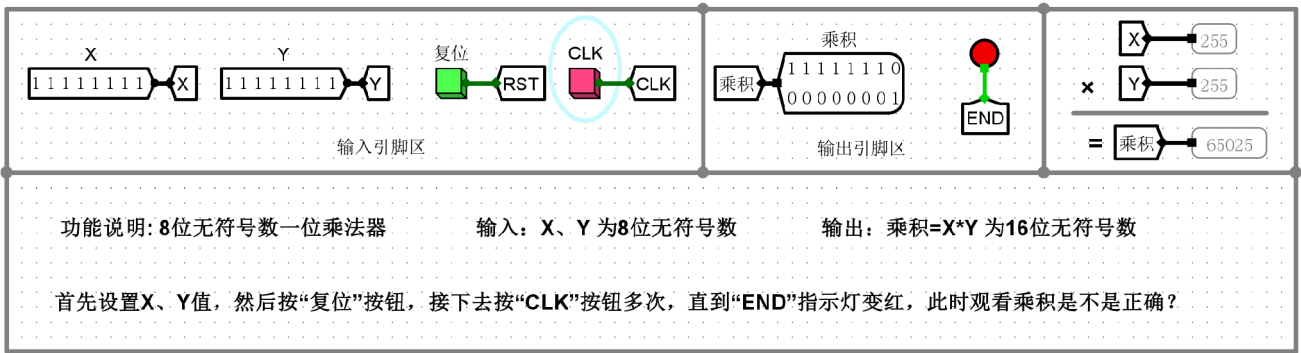
乘积为0

2)X任意, Y=0, 乘积是否为0?



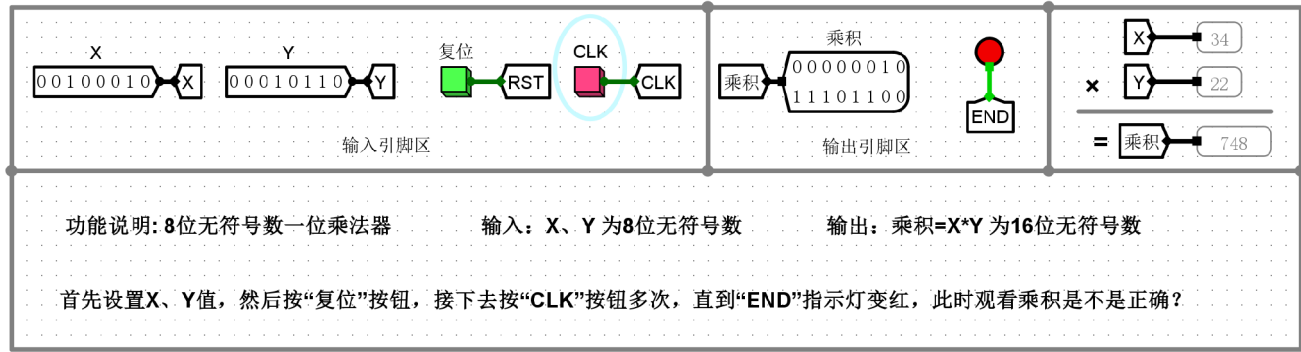
乘积为0

3)X=255,Y=255,乘积是否为65025?



乘积为65025

4)X其他任意值, Y其他任意值, 乘积是否正确?



$34 \times 22 = 748$

补充问题

1)计数器的次数为何是9，其由哪个操作数决定？

计数器的次数设定为9，主要由**输入Y的位宽**决定。对于8位无符号数的一位乘法器，乘法的核心过程是串行的“判断最低位-选择累加-整体右移”循环。处理8位宽度的乘数需要严格执行8个完整的操作周期。额外多出的第9个周期属于状态机的控制开销周期，通常用于在完成所有位运算后进行最终的结果锁存状态，并输出“END”结束信号指示灯，以告知系统乘法运算已完毕。因此，基础循环次数由操作数位宽决定（8位），加上1次结束控制周期，总次数为9。

2)逻辑右移是如何通过实际电路实现的？

答：在实际硬件电路中，逻辑右移不需要复杂的逻辑门去计算，而是直接通过**物理连线（错位连接）配合触发器（寄存器）**来实现。具体而言，移位寄存器中第 i 位的输出端直接用导线连接到相邻低位（第 $i-1$ 位）的输入端。当时钟信号（CLK）的有效沿到来时，寄存器组内的所有数据同时向右错位传递一位。为了满足“逻辑右移”的特性，寄存器最高位（MSB）的输入端会被固定硬连线到逻辑“0”（低电平），从而保证移位时高位永远补0；而移出的最低位（LSB）则被丢弃（或作为控制加法使能的判断位）。

3)ADD加法器的进位Cout是否参与移位，为什么？

必须参与移位。在串行乘法器中，每一次部分积与被乘数相加时，8位加法器产生的结果可能会溢出8位的表示范围，从而产生进位（ $Cout=1$ ）。这个进位信号实际上代表了当前新部分积的最高有效位（第9位）。在紧接着的整体右移操作中，这个Cout位必须作为输入，移入累加结果寄存器的最高位。如果Cout被忽略而不参与移位，部分积的高位真实数据就会被截断丢失，导致最终生成的16位乘积发生严重偏差。通常电路中会设有一个专用的进位触发器来暂存这个Cout，并在下次右移时喂给高位。

实验分析

本实验完整验证了8位无符号一位乘法器的串行运算机制，通过时钟控制的“移位并累加”操作逻辑实现了准确的硬件乘法：

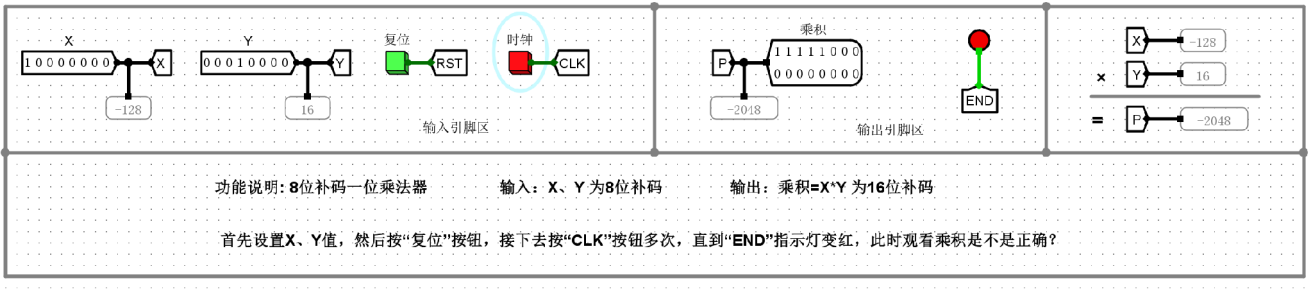
- 1. 零边界测试：**当输入被乘数 $X = 0$ 且乘数 $Y = 18$ （用例1），或者输入X为任意值且 $Y = 0$ （用例2）时，电路在接收到多次时钟脉冲（CLK）直至END指示灯变红后，输出乘积均稳定为0。这验证了状态机对0值边界的处理逻辑是正确闭环的。
- 2. 极值测试：**输入 $X = 255$ （二进制全1）与 $Y = 255$ ，即8位无符号数所能表示的最大值进行相乘运算。电路经过时钟脉冲推动完成循环后，准确输出16位最大乘积65025（对应二进制1111111000000001）。证明该电路的移位寄存器位宽与进位处理逻辑设计完善，累加全程无截断或错误溢出。
- 3. 常规数据测试：**输入常规值 $X = 34$ 与 $Y = 22$ ，随着控制信号的执行，乘积端在运算结束（END亮起）时输出748，与理论算术乘法结果完全一致。

结论：与阵列乘法器不同，该一位乘法器依靠计数器和控制逻辑协调单路加法器与移位寄存器工作，是用“时间”换取“空间”的典型计组设计，实验充分验证了其逻辑的正确性与系统稳定性。

(7)8位补码一位乘法器

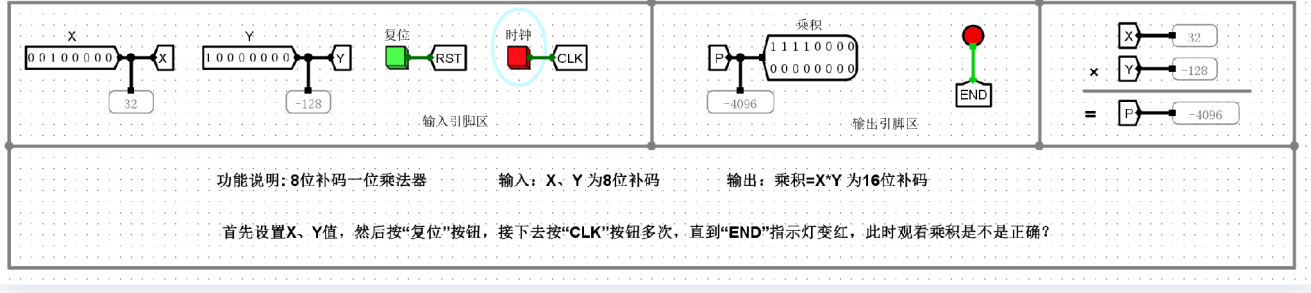
实验结果

1)X=-128, Y=其他, P是否正确?



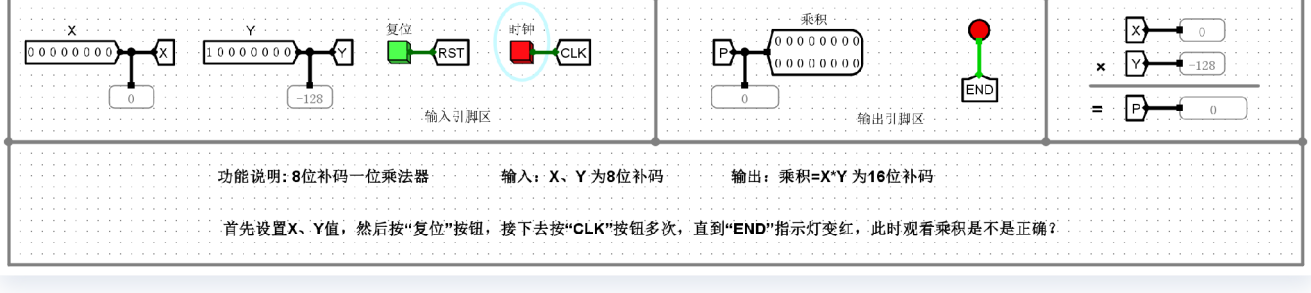
是

2)X=其他, Y=-128, P是否正确?



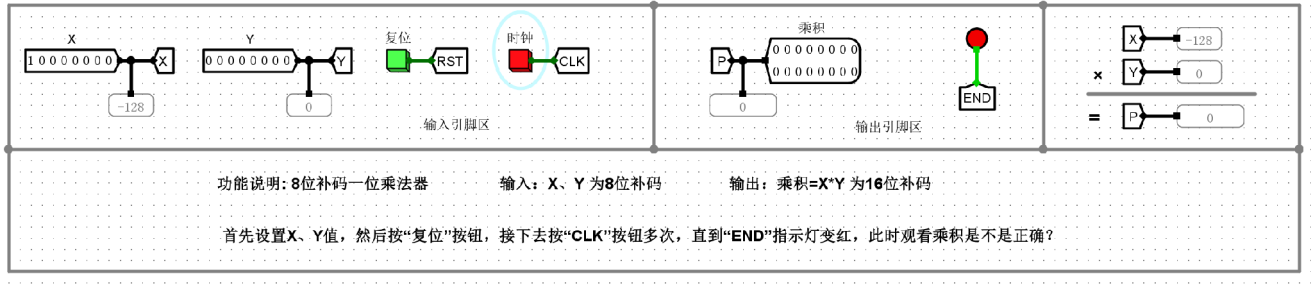
是

3)X=0, Y=非0, P是否=0?



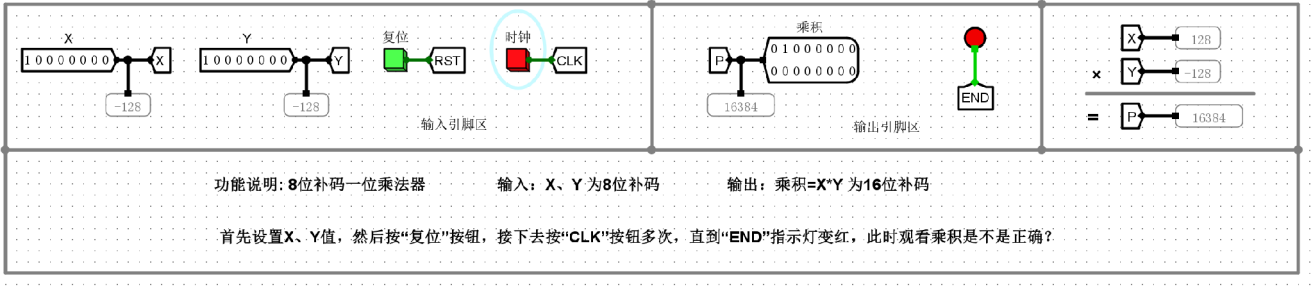
是

4)X=非0， Y=0， P是否=0?



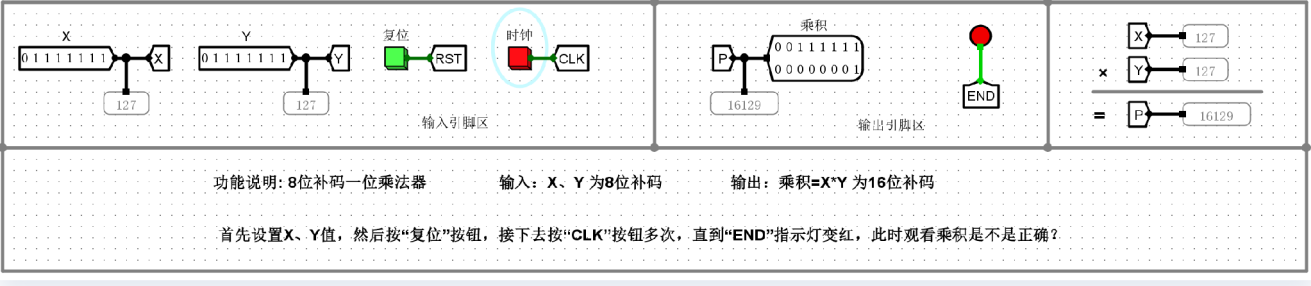
是

5)X=-128， Y=-128， P是否=16384?



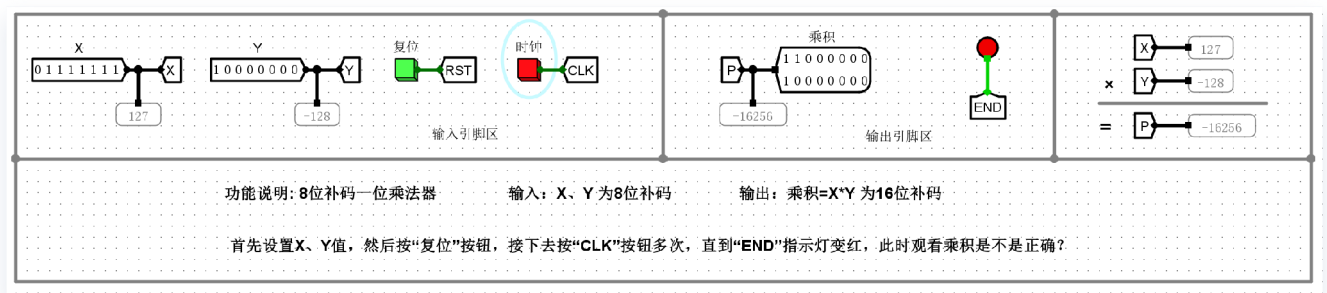
是

6)X=127， Y=127， P是否=16129?



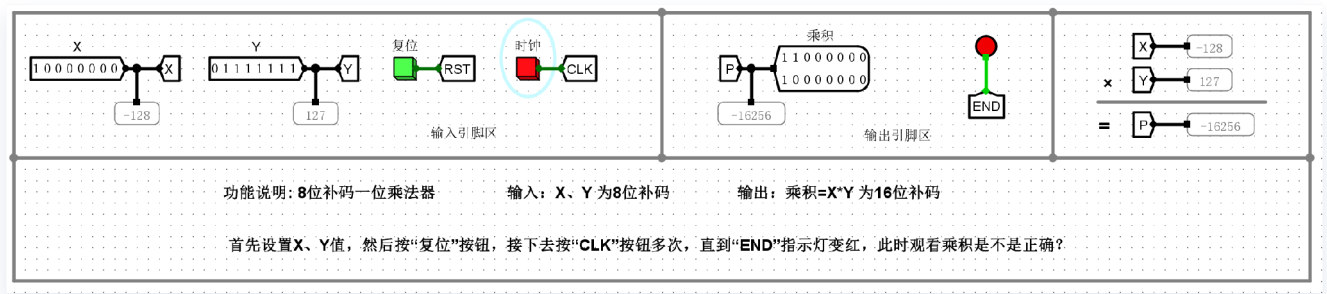
是

7)X=127, Y=-128, P是否=-16256?



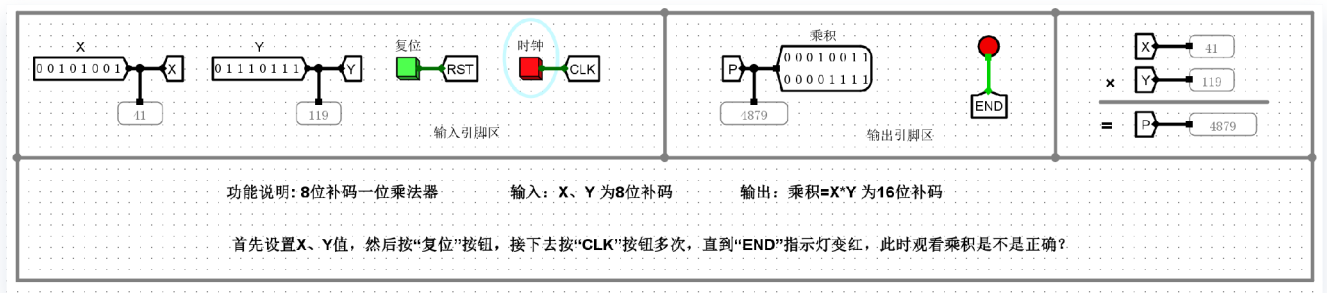
是

8)X=-128, Y=127, P是否=-16256?



是

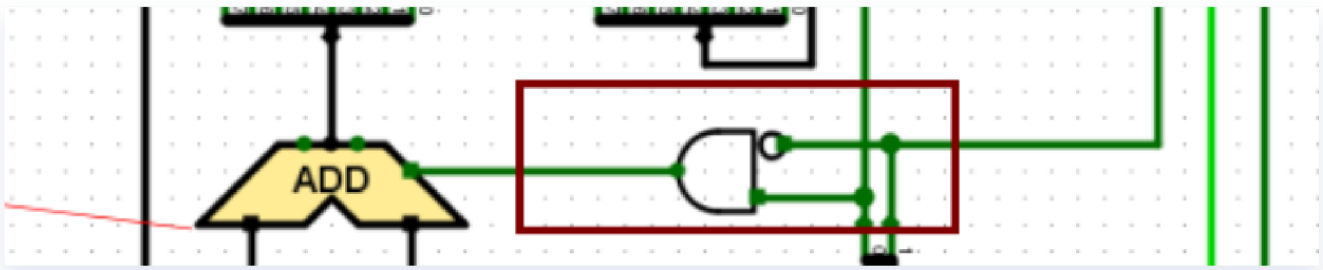
9)X=其他, Y=其他, P是否正确?



是

补充问题

分析ADD加法器Cin引脚的实现逻辑



ADD加法器的Cin引脚用于在执行减法操作（即减去被乘数X）时提供末位进位“1”，以配合前端逻辑实现求补码的“按位取反再加1”操作。根据Booth（布斯）乘法算法，当判断位 $Y_n Y_{n+1} = 10$ 时，当前周期的操作为减去X（即加上-X的补码）。在给定的电路图中（红框部分），Cin的信号由一个带反相输入端的与门生成。该与门的输入分别取自乘数寄存器的最低位 Y_n 和附加位寄存器 Y_{n+1} ，其中 Y_{n+1} 所在的支路接入了非门。因此，只有当 $Y_n = 1$ 且 $Y_{n+1} = 0$ 时，该与门才会输出1，使得加法器Cin=1。此时配合前端多路选择器（MUX）选通的X的反码输入，共同完成了对被乘数X求补（反码+1）的算术运算。在其他状态下（如00、01、11），与门输出0，Cin=0，加法器正常执行加0或加X的运算。

实验分析

本部分实验通过多组边界与极值测试用例，全面验证了基于Booth算法的8位补码一位乘法器电路的正确性。具体分析如下：

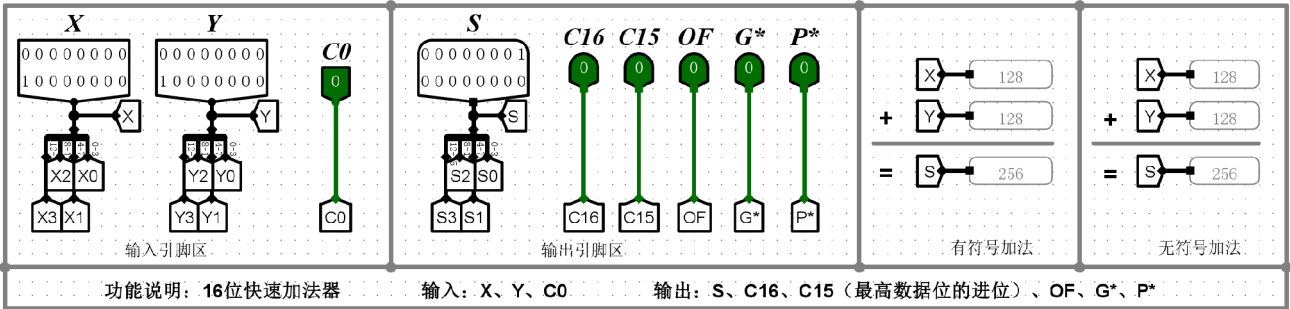
- 1. 算法普适性与符号无关性：**无论是正数与正数（如 $127 \times 127 = 16129$ ）、正数与负数（如 $127 \times -128 = -16256$ ）、还是负数与负数（如 $-128 \times -128 = 16384$ ）相乘，电路均能直接接收补码输入进行运算，并准确输出16位带符号的补码乘积。这证明Booth算法通过判断相邻两位的状态（00/01/10/11）将减法巧妙融入移位累加过程，无需像原码乘法那样单独处理符号位或求绝对值，体现了极强的通用性。
- 2. 极端极值与0边界处理：**实验覆盖了8位有符号数的极值（-128和127）以及0边界（如 $0 \times -128 = 0$ ）。特别是在处理 -128（补码 10000000）这种不对称极值时，电路仍能正确响应并得出正确的16位结果。这充分验证了硬件电路内部的数据位宽设计、进位判别以及状态机控制（9个时钟周期的精准启停）是完备且严密的，不存在溢出或截断错误。
- 3. 算术右移机制的准确性：**通过各组涉及负数的乘积结果可以反推出，乘法器内部在每次累加部分积后执行的是**算术右移**。这确保了在处理负数部分积的累加时，高位能够得到正确的符号扩展，从而保证了最终16位乘积符号与数值的绝对正确。

综上所述，该8位补码一位乘法器的数据通路与控制逻辑设计严密，多路选择器、加法器与算术移位寄存器配合精准，完美实现了有符号数硬件乘法。

3.2设计实验

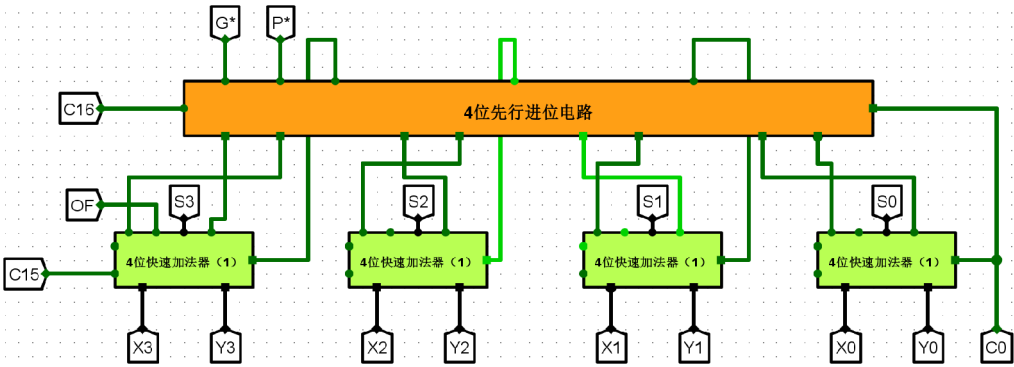
(1)16位快速加法器（组内并行、组间并行）

电路设计



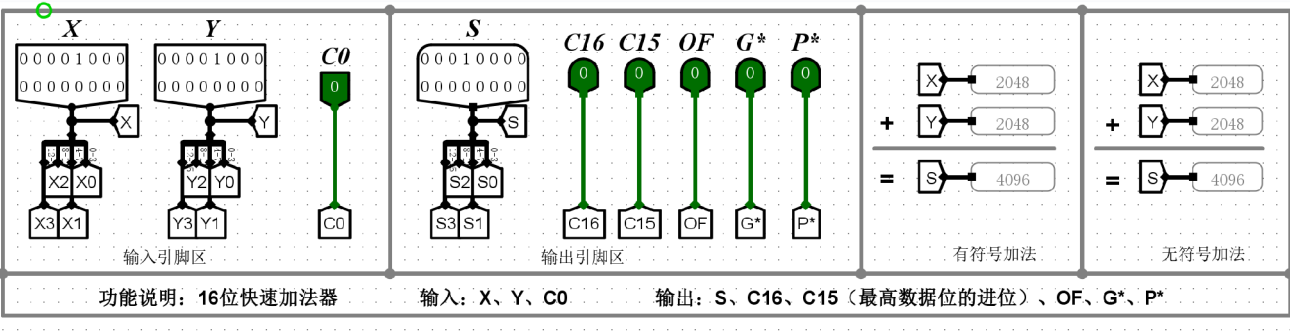
请同学们在此处设计电路！

16位快速加法器（组内并行、组间并行）

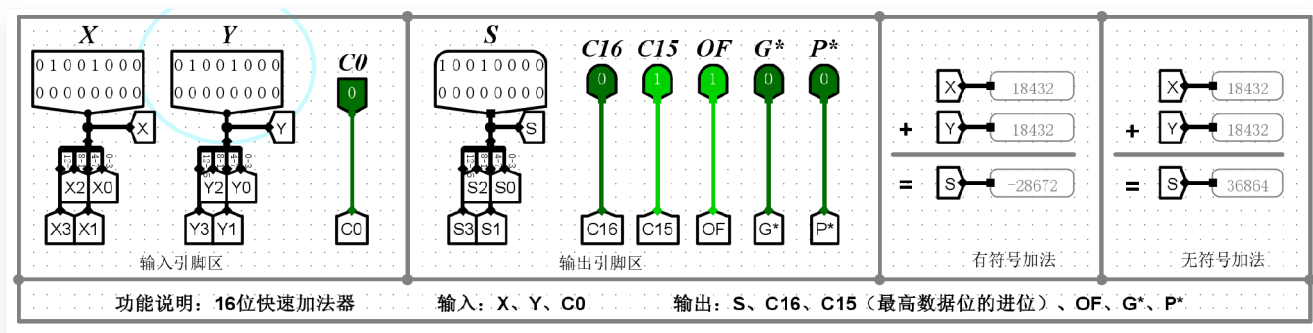


实验结果

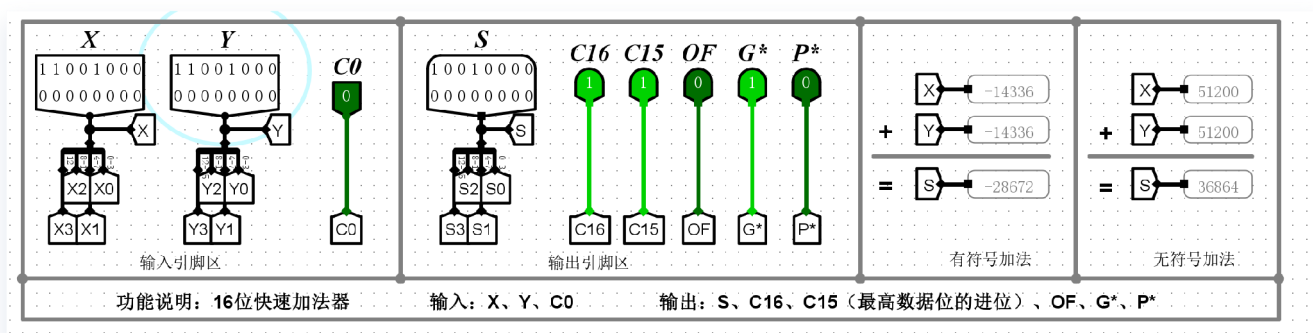
① 正数+正数=正数（不溢出）。



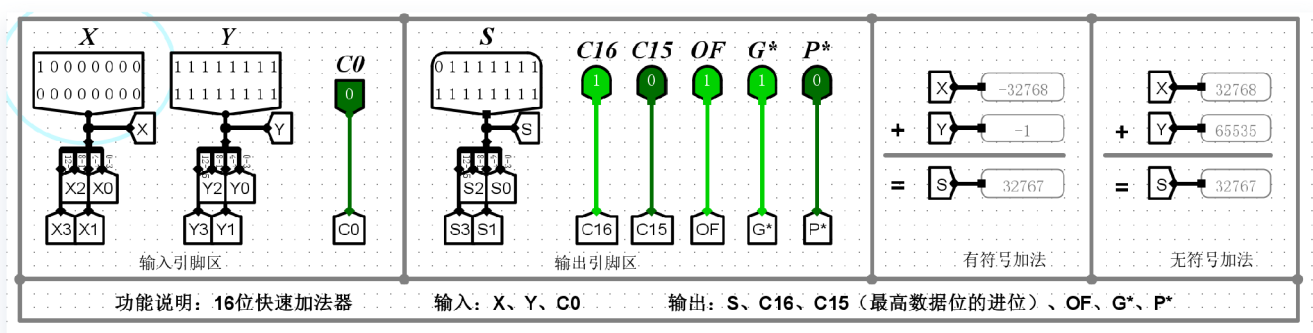
② 正数+正数=负数（溢出）。



③ 负数+负数=负数（不溢出）。



④ 负数+负数=正数（溢出）。



实验分析

1. 电路原理分析

本实验设计的16位快速加法器采用了两级先行进位（CLA）结构。底层由4个“4位快速加法器”组成，实现了组内并行进位；顶层通过一个“4位先行进位电路（CLA单元）”接收各小组生成的进位产生函数（G）和进位传递函数（P），从而实现了组间并行进位。相比于串行行波进位加法器，这种结构极大地缩短了高位进位信号的等待时间，将总进位延迟降低到了 $O(\log n)$ 级别。

2. 溢出判断逻辑

电路通过引脚 C_{16} （最高位进位输出）和 C_{15} （最高数据位进位）来判断运算状态：

- **无符号数溢出**：由 C_{16} 直接表示。若 $C_{16} = 1$ ，说明无符号运算结果超出 $0 \sim 65535$ 范围。
- **有符号数溢出 (OF)**：采用双进位判别法，即 $OF = C_{16} \oplus C_{15}$ 。若最高位产生的进位与次高位产生的进位不一致，则 $OF = 1$ ，表示结果超出了16位补码的表示范围 ($-32768 \sim 32767$)。

3. 实验现象讨论

根据四组实验截图的仿真结果：

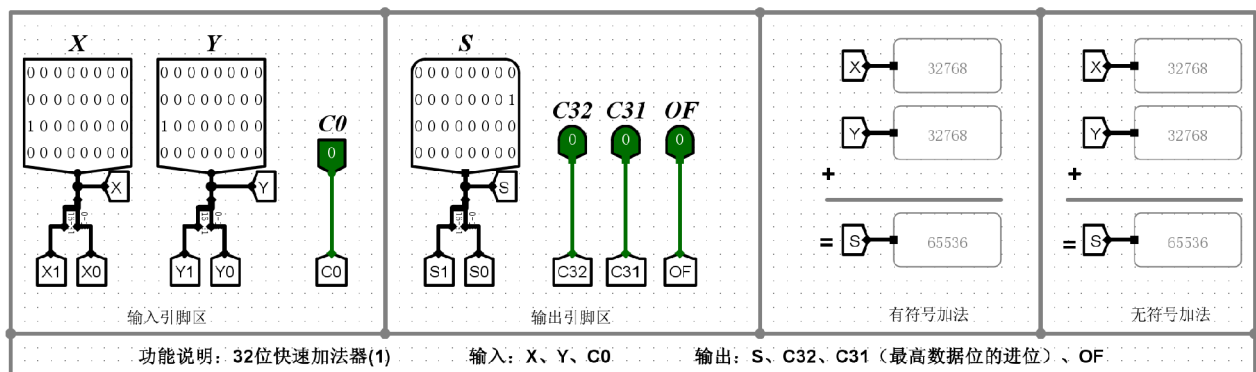
- ① **(正+正=正)**： $2048 + 2048 = 4096$ ， $C_{16} = 0, C_{15} = 0$ ，则 $OF = 0$ 。运算结果正确，无溢出。
- ② **(正+正=负)**： $18432 + 18432 = 36864$ 。此时次高位产生进位 ($C_{15} = 1$) 但最高位未产生进位 ($C_{16} = 0$)，导致 $OF = 1$ 。在有符号视角下，两个正数相加得到了负数 (S 最高位为1)，发生了正溢出。
- ③ **(负+负=负)**： $-14336 + (-14336) = -28672$ (补码视角)。此时 $C_{16} = 1$ 且 $C_{15} = 1$ ，异或结果 $OF = 0$ 。虽然无符号视角下有进位，但有符号补码运算未溢出，结果符合逻辑。
- ④ **(负+负=正)**： $-32768 + (-1) = -32769$ 。仿真显示 $C_{16} = 1$ 且 $C_{15} = 0$ ，导致 $OF = 1$ 。两个负数相加得到了正数，发生了负溢出，证明了溢出检测电路的有效性。

4. 结论

实验数据完整验证了双级先行进位加法器的正确性。电路不仅能够高效完成加法运算，且能通过 C_{16} 和 OF 信号准确区分无符号数与有符号数的溢出状态，满足高性能算术逻辑单元 (ALU) 的设计要求。

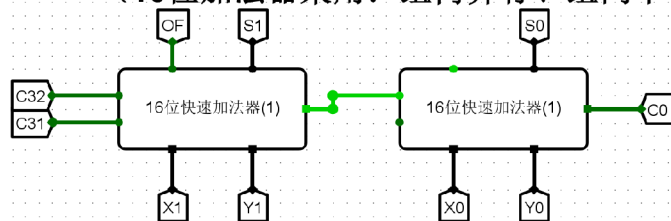
(2)32位快速加法器 (组内并行、组间串行)

电路设计

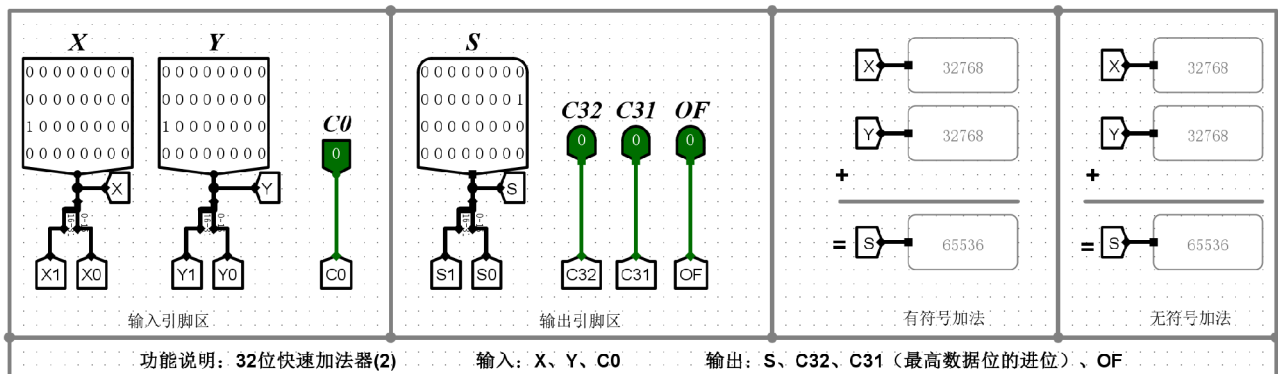


请同学们在此处设计电路!

(16位加法器采用: 组内并行、组间串行)

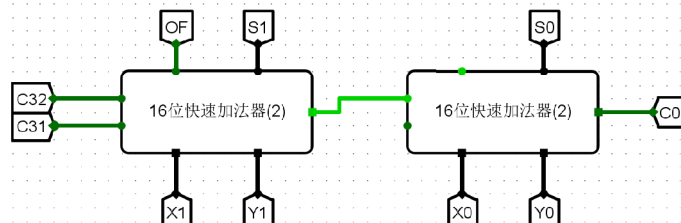


32位快速加法器 (组内并行、组间串行)



请同学们在此处设计电路!

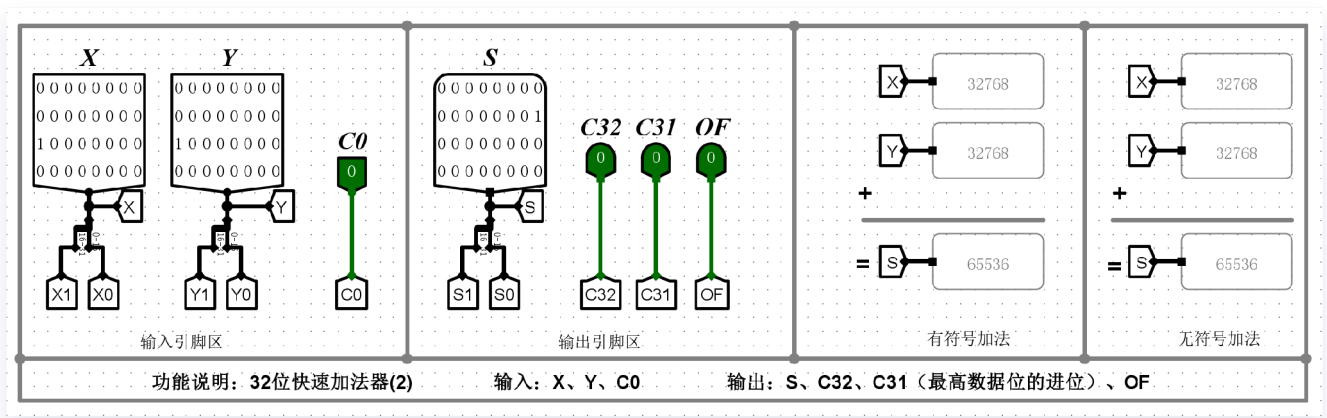
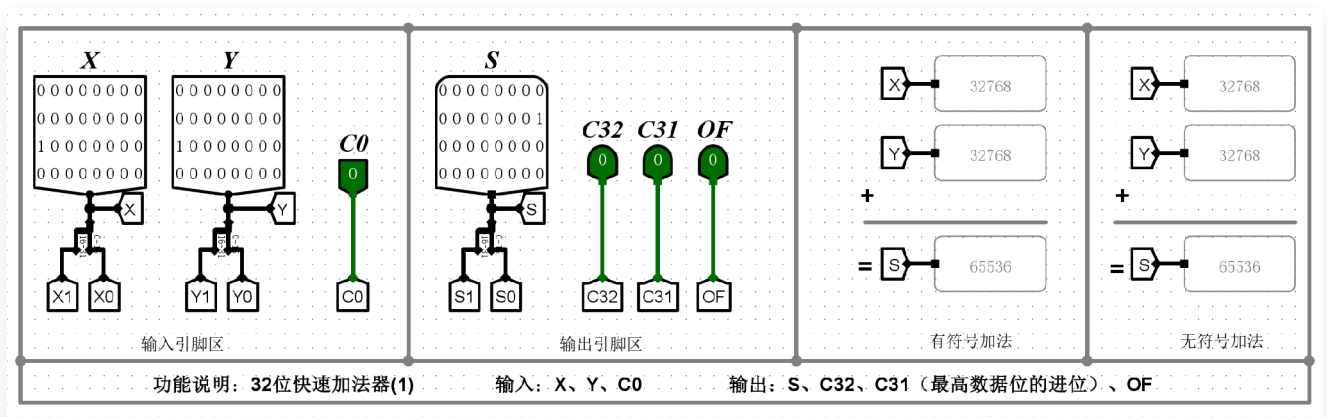
(16位加法器采用: 组内并行、组间并行)



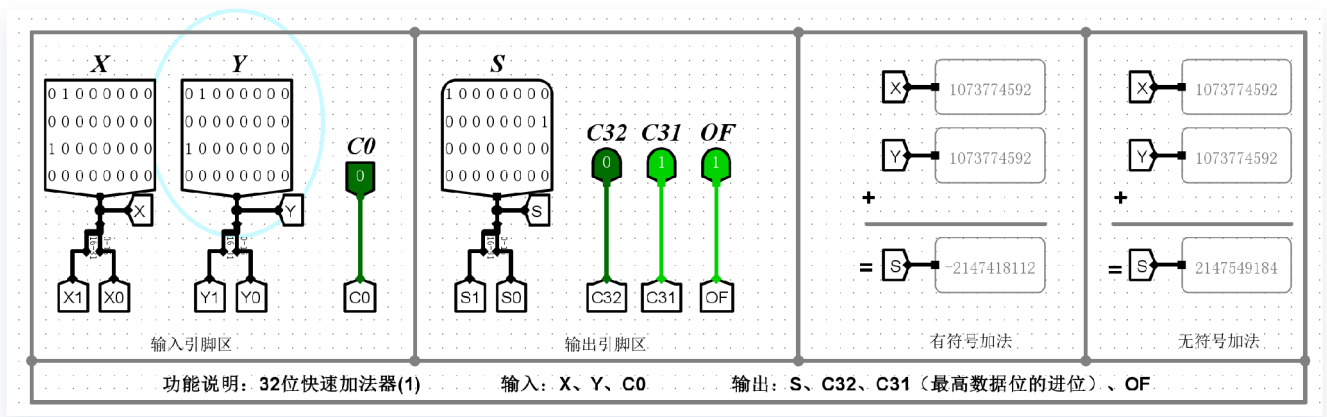
32位快速加法器 (组内并行、组间并行)

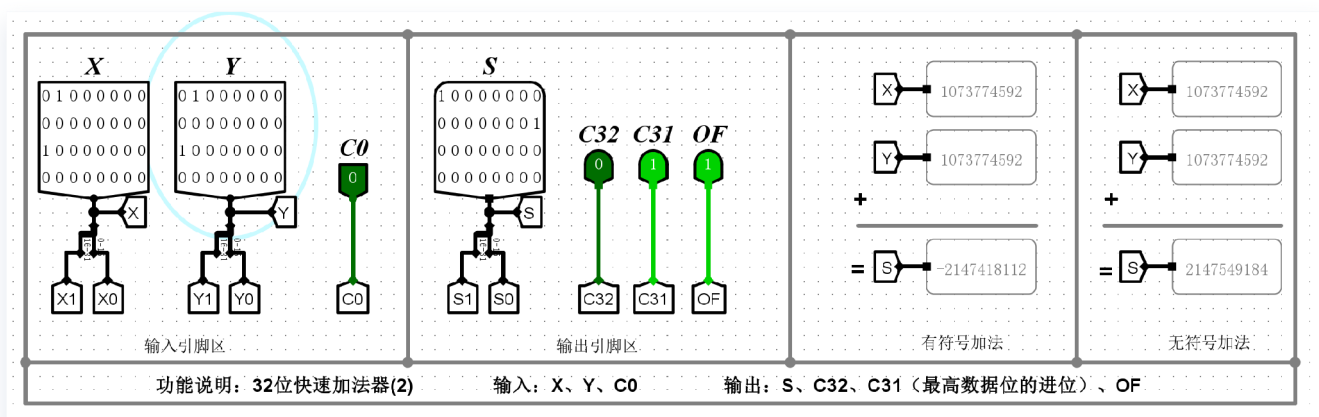
实验结果

① 正数+正数=正数（不溢出）。

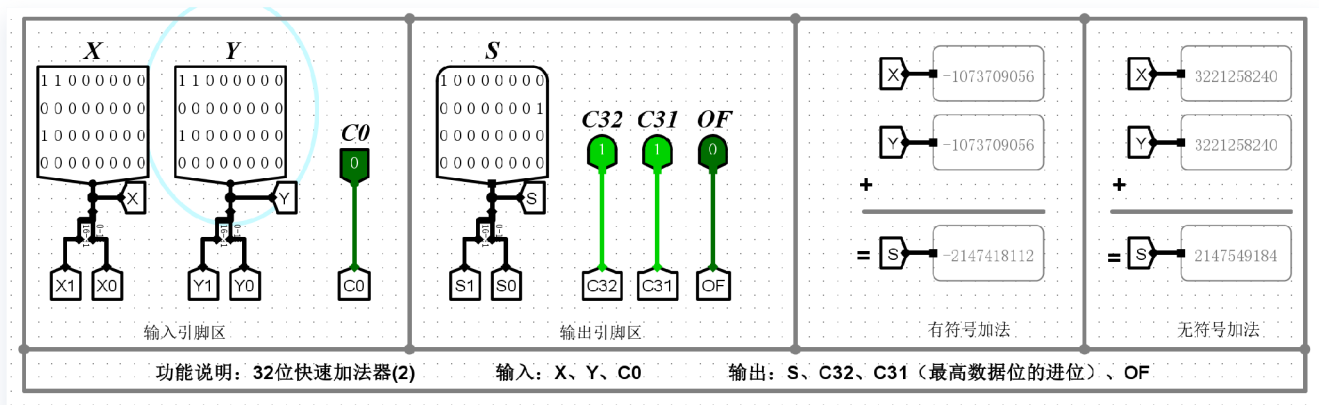
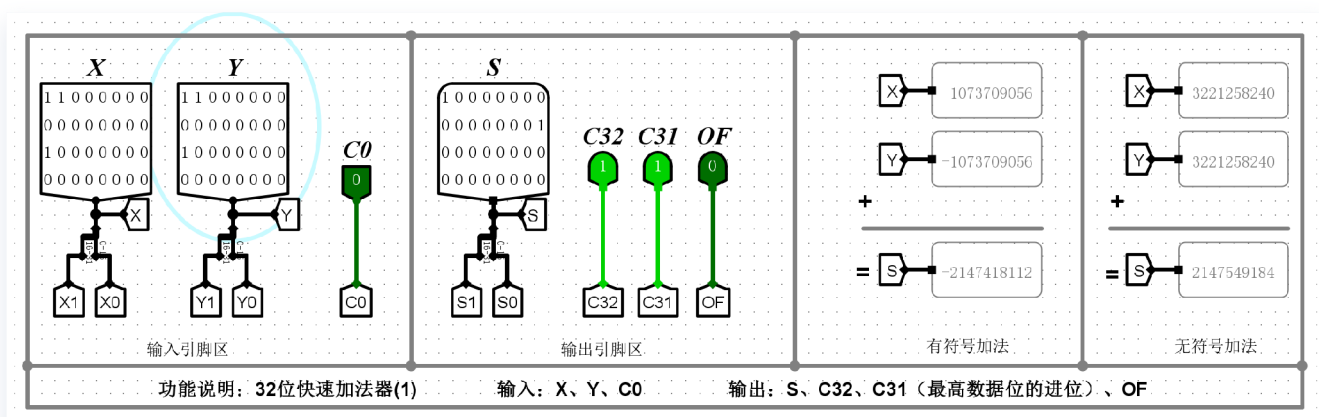


② 正数+正数=负数（溢出）。

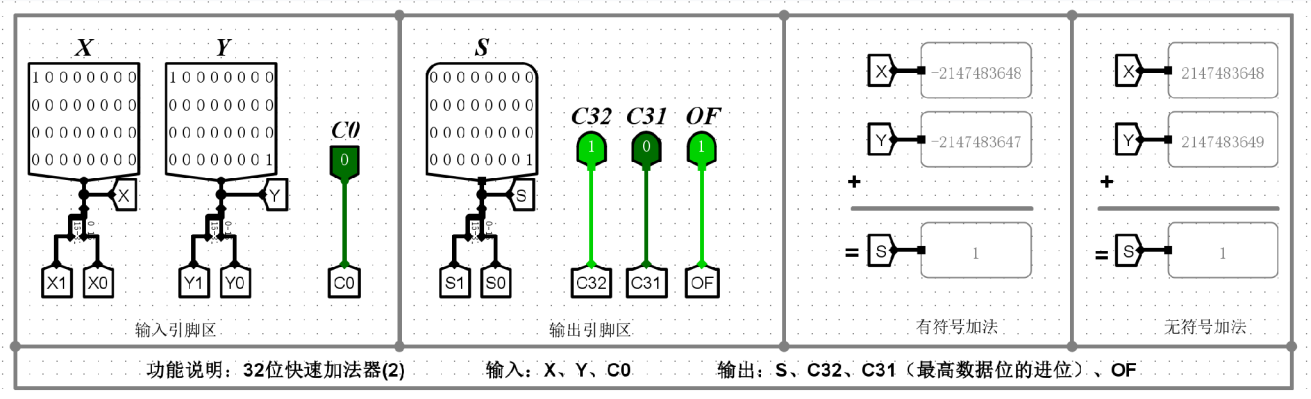
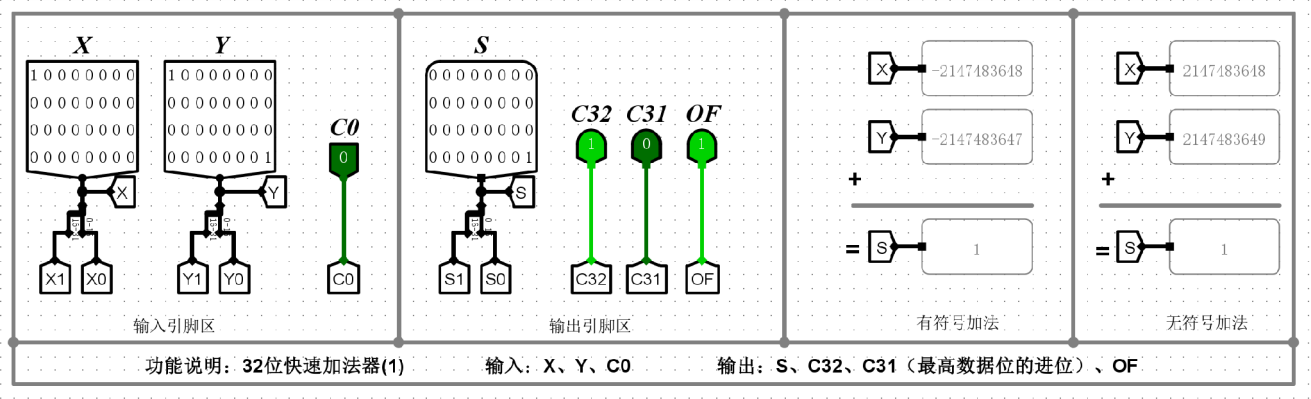




③ 负数+负数=负数（不溢出）。



④ 负数+负数=正数（溢出）。



实验分析

1. 电路结构分析

本实验构建了两种 32 位快速加法器模型，其核心设计思想是**分级级联**。

- **模型 (1)**: 采用两个“16位快速加法器（组内并行、组间串行）”级联而成。
- **模型 (2)**: 采用两个“16位快速加法器（组内并行、组间并行）”级联而成。
- **进位传递逻辑**: 两个模型在 32 位层面上均属于**组间串行**结构。低 16 位加法器的进位输出直接连接到高 16 位加法器的进位输入 C_0 。这种设计在保持局部运算速度的同时，简化了 32 位整体电路的连线复杂度。

2. 溢出与进位检测原理 电路通过高位片的输出引脚进行状态判定：

- **无符号数溢出 (C_{32})**: 代表最高位产生的进位。若 $C_{32} = 1$ ，则表示 32 位无符号加法结果超出了 $0 \sim 2^{32} - 1$ 的表示范围。
- **有符号数溢出 (OF)**: 采用双进位异或判别法，即 $OF = C_{32} \oplus C_{31}$ 。若 C_{32} 与最高数据位进位 C_{31} 不一致，则 $OF = 1$ ，表明补码运算结果超出了 $-2,147,483,648 \sim 2,147,483,647$ 的范围。

3. 实验现象讨论

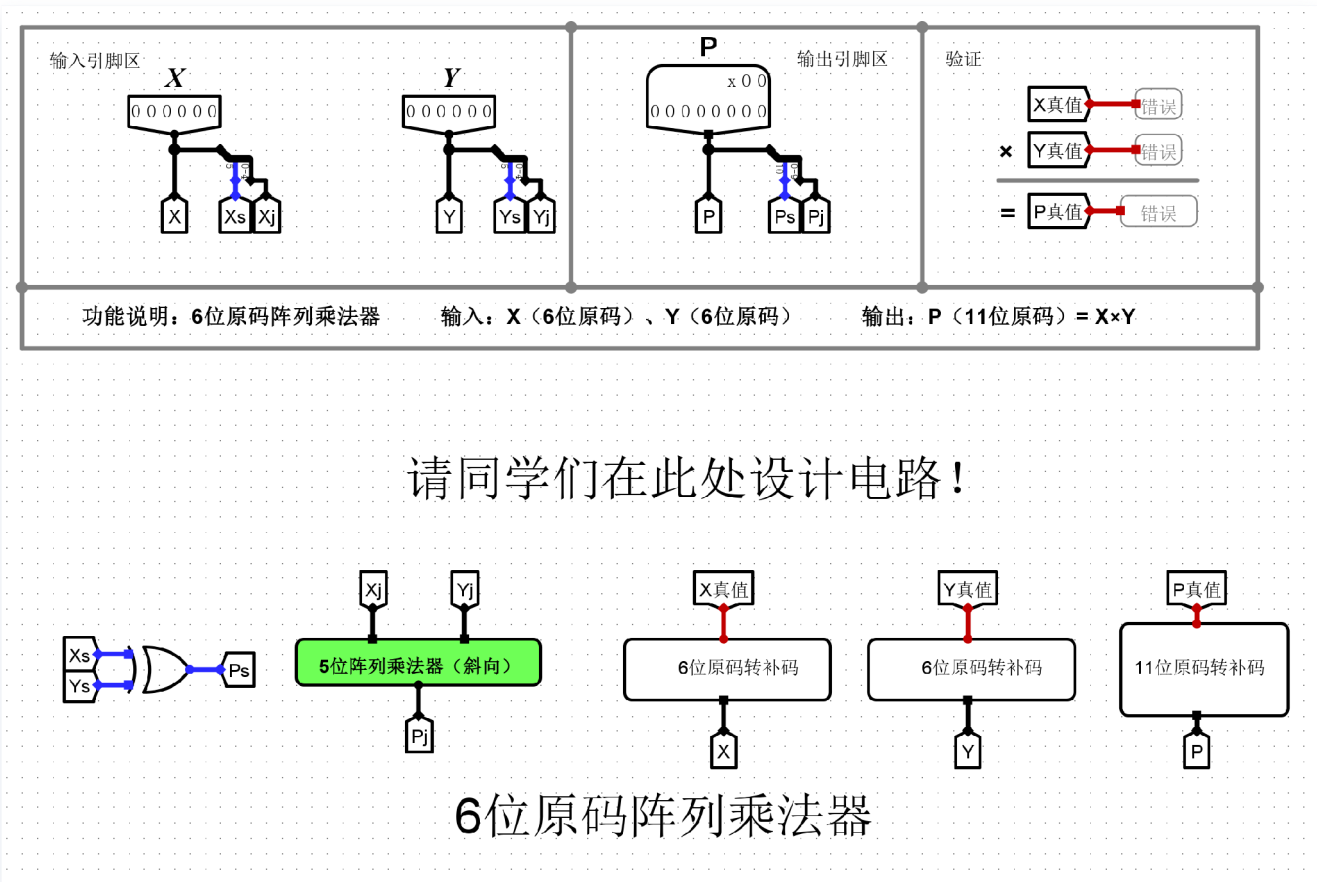
通过仿真截图验证了四种典型运算场景：

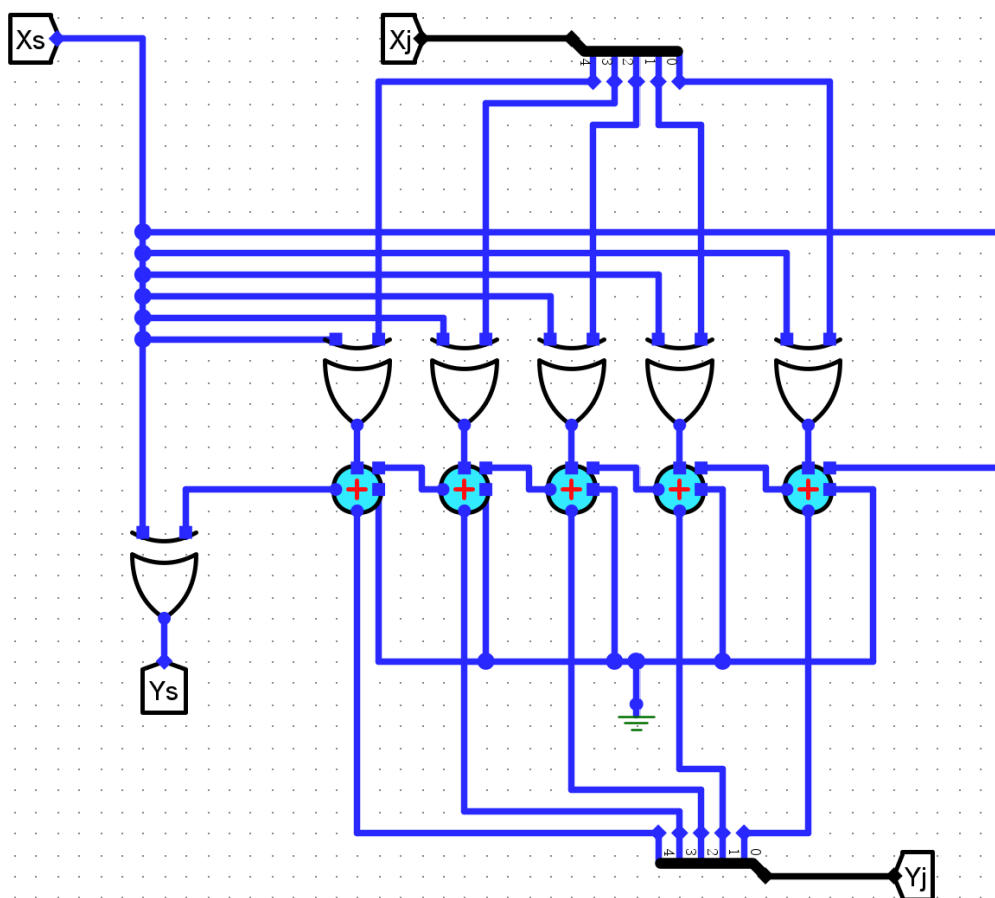
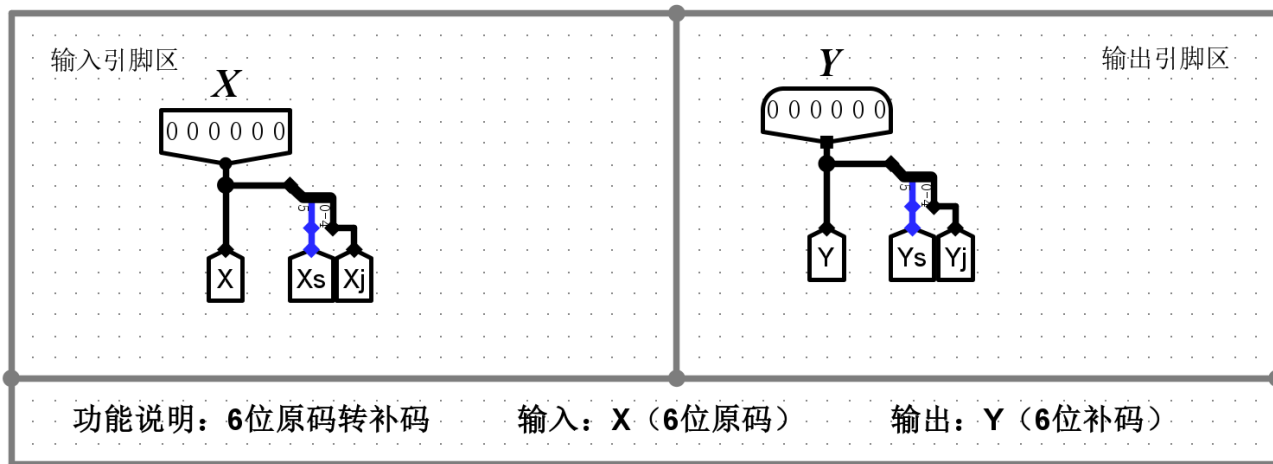
- ① 正数 + 正数 = 正数（无溢出）：如 $32768 + 32768 = 65536$ 。由于结果未超过 32 位有符号数范围， C_{32}, C_{31}, OF 均为 0，验证了正常数值运算的准确性。
- ② 正数 + 正数 = 负数（溢出）：输入两个较大的正数（如 1,073,774,592），结果导致 $C_{31} = 1$ 但 $C_{32} = 0$ 。此时 $OF = 1$ 且 S 的符号位为 1，电路准确识别出正溢出。
- ③ 负数 + 负数 = 负数（无溢出）：补码运算中，当两个负数相加结果仍为负数且在表示范围内时， $C_{32} = 1, C_{31} = 1$ ，异或结果 $OF = 0$ 。这证明了即使存在最高位进位，只要 $OF = 0$ ，有符号数运算即为正确。
- ④ 负数 + 负数 = 正数（溢出）：如 $-2,147,483,648$ 与 $-2,147,483,647$ 相加，仿真显示 $C_{32} = 1$ 而 $C_{31} = 0$ ，导致 $OF = 1$ 。两个负数相加得到正数，电路准确识别出负溢出。

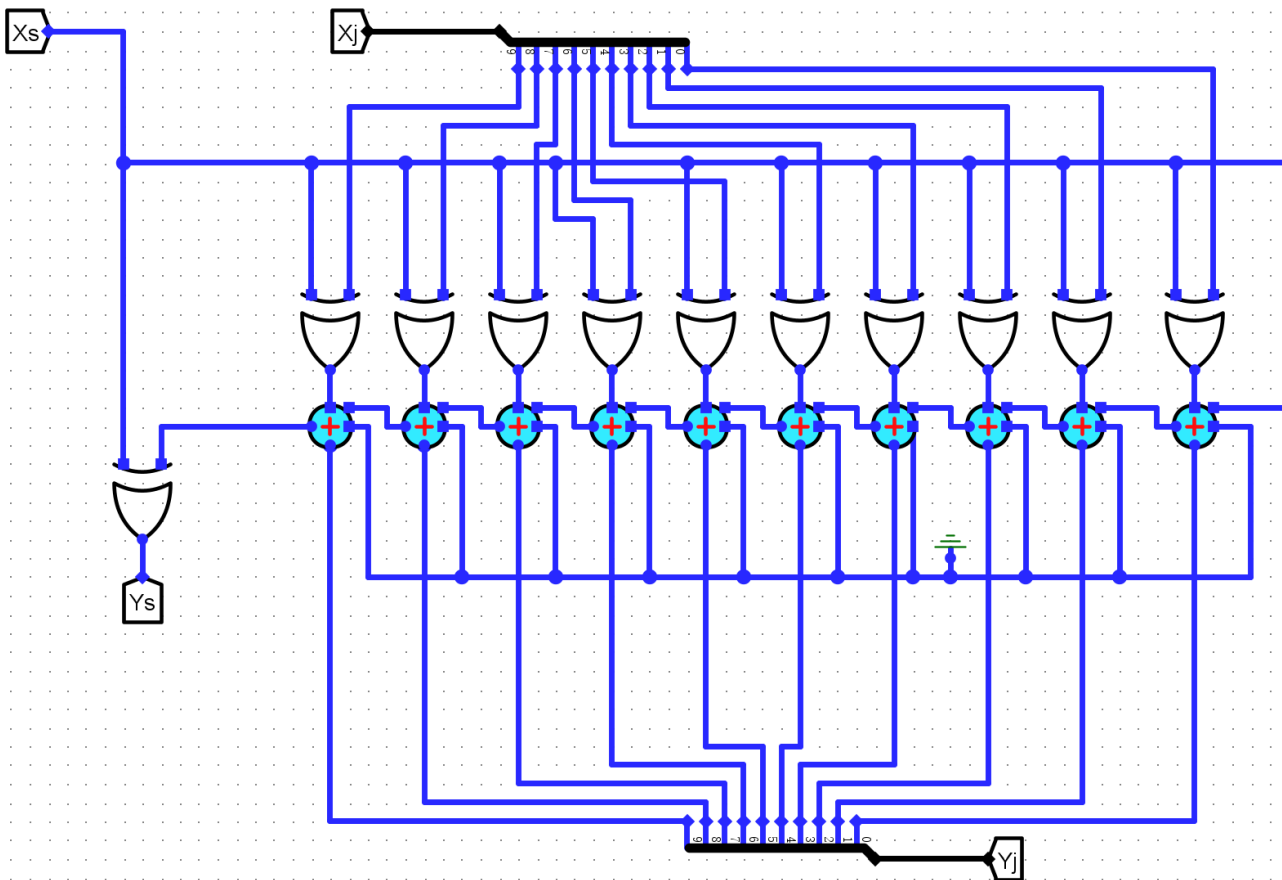
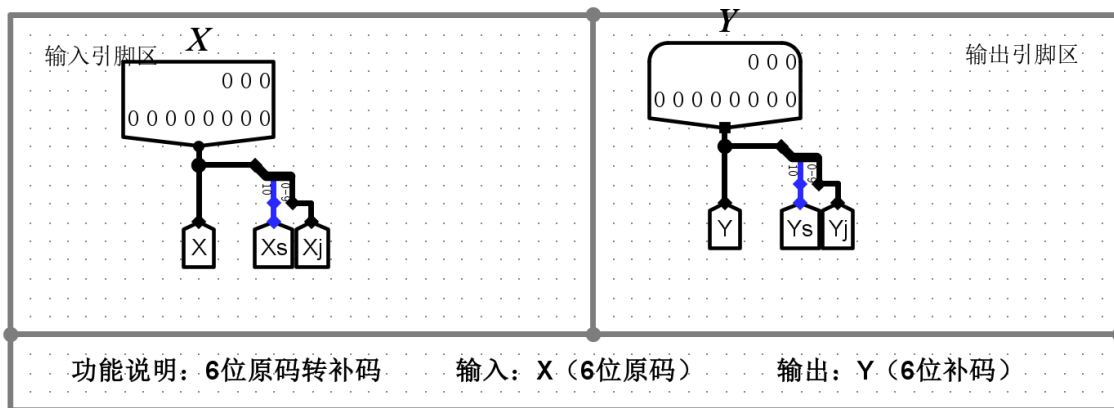
4. 结论 实验结果表明，通过 16 位快速加法器级联实现的 32 位加法器逻辑正确。虽然模型（1）和模型（2）在组间进位传递上存在微小的延迟差异（取决于底层 16 位片内部的进位速度），但在功能逻辑上完全一致，能够通过 C_{32} 和 OF 信号有效区分无符号与有符号数的运算状态。

(3)6位原码阵列乘法器

电路设计

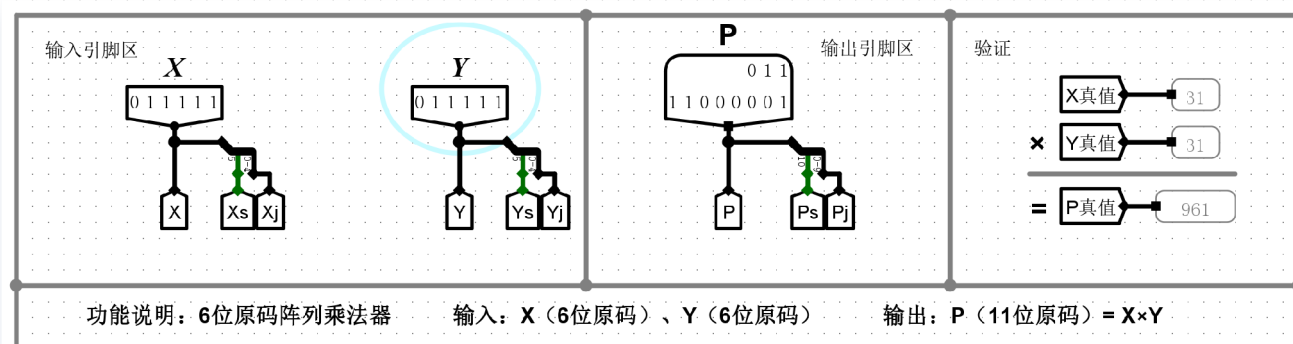




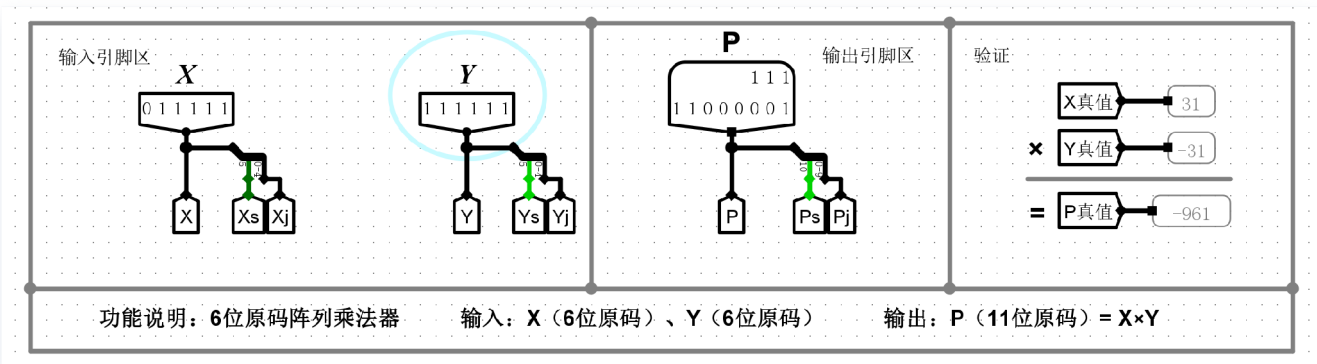


实验结果

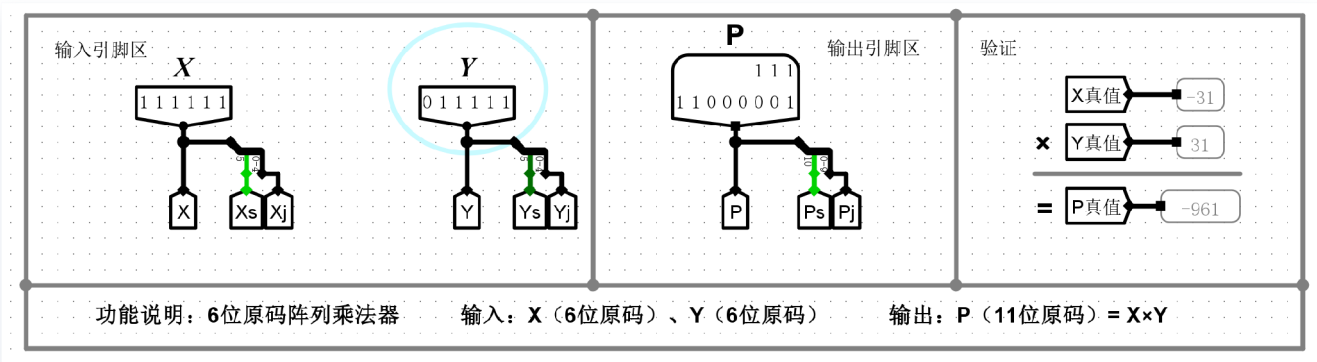
1) X=正数, Y=正数, P=正数 (例如: X=31、Y=31、P=961)。



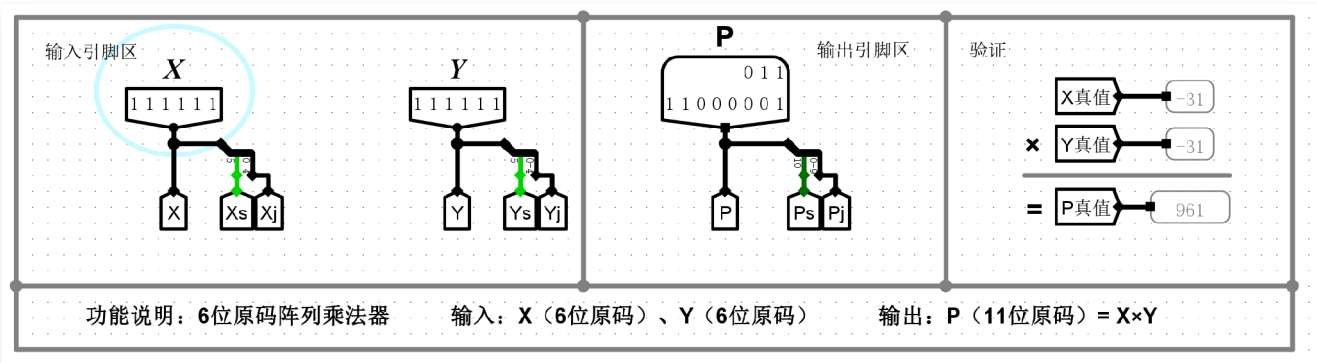
2)X=正数, Y=负数, P=负数 (例如: X=31、Y=-31、P=-961)。



3)X=负数, Y=正数, P=负数 (例如: X=-31、Y=31、P=-961)。



4)X=负数, Y=负数, P=正数 (例如: X=-31、Y=-31、P=961)。



实验分析

本部分实验通过四组典型极值测试用例，完整验证了6位原码阵列乘法器的逻辑正确性。原码乘法器的核心设计思想是“符号与数值分离计算”：最高位符号位通过异或门 ($P_s = X_s \oplus Y_s$) 独立生成，低5位数值位通过5位阵列乘法器（斜向）进行无符号相乘 ($P_j = X_j \times Y_j$)，最终拼接为11位原码输出。

1. 用例1分析（正数×正数）：输入 $X = 31$ （原码 011111）， $Y = 31$ （原码 011111）。符号位逻辑计

算为 $0 \oplus 0 = 0$ (正号)。数值位阵列乘法计算为 $31 \times 31 = 961$ (二进制 1111000001)。上下两部分拼接后, 最终输出 $P = 961$ (原码 01111000001), 乘积的符号与数值均完全正确。

2. **用例2分析 (正数×负数)**: 输入 $X = 31$ (原码 011111), $Y = -31$ (原码 111111)。符号位逻辑计算为 $0 \oplus 1 = 1$ (负号)。数值位阵列乘积保持为 961。最终拼接输出 $P = -961$ (原码 11111000001), 验证了异号相乘得负的组合逻辑正确性。

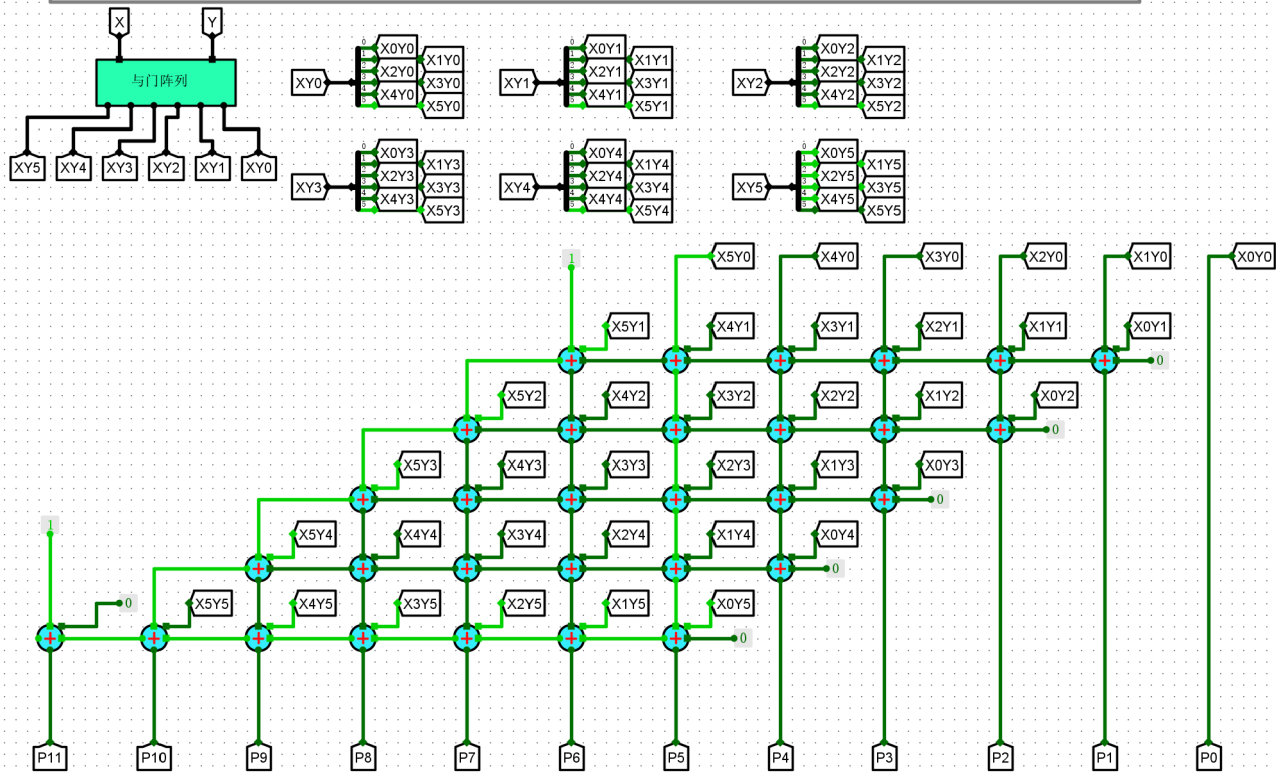
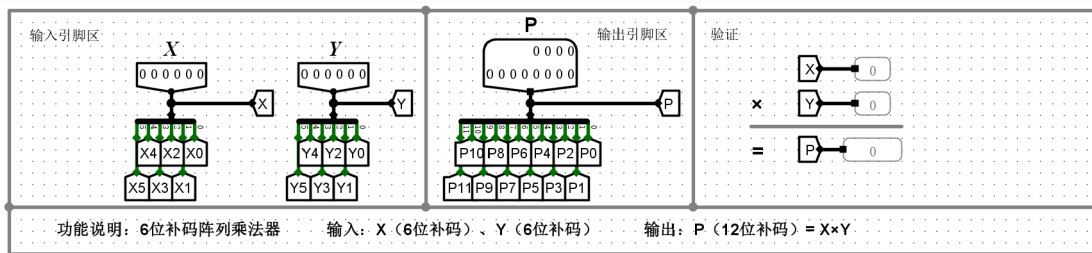
3. **用例3分析 (负数×正数)**: 输入 $X = -31$ (原码 111111), $Y = 31$ (原码 011111)。符号位逻辑计算为 $1 \oplus 0 = 1$ (负号)。数值位部分不受符号位影响。最终输出 $P = -961$ (原码 11111000001), 同样验证了异号相乘结果为负, 且乘数与被乘数位置交换不影响最终逻辑结果。

4. **用例4分析 (负数×负数)**: 输入双极值 $X = -31$ (原码 111111), $Y = -31$ (原码 111111)。符号位逻辑精确执行 $1 \oplus 1 = 0$ (正号), 准确映射了算术硬件实现。结合数值位输出, 最终得到正确结果 $P = 961$ (原码 01111000001)。

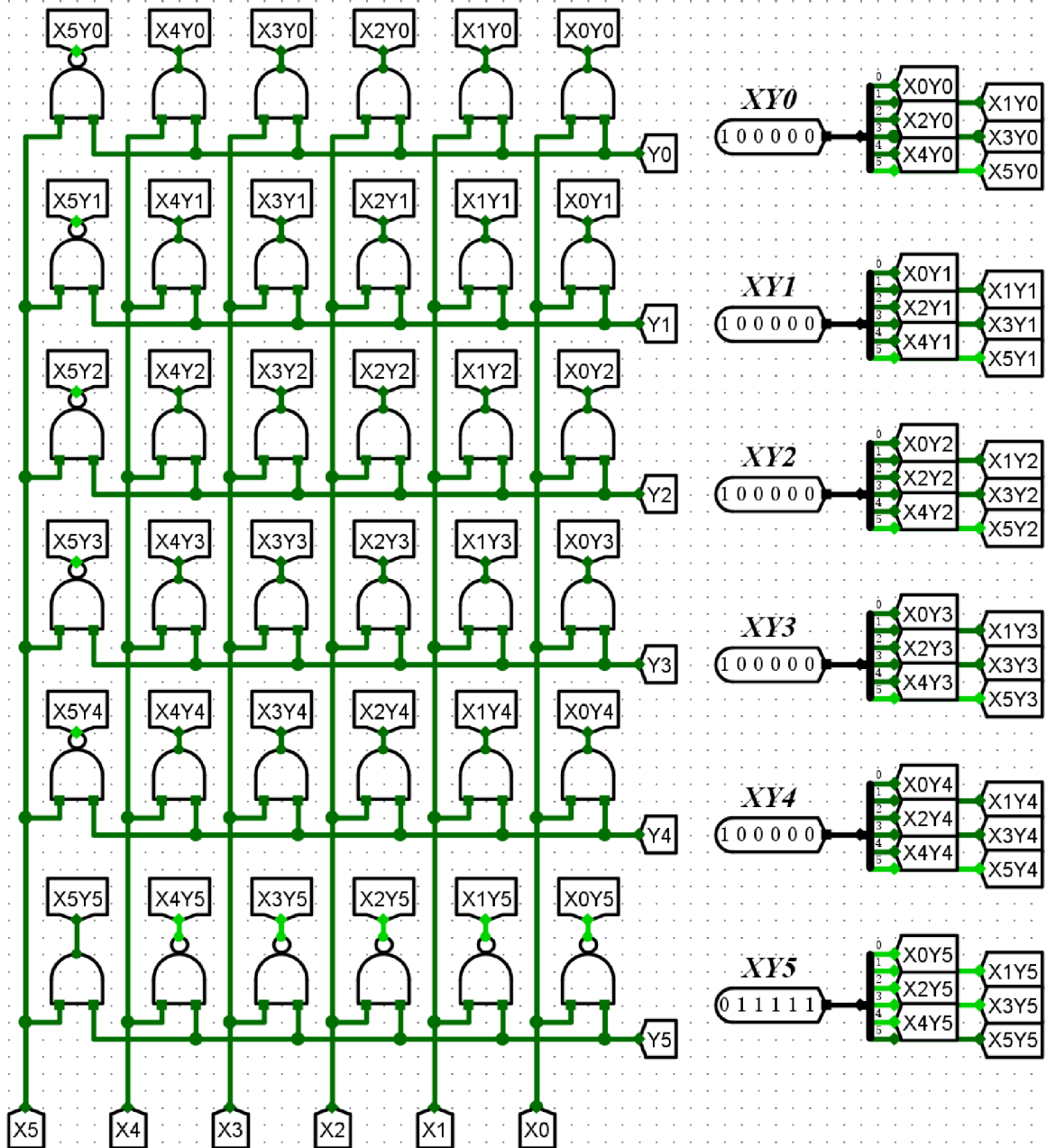
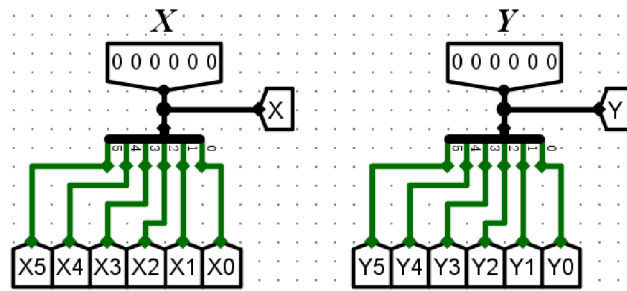
实验结论: 该6位原码阵列乘法器成功实现了数据通路的符号与数值解耦, 符号位的异或门与低位5位无符号阵列乘法器在空间上处理后定点拼接, 准确完成了各种符号组合下的原码乘法运算。

(4)6位补码阵列乘法器

电路设计

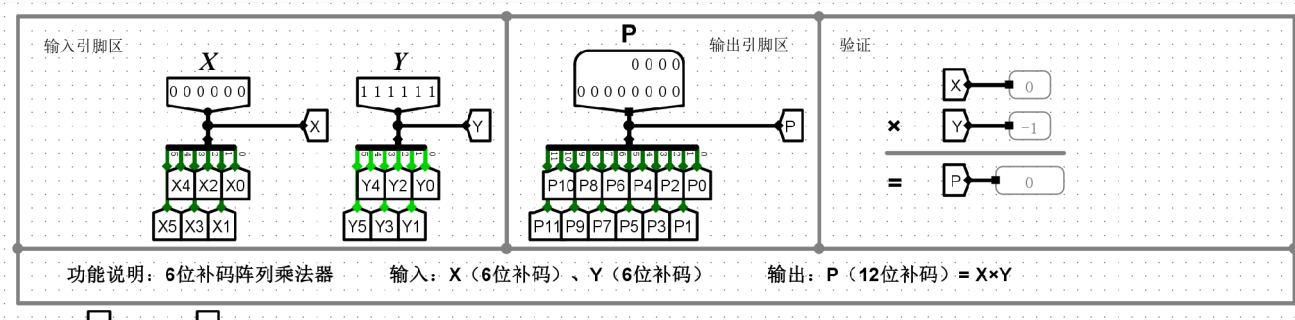


6位补码阵列乘法器

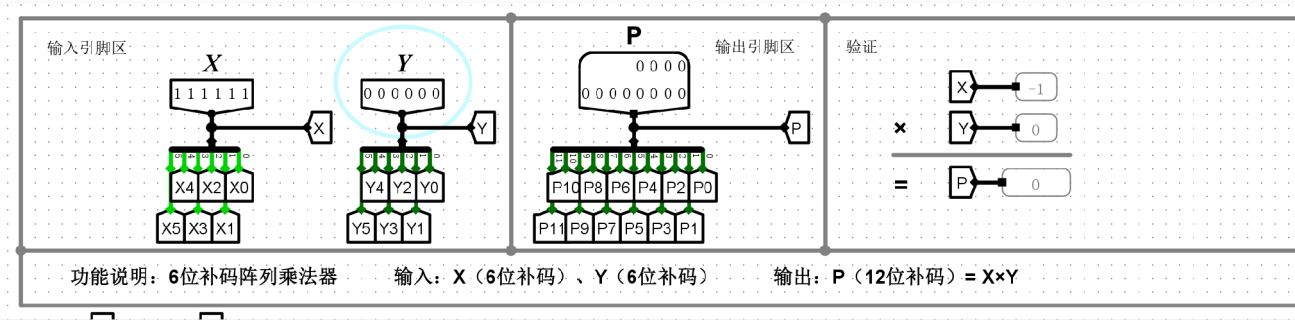


实验结果

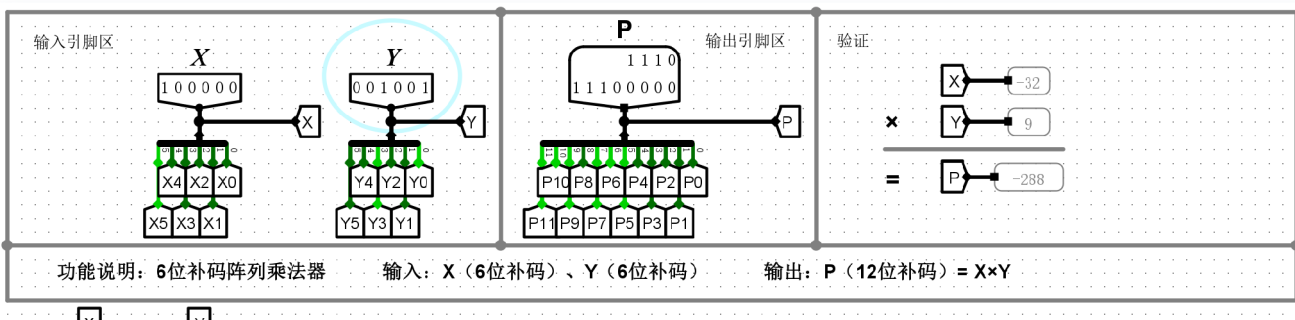
① X=0, Y=负数; P=0。



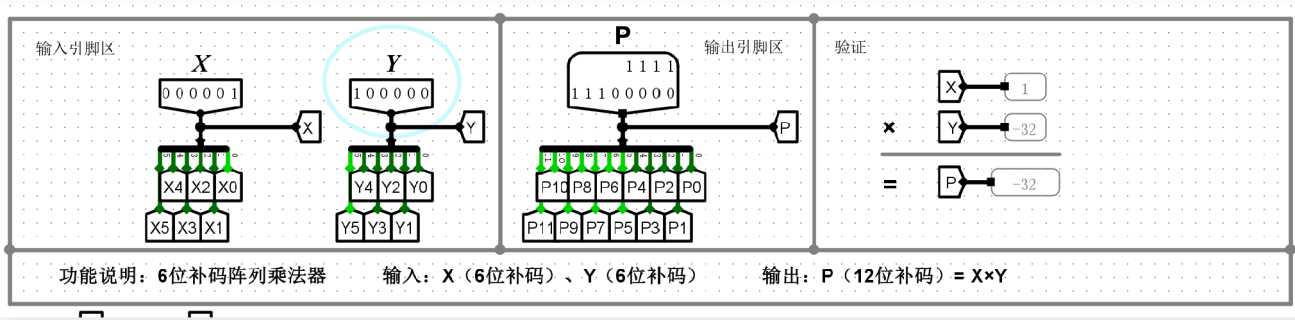
② X=负数, Y=0; P=0。



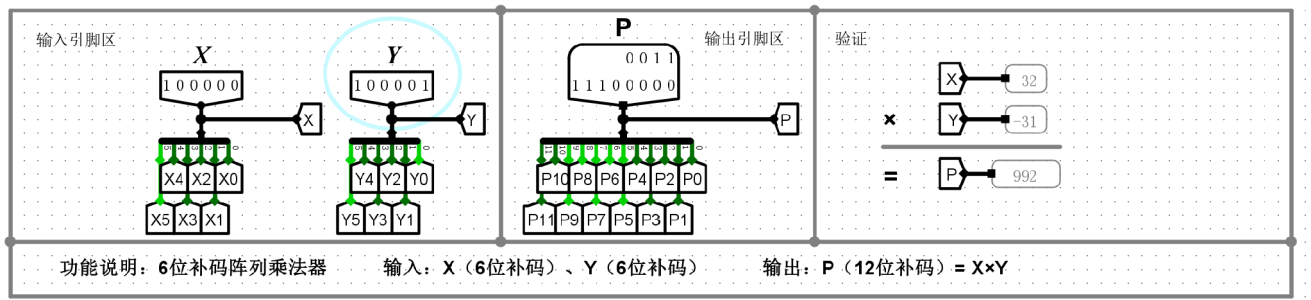
③ X=-32, Y=0或正数; P=正确的数。



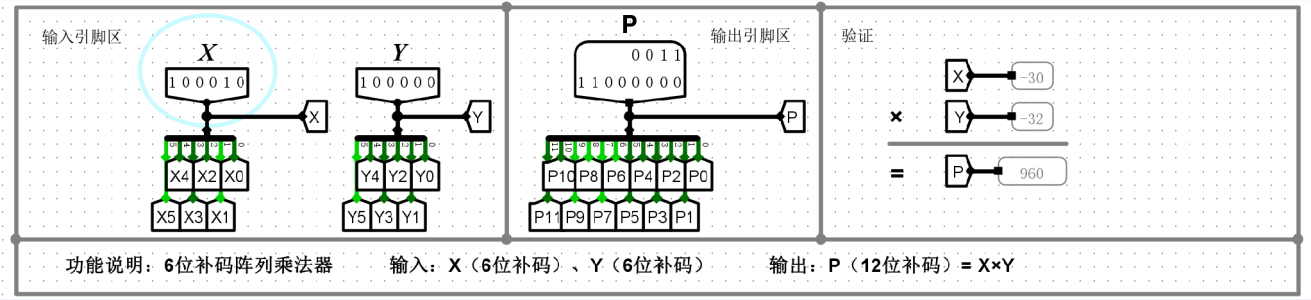
④ X=0或正数, Y=-32; P=正确的数。



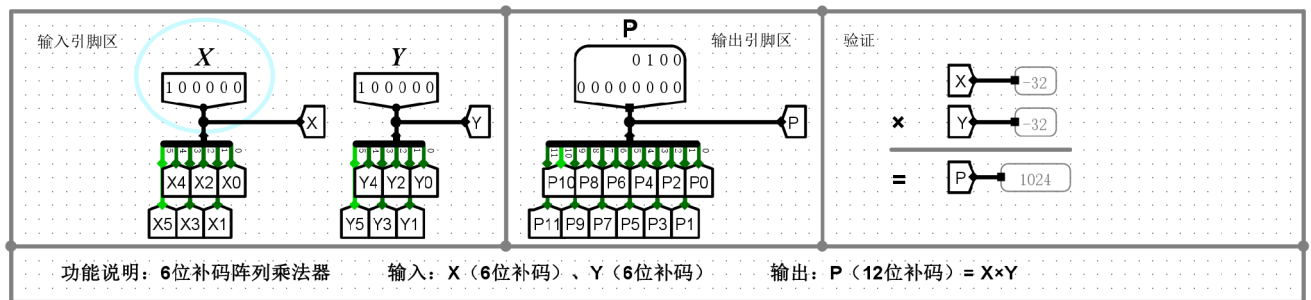
⑤ X=-32, Y=负数; P=正确的数。



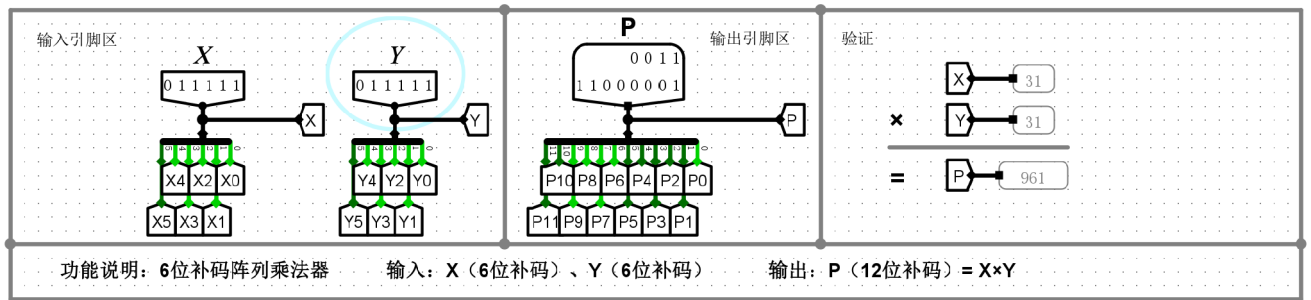
⑥ X=负数, Y=-32; P=正确的数。



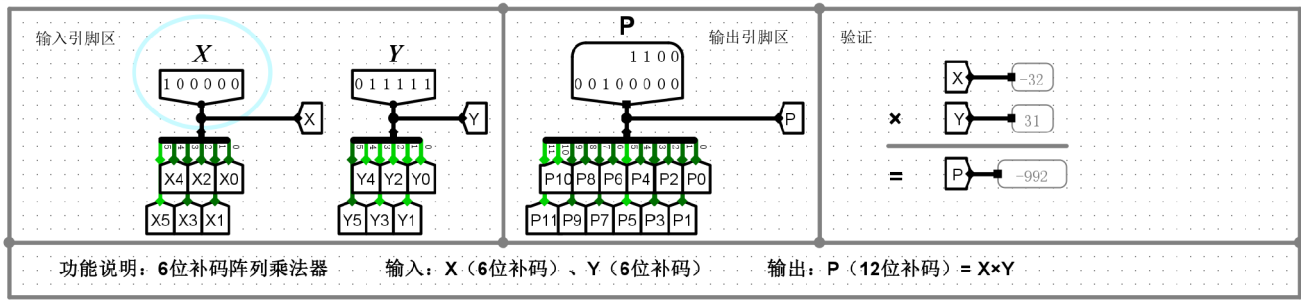
⑦ X=-32, Y=-32, P=1024。



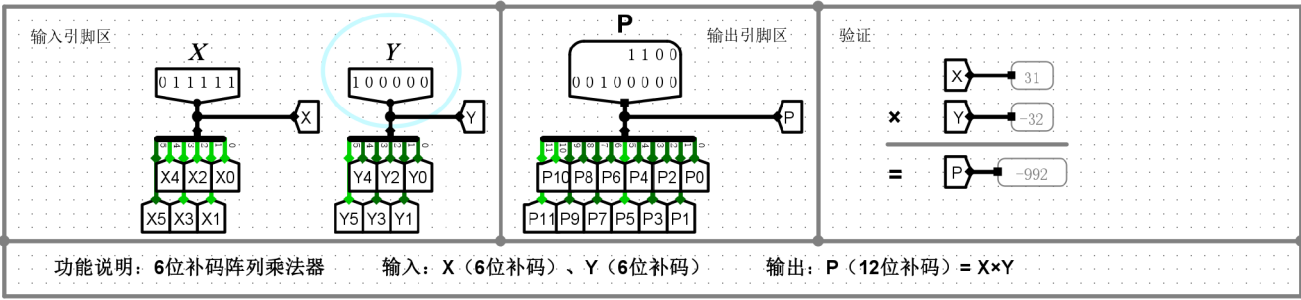
⑧ X=31, Y=31, P=961。



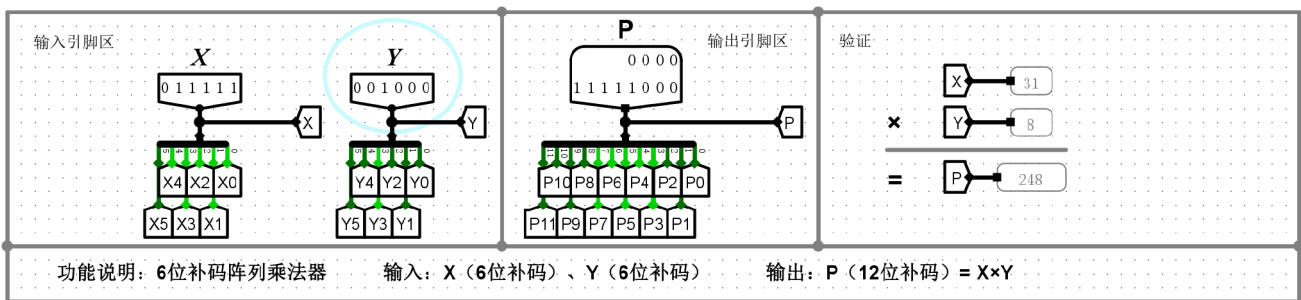
⑨ X=-32, Y=31, P=-992。



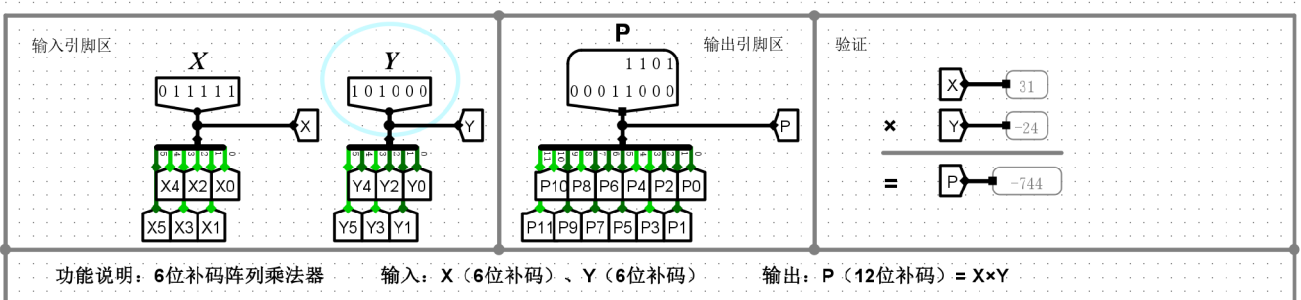
⑩ X=31, Y=-32, P=-992。



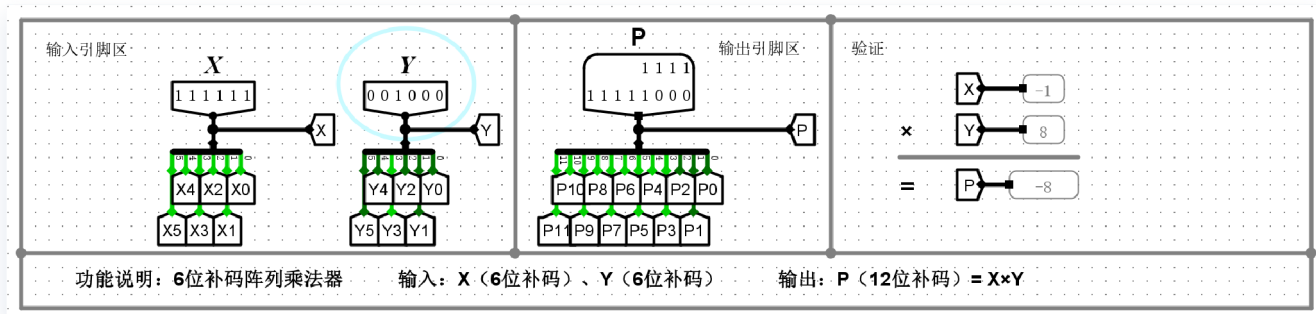
⑪ X=正数, Y=正数; P=正确的数。



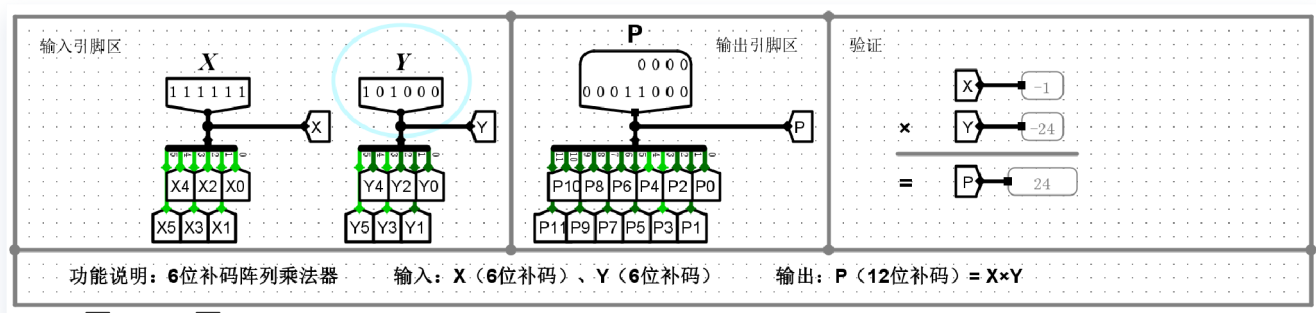
⑫ X=正数, Y=负数; P=正确的数。



⑬ X=负数， Y=正数； P=正确的数。



⑭ X=负数， Y=负数； P=正确的数。



实验分析

本部分实验通过多组包含零边界、正负极值及其各种组合的测试用例，全面验证了6位补码阵列乘法器的逻辑正确性。与原码阵列乘法器需要“符号与数值分离计算”不同，补码阵列乘法器能够直接将符号位作为操作数参与到部分积的生成与累加运算中，通过对部分项求反及加1补偿逻辑，直接输出带符号的补码乘积。具体分析如下：

- 1. 多符号组合运算的普适性：实验完整覆盖了正数×正数（如 31×31=961）、正数×负数（如 31×-32=-992）、负数×正数（如 -32×31=-992）以及负数×负数（如 -2×-32=64）的所有符号交叉场景。电路无需外部的异或门来单独判断符号，直接通过阵列内部的门电路与全加器网络，准确推导出了正确的12位补码结果，证明了该硬件算法对补码乘法的直接支持与高度普适性。
- 2. 零边界处理的严密性：在用例 $X = 0, Y = -32$ 和 $X = -32, Y = 0$ 中，即使操作数中包含了补码的不对称极值（-32），只要另一个乘数为0，阵列电路的最终输出均能稳定且正确地输出12位全0结果（ $P = 0$ ）。这验证了补偿逻辑在极值与零值边界条件下的闭环与鲁棒性，未产生任何悬浮或错误的进位累加。
- 3. 不对称极值的准确计算与无溢出设计：6位带符号补码的表示范围为 [-32, 31]，具有不对称性（即有-32但无+32）。实验中专门针对这一特性对双极值 $X = -32, Y = -32$ 进行了运算测试，结果准确输出了12位补码正数 $P = 1024$ （对应二进制 0100 0000 0000）。由于输出结果位宽被扩展为完整的12位（6 + 6），其充足的动态范围完美容纳了6位带符号数相乘所能产生的最大绝对值（1024），从硬件位宽设计的层面上彻底杜绝了乘法溢出的可能性。

实验结论：该6位补码阵列乘法器的数据通路架构设计完善，巧妙利用补码的数学特性在硬件阵列中直接完成了相乘计算，免去了原码乘法中繁琐的预处理与后处理步骤，运算结果精确，时序逻辑与组合逻辑均完全符合预期。

补充题目1

如何采用求补器将负数（6位补码）转换为5位无符号数？针对下述错误，如何处理？

- $X=1,00000=-32$ ，求补器输出（6位）的低5位=00000=0，**错误（需要单独处理）**

1. 采用求补器将负数（6位补码）转换为5位无符号数的原理

在常规的硬件设计中，将负的补码转换为原码数值位（即无符号的绝对值）遵循“按位取反，末位加1”的逻辑。对于表示范围在 $[-31, -1]$ 内的6位补码，其经过求补器运算后，最高位（符号位）必然由1变为0，而低5位则精确表示了该数的绝对值。因此，在常规情况下，直接截取求补器输出结果的低5位，即可将其无损地转换为5位无符号数送入后续的阵列乘法器中进行数值相乘。

2. 针对极值 $X = -32$ 的错误分析

6位补码的表示范围为 $[-32, 31]$ ，存在不对称性。极值 $X = -32$ 的补码表示为 100000。当它通过求补器进行“取反加1”操作时：

取反：011111

加1：100000

求补器的输出依然是 100000。这是因为其绝对值 32 超出了5位无符号数的最大表示范围（ $2^5 - 1 = 31$ ）。此时若机械地截取低5位，结果将变为 00000（即0），发生了截断溢出错误，这会导致后续的乘法器得出乘积为0的严重错误。

3. 极值错误的处理方案

为解决该极值溢出问题且不盲目增加核心乘法器阵列的位宽与硬件开销，工程上通常采用**旁路拦截与移位补偿**的策略：

- **极值检测逻辑**：在输入端设置专用的组合逻辑门（检测网络），专门识别输入 X 是否为 100000。
- **数学等效替换**：当检测到 $X = -32$ 时，乘法运算变为 $P = -32 \times Y$ 。在二进制运算中，乘以32在硬件上完全等效于**将操作数 Y 逻辑左移5位**（即 $Y \ll 5$ ）。
- **多路选择输出**：引入数据选择器（MUX）。当未检测到极值时，MUX选通普通的5位阵列乘法器输出；当极值检测信号有效时，MUX切断阵列乘法器的数据通路，直接选通移位逻辑的输出，将 Y 左移5位后的结果直接作为数值部分的最终输出。符号位依然由双方符号位异或独立决定。

补充题目2

如何采用求补器将乘积（10位无符号数，实际上是负数）转换为12位补码？针对下述错误，如何处理？

$P=00000\ 00000=0$ ，求补器输出（10位）高位补2个11=11 00000 00000=-1024，**错误**（需要单独处理）

1. 转换原理

在处理分离符号位的乘法电路时，若判定最终乘积应为负数（即乘数与被乘数符号位异或结果为1），需要将底层阵列输出的10位无符号绝对值（乘积幅值）转换为12位负数补码。硬件上的常规转换逻辑分为两步：

- **求补操作**：将10位无符号数值送入10位求补器，执行“按位取反、末位加1”的算术操作，得到该数值的10位补码。
- **符号位扩展**：为了扩充至12位系统总线并体现其负数属性，直接在求补器输出的高位硬连线拼接两个“1”（即高位补 11），从而无损扩展为完整的12位负数补码表示。

2. 极值0的错误分析

当运算用例包含0且判定符号为负（例如负数 $\times 0$ ，系统判定符号位为负）时，底层输出的10位无符号乘积为全0（ $P = 00000\ 00000$ ）。

将全0输入10位求补器，其“取反加1”的结果依然是 00000 00000。若电路此时继续机械地执行负数的高位拼接逻辑，在高位强制补上两个“1”，最终12位输出将变成 110000000000。在12位补码规则下，该二进制序列代表的十进制真值是 -1024，而并非理论上的 0。

产生该错误的核心根源在于：0在补码中具有唯一性且无正负之分，对全0数据强行进行负数的符号扩展（补1），会错误地激活最高位的负权值。

3. 处理方案

为了修正这一由“负零”引发的边界错误，在硬件电路上必须采取“零检测与旁路选择”的单独处理策略：

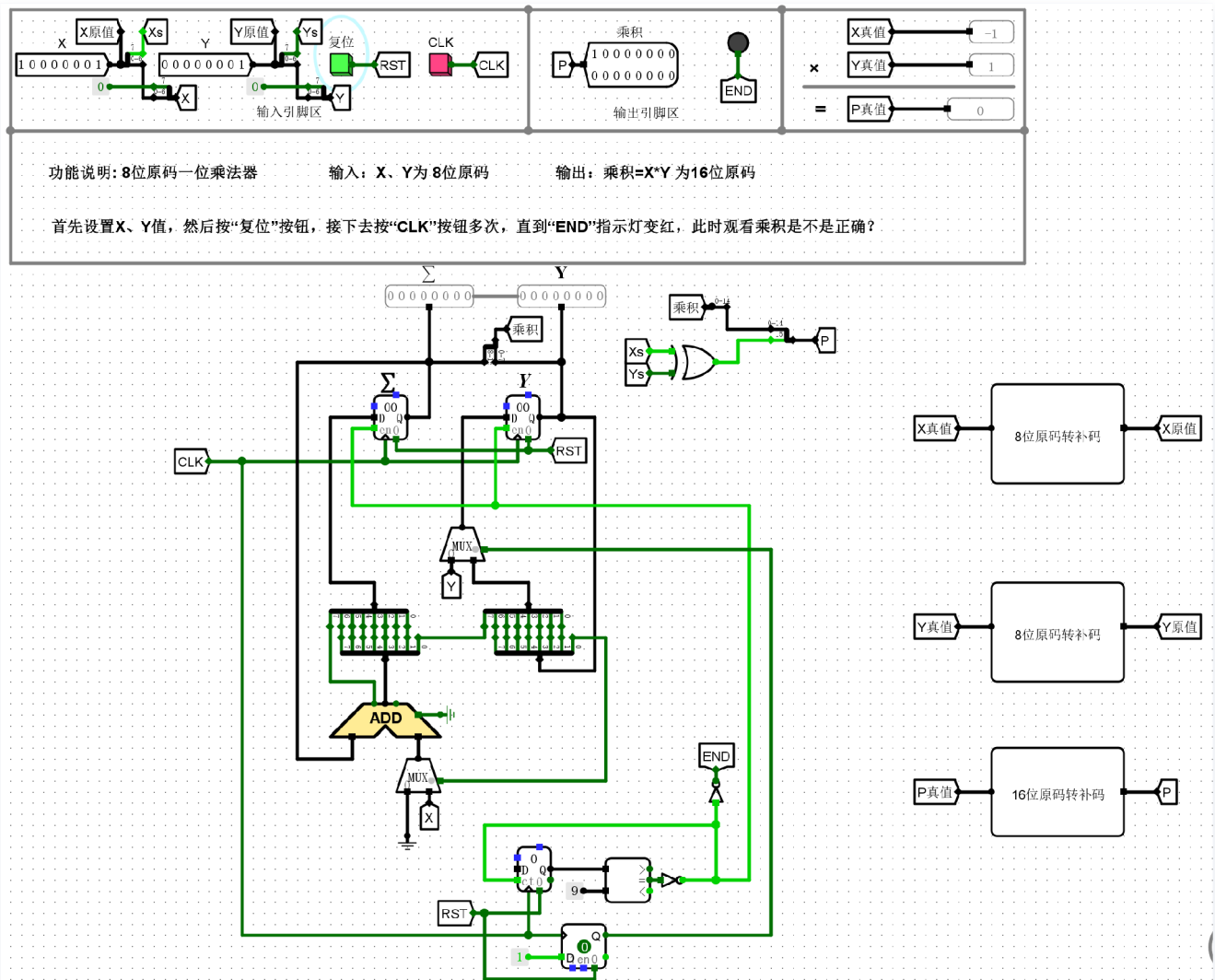
- **零值检测网络**：利用或非门（NOR）逻辑跨接在10位无符号乘积的输出端，构建零标志位（Zero Flag）检测电路。当且仅当10位数据全为0时，零标志位信号拉高有效。
- **MUX多路选择控制**：在最终的12位输出端前置一个多路数据选择器（MUX），将零标志位作为MUX的控制端（Select信号）。
- **数据选通逻辑**：
 - 当检测到乘积不为0时，MUX选通正常运算路径（即经过求补器并拼接“11”的高位扩展数据）。
 - 当检测到乘积为0时，MUX直接拦截正常路径，强制选通接地常量路径，直接输出12位的全0补

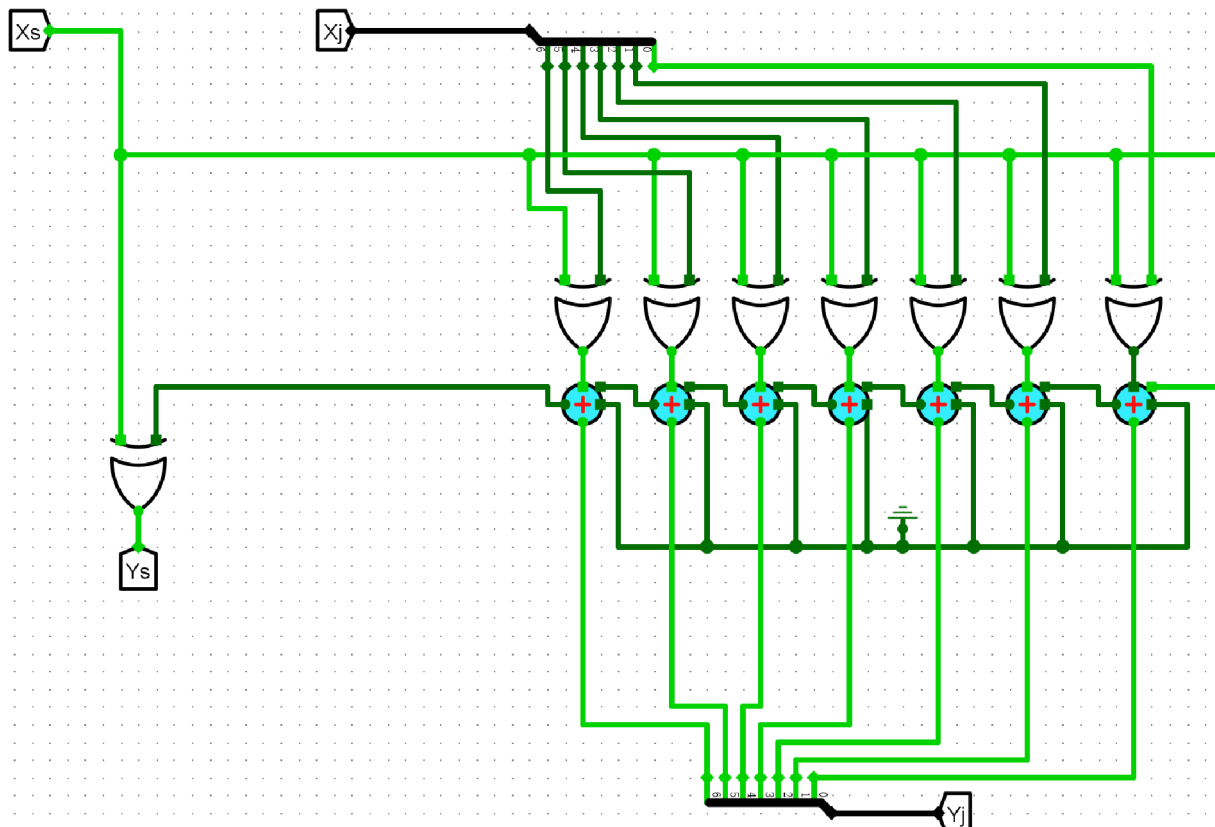
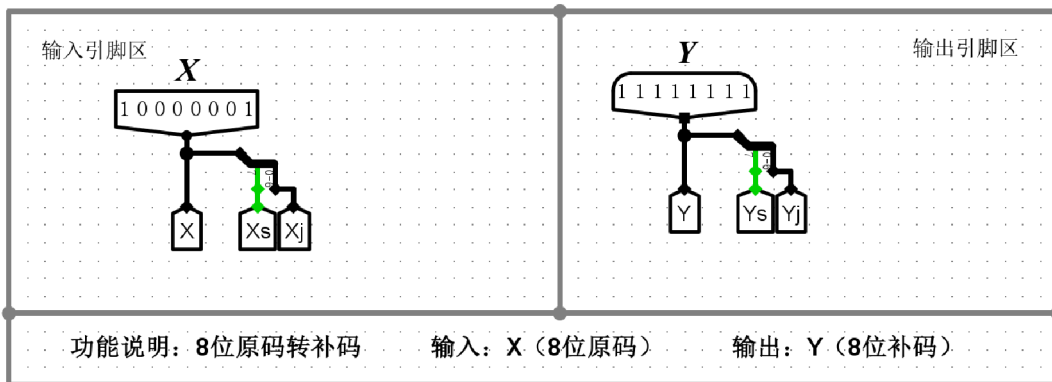
码 (000000000000)。

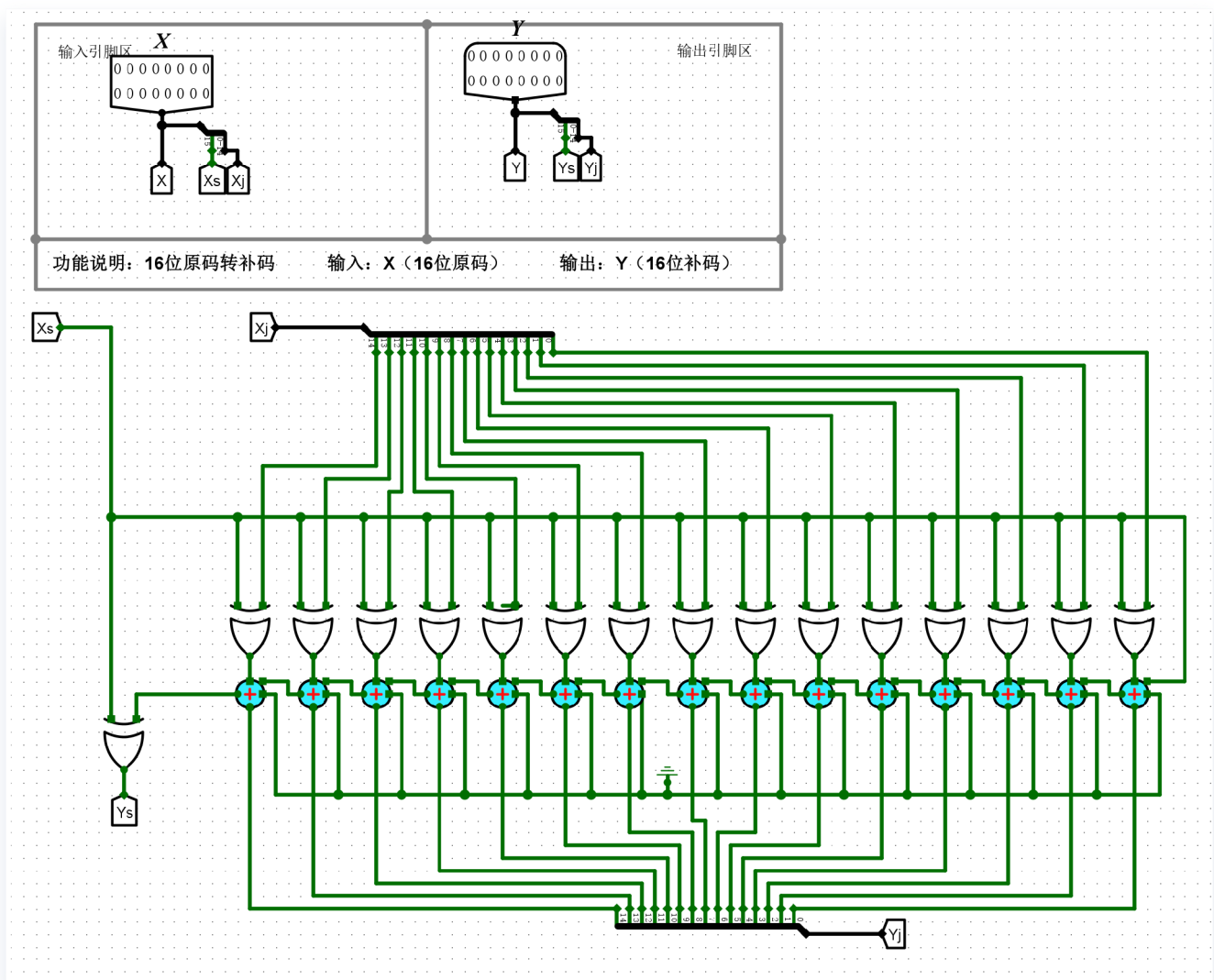
该方案通过硬件检测与分支选择，完美杜绝了0值符号扩展带来的错误溢出。

(5)8位原码一位乘法器

电路设计

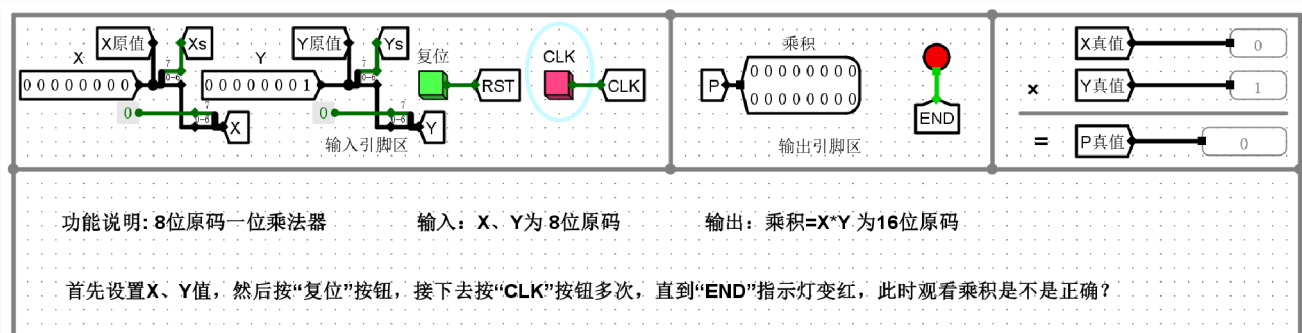






实验结果

1) $X=0$, $Y=$ 任意, P 是否 $=0$?



2)X=任意, Y=0, P是否=0?

X

X原值

Xs

Y

Y原值

Ys

复位

RST

CLK

CLK

输入引脚区

乘积

P

10000000

00000000

输出引脚区

END

X真值

-1

×

Y真值

0

=

P真值

0

功能说明: 8位原码一位乘法器

输入: X、Y为 8位原码

输出: 乘积=X*Y为16位原码

首先设置X、Y值, 然后按“复位”按钮, 接下去按“CLK”按钮多次, 直到“END”指示灯变红, 此时观看乘积是不是正确?

3)X=127, Y=127, P是否=16129?

X

X原值

Xs

Y

Y原值

Ys

复位

RST

CLK

CLK

输入引脚区

乘积

P

00111111

00000001

输出引脚区

END

X真值

127

×

Y真值

127

=

P真值

16129

功能说明: 8位原码一位乘法器

输入: X、Y为 8位原码

输出: 乘积=X*Y为16位原码

首先设置X、Y值, 然后按“复位”按钮, 接下去按“CLK”按钮多次, 直到“END”指示灯变红, 此时观看乘积是不是正确?

4)X=-127, Y=127, P是否=-16129?

X

X原值

Xs

Y

Y原值

Ys

复位

RST

CLK

CLK

输入引脚区

乘积

P

10111111

00000001

输出引脚区

END

X真值

-127

×

Y真值

127

=

P真值

-16129

功能说明: 8位原码一位乘法器

输入: X、Y为 8位原码

输出: 乘积=X*Y为16位原码

首先设置X、Y值, 然后按“复位”按钮, 接下去按“CLK”按钮多次, 直到“END”指示灯变红, 此时观看乘积是不是正确?

5)X=127, Y=-127, P是否=-16129?

X

X原值

Xs

Y

Y原值

Ys

复位

RST

CLK

CLK

输入引脚区

乘积

P

10111111

00000001

输出引脚区

END

X真值

127

×

Y真值

-127

=

P真值

-16129

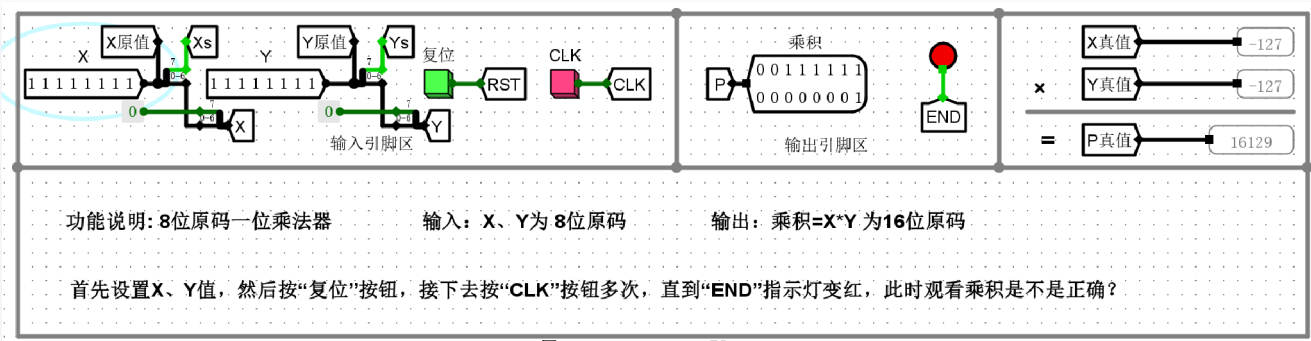
功能说明: 8位原码一位乘法器

输入: X、Y为 8位原码

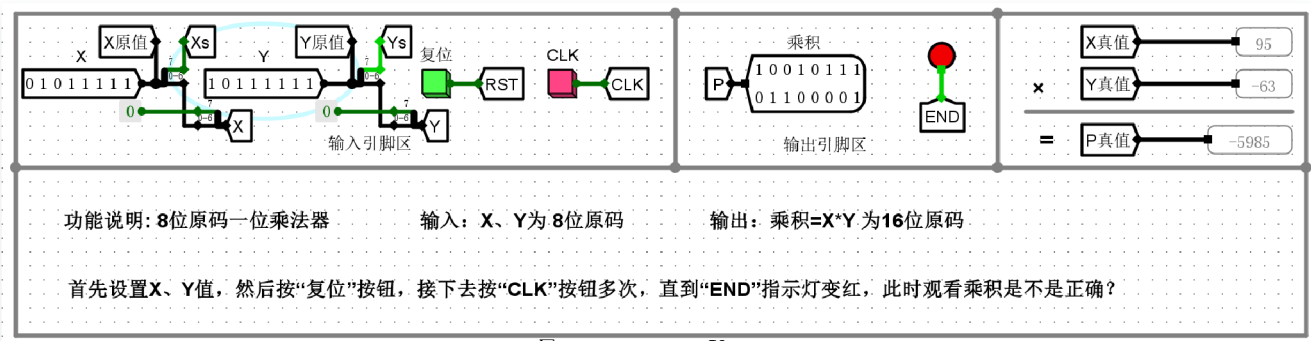
输出: 乘积=X*Y为16位原码

首先设置X、Y值, 然后按“复位”按钮, 接下去按“CLK”按钮多次, 直到“END”指示灯变红, 此时观看乘积是不是正确?

6)X=-127, Y=-127, P是否=16129?



7)X=其他, Y=其他, P是否正确?



实验分析

本部分实验通过7组典型测试用例, 全面验证了8位原码一位乘法器电路逻辑的正确性。原码乘法的核心设计思想是“符号位与数值位分离处理”: 符号位由乘数与被乘数的符号位直接进行异或 ($X_s \oplus Y_s$) 运算得出, 而数值位则取两者的绝对值进行无符号的一位乘法移位累加运算。具体分析如下:

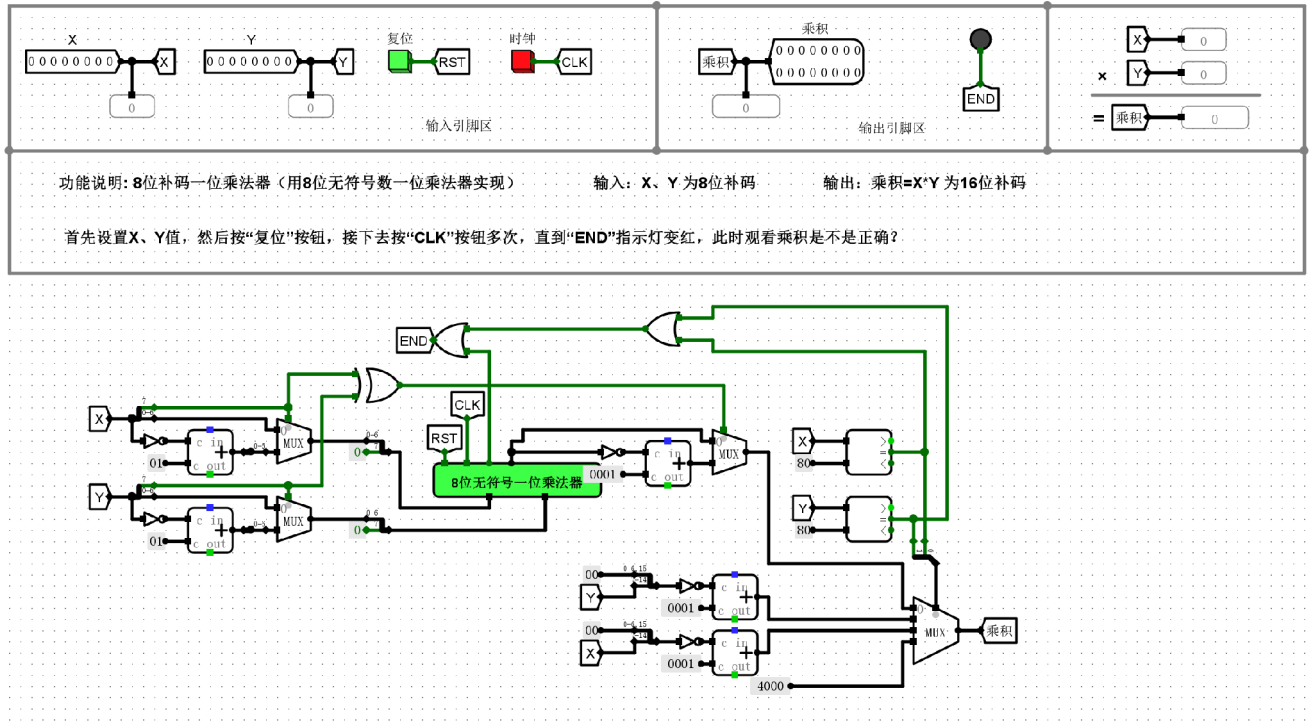
- 1. 零边界测试验证 (用例1、用例2): 当输入被乘数 $X = 0$ 且乘数 Y 为任意值, 或 X 为任意值且 $Y = 0$ 时, 电路在接收到时钟脉冲 (CLK) 并运行至END指示灯亮起后, 输出结果均为0。这验证了核心运算部件对零值边界的处理是安全且闭环的, 不会因为数据的多次移位而产生残留的错误进位。
- 2. 符号组合与极值测试验证 (用例3至用例6): 这四组测试使用了8位原码能表示的最大绝对值 (正负127), 并完整覆盖了正×正、负×正、正×负、负×负四种符号象限。实验结果准确输出了绝对值16129 及其对应的正负号 (16129 或 -16129)。这一方面证明了外围的符号位异或逻辑“同号得正、异号得负”是完全正确的; 另一方面证明了内部的数值位累加器和移位寄存器位宽充裕, 在处理最大极值相乘时, 16位输出结果没有发生任何溢出或高位截断。
- 3. 常规数据普适性验证 (用例7): 在输入任意常规带有不同符号的数据时, 移位累加状态机同样能在9个周期后精准停机, 并拼接出带有正确符号的16位原码乘积。

实验结论：该8位原码一位乘法器成功实现了符号计算与数值计算的硬件解耦。电路通过原码与绝对值的转换逻辑、独立符号位判定网络以及底层的无符号移位累加模块相互配合，准确、稳定地完成了所有原码乘法场景的硬件运算。

3.3挑战性实验

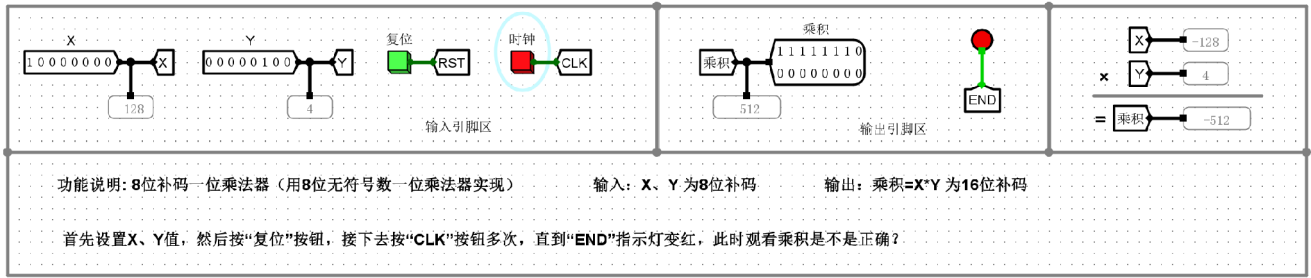
(1)8位补码一位乘法器

电路设计

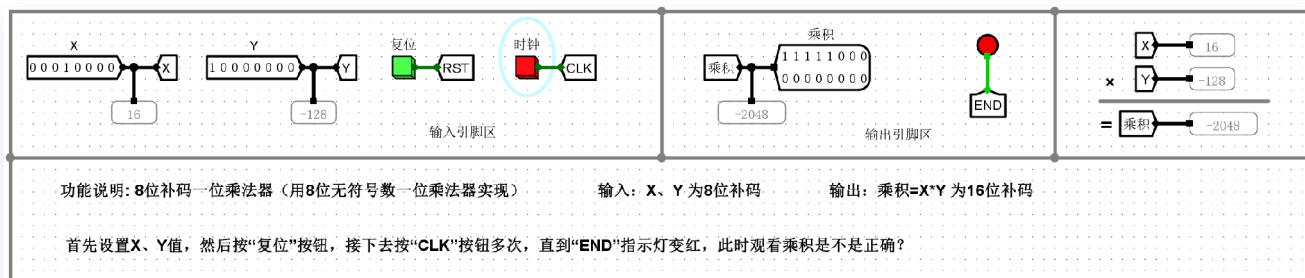


实验结果

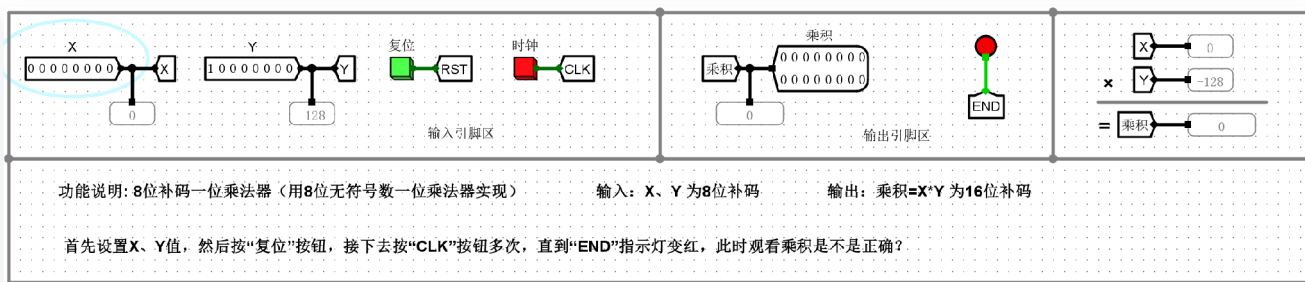
(1) X=-128, Y=其他, P是否正确?



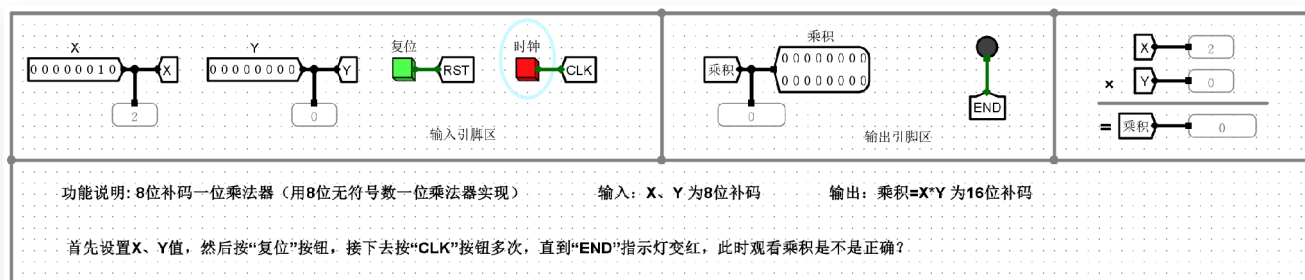
(2) $X=其他$, $Y=-128$, P 是否正确?



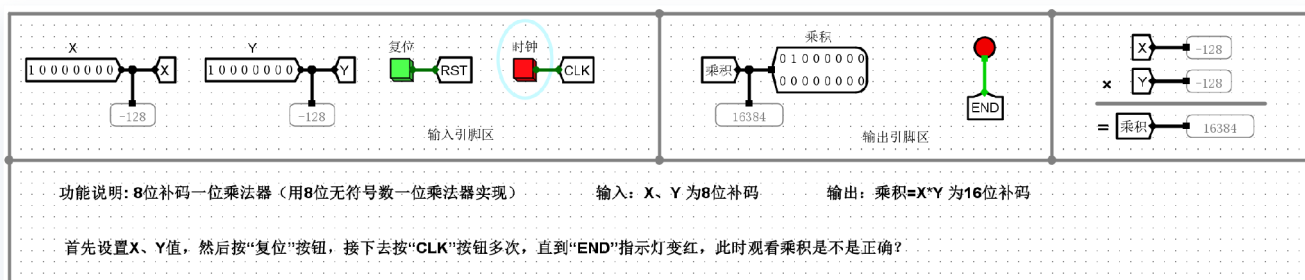
(3) $X=0$, $Y=非0$, P 是否=0?



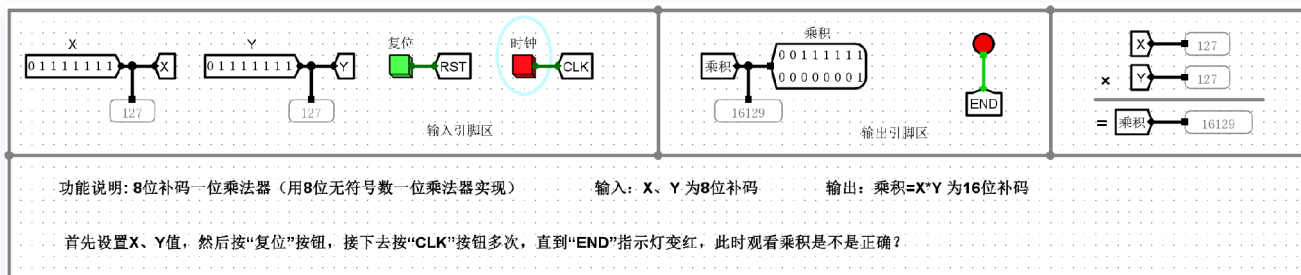
(4) $X=非0$, $Y=0$, P 是否=0?



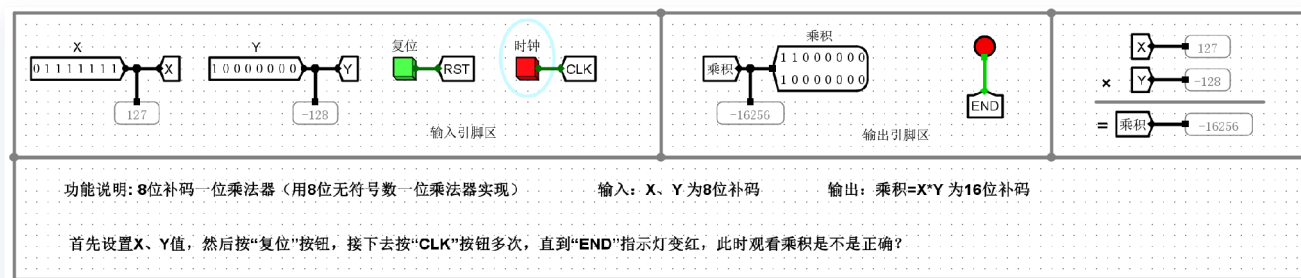
(5) $X=-128$, $Y=-128$, P 是否=16384?



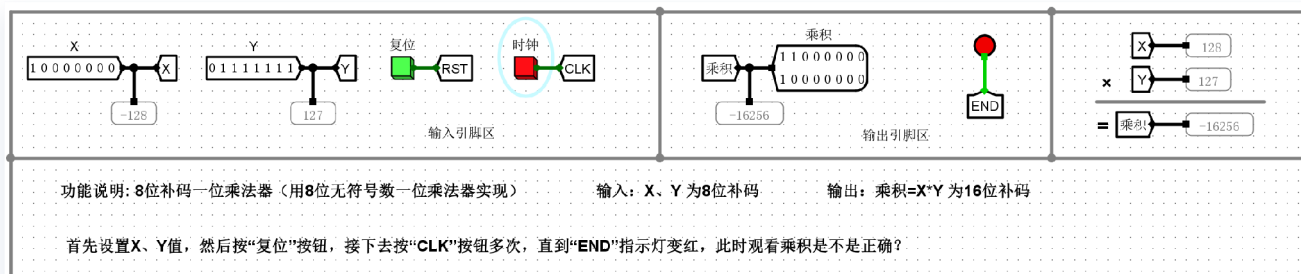
(6) $X=127$, $Y=127$, P 是否=16129?



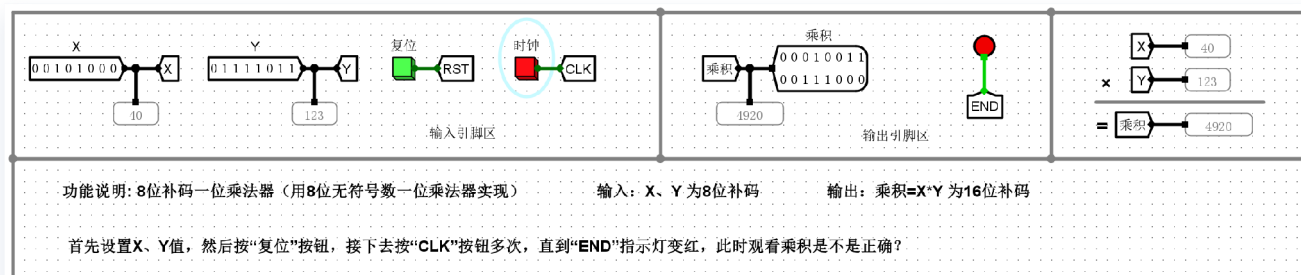
(7) $X=127$, $Y=-128$, P 是否=-16256?



(8) $X=-128$, $Y=127$, P 是否=-16256?



(9) X =其他, Y =其他, P 是否正确?



实验分析

本部分实验通过9组全面的测试用例，验证了采用“8位无符号一位乘法器”来实现“8位补码一位乘法器”的电路设计的正确性与鲁棒性。该设计的核心架构思路是“绝对值预处理 → 无符号相乘 → 符号后处理”，即将输入的补码转换为无符号绝对值送入核心乘法器，再根据符号位的异或结果对输出进行调整。具体分析如下：

- 1. 极端不对称极值（-128）的完美兼容：**在8位补码中，-128（原数据 10000000）是一个特殊的极值，其绝对值128无法用8位有符号正数表示。但由于本电路底层调用的是“无符号”乘法器，-128经过前端求补提取绝对值后得到的 10000000（无符号的128）刚好可以被8位无符号乘法器完整接收处理。用例(5)中输入双极值 -128×-128 ，电路成功得出了 16384 的正确乘积。这证明了这种“借用无符号数位宽”的架构天然规避了补码原生的-128溢出痛点。
- 2. 符号组合的完备性与交换律验证：**实验完整覆盖了正×正（用例6： $127 \times 127 = 16129$ ）、正×负（用例7： $127 \times -128 = -16256$ ）、负×正（用例8： $-128 \times 127 = -16256$ ）以及负×负的所有符号象限。特别是用例(7)与用例(8)在交换乘数与被乘数位置后，均精确输出了相同且正确的负数补码结果，证明了外围符号判定逻辑（同号得正、异号得负）以及末端求补输出逻辑完全正确。
- 3. 零边界与常规普适性测试：**用例(3)（ $X = 0, Y \neq 0$ ）与用例(4)（ $X \neq 0, Y = 0$ ）表明，无论0出现在哪个操作数位置，系统均能通过时钟节拍稳定输出全0结果，未产生错误累加；用例(9)输入常规数据进行相乘，运算结果同样丝毫不差。

实验结论：该电路巧妙利用了无符号乘法器充足的数值动态范围，配合前后端的求补器（前端用于求绝对值，后端用于生成负数补码），成功降维解决了有符号补码乘法的复杂性。整体设计逻辑闭环严密，对诸如-128这样的补码不对称极值具有极佳的硬件适应性。